



BOOKSHELF / DEVELOPER STUDY EDITION

Understand your website code.

Today's changes, the logic behind them,
and a complete notification walkthrough.

Architecture and database design

Packages, Python, SQL, JavaScript and CSS

Worked examples and line-numbered source

Review findings and future improvements

SESSION SNAPSHOT: 77da6f7

Prepared 9 September 2026 | Main website emphasis

Contents and reading routes

Read chapters 1-6 for the foundations, chapters 7-14 for communications, and the remaining chapters for PYQs, resources, badges, presence, interface changes, and maintenance. The source atlas follows the teaching chapters. Its file sections are also PDF bookmarks.

Contents and reading routes	2
01 How to use this guide	5
02 What changed and why	6
03 The architecture: where each responsibility lives	7
04 The database model, explained	9
05 Python packages: what pip installs and why	11
06 Reading the syntax without guessing	13
07 Authentication and permission boundaries	15
08 Announcement logic: one editable official bar	16
09 Sending notifications: the complete journey	17
10 Why retrying a send does not create another message	19
11 Recipient search and selection in the browser	20
12 Student inbox queries and pagination	21
13 Bell, unread counter, and read acknowledgements	22
14 Notification worked examples and failure cases	23
15 PYQs: metadata, hashes, upload and editing	24
16 Ordinary resources and dashboard filters	25
17 Trusted authors and profile badges	26
18 Recent presence: what the active count really means	27

19 Dashboard layout fixes and the admin redesign	28
20 The standalone bulk uploader, in context	29
21 Animation refinements included in the commit	30
22 Testing, review findings, and safe maintenance	31
23 Learning exercises with answers	32
24 Glossary and reference shelf	33
Source atlas - the changed code, with reading notes	34
A01 communications_service.py	35
A02 communications_routes.py	49
A03 static/notifications.js	61
A04 static/admin-communications.js	65
A05 presence_service.py	69
A06 static/student-activity.js	70
A07 static/admin-presence.js	71
A08 pyq_service.py	73
A09 app.py	77
A10 database.py	84
A11 db_agent.py	85
A12 admin_user_service.py	87
A13 storage_agent.py	88
A14 templates/admin-communications.html	89
A15 templates/dashboard.html	91
A16 static/dashboard.js	97
A17 templates/_student_activity.html	101
A18 templates/_contributor_badge.html	102
A19 static/contributor-badges.css	103

A20 static/notifications.css	104
A21 templates/admin-dashboard.html	105
A22 static/admin-dashboard.css	110
A23 templates/admin-pyqs.html	122
A24 static/admin-pyqs.css	125
A25 static/admin-communications.css	127
A26 templates/resource-catalogue.html	128
A27 static/resource-catalogue.css	130
A28 templates/pyqs.html	131
A29 static/pyqs.css	133
A30 static/dashboard.css	134
A31 templates/contribute.html	137
A32 static/script.js	138
A33 static/grid-bg.js	141
A34 static/page-wheel.js	144
A35 static/page2-narrative.js	147
A36 static/three.js	148
A37 .gitignore	149
A38 COMMUNICATIONS.md	150
A39 static/images/ce-icon.svg	152
A40 static/images/ece-icon.svg	153
A41 static/images/ee-icon.svg	154
A42 static/images/me-icon.svg	155

Verification record and source coverage

156

01 How to use this guide

This is a learning guide to the Bookshelf website work discussed in this conversation and captured in commit 77da6f7. That commit is dated 9 September 2026, 00:44 IST; the conversation spans the previous evening and the period after midnight. "Today" here means that development session, rather than a claim that every older feature was first written on one calendar date. The snapshot is the main website code at the time of this document. Actual database contents and hosting settings were not queried to write it.

Read the architecture and package chapters first. Then follow the notification chapters in order. The code atlas at the end reproduces the changed source with original file line numbers, and provides reading notes. New notification files are shown in full; for existing files the atlas shows the changed regions and nearby context. Blank lines and closing punctuation have no independent business behaviour, but their role in grouping code is explained in the syntax chapter.

The guide explains the implementation that exists, including its limitations. It does not silently turn proposed improvements into completed features. In particular, in-site notifications exist; email, operating-system push, WebSockets, and a production job queue do not. The upload manager remains a separate local tool. This document does not change application code, send messages, upload PDFs, or migrate the database.

There are three useful reading passes. First, understand what travels between browser, server, database, and Azure. Second, trace one concrete example through the routes and SQL. Third, use the file names and line numbers to compare the explanations with your editor. If a line number later changes, search for the function name or CSS selector instead.

02 What changed and why

The initial question was where thousands of previous-year question papers should live. The design separates the physical PDF from its searchable metadata: Azure stores the file; PostgreSQL stores subject code, subject name, year, file identity, and URL. A dedicated pyqs table expresses the fact that these papers have years and subject codes but do not belong to a student's semester selection. It also keeps the existing resource table useful for notes, assignments, and digital books.

The website gained a shared PYQ archive, a separate admin list, single-PDF upload, and editable subject names. The bulk importer was deliberately kept outside the website's import/configuration chain. Both tools can operate on the same PostgreSQL table and Azure container without importing each other's Python modules. Sharing data is different from linking application code.

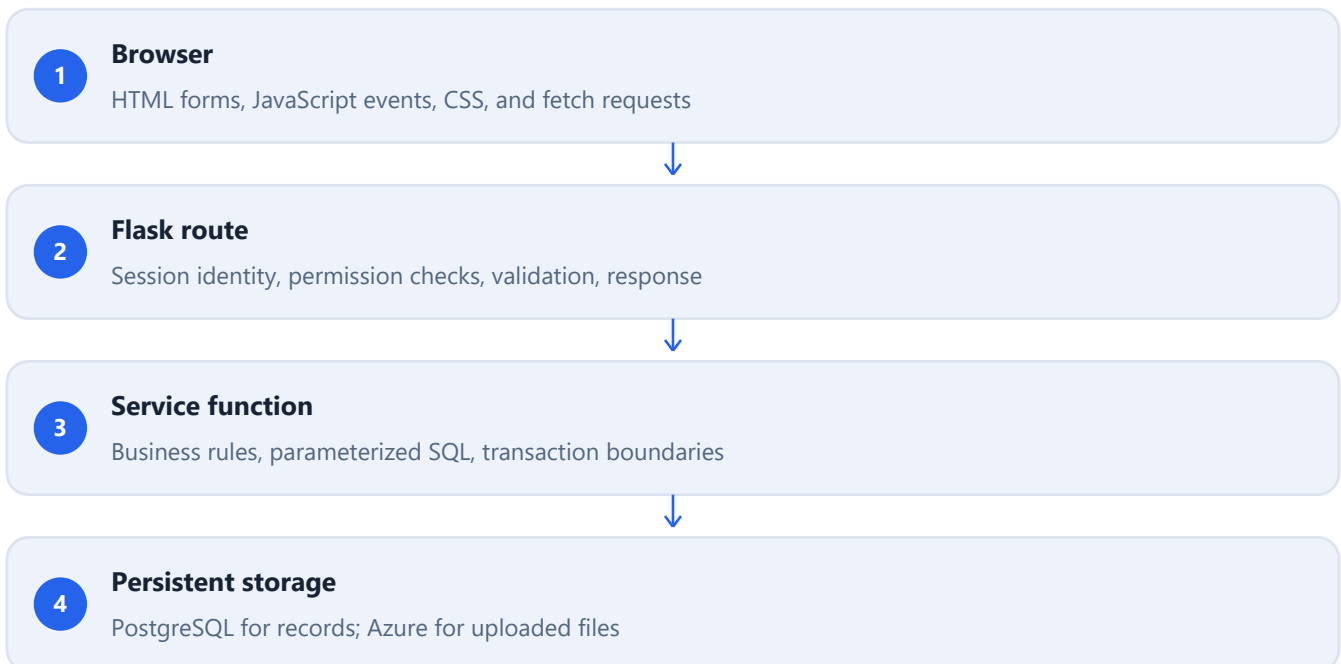
The notes, assignments, and digital-books sections now query verified resources. Department and semester selection become URL filters, so the correct library is opened without hard-coded lists. The uploader is presented as Admin or the contributor's display name, with a User fallback. Admin uploads have an official badge. Administrators can assign Trusted or Best Contributor to students; a shared rendering macro prevents badge markup from diverging across pages.

The dashboard received matching line icons for Electronics, Electrical, Mechanical, and Civil. Department labels align because every text block fills the card width and every heading reserves two lines. The mobile navigation uses five compact slots with a shorter visible PYQs label. Selecting a department reveals and scrolls to semesters; selecting a semester reveals categories. Changes reset stale selections and account for reduced-motion preferences.

The former PYQ promotional bar became an editable official announcement. A new admin communications page manages this banner and sends separate private notifications to all eligible students or a selected group. The student dashboard has a bell, unread count, inbox, profile photo, and awarded profile badge. A later visual refinement makes the profile Best Contributor badge violet while the resource badge keeps its subdued bronze appearance.

Recent student presence is now visible to administrators: a prominent active count and a list that can be hidden without stopping count updates. Finally, the admin pages were restyled with charcoal navigation, cool neutral backgrounds, white panels, blue actions, larger text, and responsive student cards. The latest commit also contains landing-page animation scheduling refinements; these are documented separately because they are not notification logic.

03 The architecture: where each responsibility lives



Browser -> Flask route -> service function -> PostgreSQL is the normal metadata path. Browser -> Flask upload route -> Azure is the file path. Flask then saves the returned Azure URL in PostgreSQL. When someone opens a resource link, the browser requests that URL; the Python server is not streaming every document through the dashboard route.

app.py creates the Flask application, reads configuration, starts OAuth, registers routes, checks permissions, and chooses HTML or JSON responses. database.py owns user/profile persistence and creates the base tables. db_agent.py owns ordinary resources. pyq_service.py owns the website's PYQ catalogue operations. admin_user_service.py owns moderation and awarded badges. communications_service.py owns announcements, notification membership, and read status. communications_routes.py connects that service to HTTP. presence_service.py records and queries recent activity.

templates/ contains Jinja HTML. Jinja executes on the server before the browser receives the page. static/*.js executes in the browser afterward. CSS controls appearance, not permission. Hiding a button is useful interface behaviour, but only a server-side check can prevent an unauthorized request.

The service/route boundary is intentional. A service can validate data and execute SQL without knowing whether its caller is an HTML form or another Python function. A route knows about request.form, session, redirects, and status codes. This separation lets tests replace database calls while exercising real Flask access checks.

There is one configured PostgreSQL database in the website connection code, with several tables. A Python module named database.py is not a separate database. database.py and db_agent.py each construct connections using the same DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, and DB_NAME environment variable names. The actual values determine the server and database; this guide does not disclose them.

The runtime does not use SQLite for website notifications. SQLite belongs to the standalone import manager's local job journal. Azure containers are object-storage groupings, not SQL tables. The current website mapping sends PYQs to pyq, assignments to assignments, and both notes and digital books to notes. The category field in PostgreSQL distinguishes a digital book from a note even when their blobs share a container.

04 The database model, explained

Data	Parent / relation	Why it exists
users	Identity root	Account, profile, moderation, badge and recent presence
student_notifications	One row per send	Message content and retry identity
student_notification_recipients	notification_id + user_id	Delivery membership and private read timestamp
official_announcement	Singleton id = 1	One editable dashboard banner
resources / pyqs	Blob URL points to Azure	Separate metadata for ordinary resources and shared papers

users is the identity table. Its numeric id is used as a foreign key by notifications and warnings. email links OAuth identity to the account. google_name records the Google name; preferred_name records the student's chosen name. department, urn, crn, semester_no, and profile_completed support onboarding. is_banned and banned_at support moderation. contributor_badge stores one of an empty string, trusted, or best_contributor. last_seen_at is a nullable timestamp used only for approximate presence.

resources stores title, category, branch, semester, subject_name, blob_url, file_name, file_size, status, uploaded_by, and creation time. A row contains metadata, not PDF bytes. status determines whether an item appears in the public verified catalogue. uploaded_by currently stores an email or legacy admin label; that raw value should not be exposed by a public API.

pyqs stores id, subject_code, subject_name, year, blob_url, original_filename, file_size, file_hash, uploaded_by, created_at, and updated_at. file_hash is unique. The standalone importer additionally manages nullable import_source_key and its unique index. The website schema function currently adds updated_at but does not itself add import_source_key, and website listing/uploading does not depend on that field. Do not confuse a field added by the separate manager with a website-owned migration.

user_warnings stores a warning id, owning user_id, message, creating administrator, creation time, and an acknowledged_at field. The current notification bell does not read this table. Warnings are moderation records displayed through their existing dashboard flow. A warning and an official notification are different records with different purposes.

official_announcement stores exactly one banner row. The integer primary key is constrained to id = 1. title and body contain its text; link_url is optional; active decides whether it is visible. revision is an integer version counter. updated_by and updated_at record the last administrator and time. Removing the banner sets active to false while keeping the text for later editing.

student_notifications stores one row per send: id, title, body, link_url, audience, created_by, created_at, and request_key. audience is restricted to all or selected. request_key is a unique UUID used to recognize a retry of the same form submission. It is not a login token and does not decide which students are authorized to read a message.

`student_notification_recipients` stores one row per notification/student pair. Its composite primary key is (`notification_id`, `user_id`), preventing the same recipient from being added twice to a message. `read_at` is NULL for unread and a timestamp after acknowledgement. Both foreign keys use ON DELETE CASCADE: deleting the parent notification or user removes corresponding membership rows. This is referential cleanup, not a bulk "delete all notifications" feature in the UI.

Why two notification tables? If 1,000 students receive one message, the text is stored once and 1,000 small membership rows record delivery/read state. A single global read flag would be wrong: one student reading cannot mark the message read for everyone. Storing 1,000 full copies of the body would waste space and make edits inconsistent. The join table gives each student independent state while sharing the message content.

Indexes support the intended queries. The inbox index starts with `user_id` and orders `notification_id` descending, matching the student's newest-first list. The partial unread index contains only membership rows whose `read_at` is NULL. The presence index excludes NULL timestamps. An index costs storage and extra work on writes; it is justified here by repeated read queries. An index does not guarantee every query will be fast for every data size, and the test suite does not benchmark a production database.

05 Python packages: what pip installs and why

requirements.txt contains seven direct package names without version pins. These are runtime requirements, not seven databases. Installing a package makes its modules available to Python; a package's installation name and import name can differ. Version pinning and a separate reproducible test environment are useful follow-up improvements, because a future install can otherwise resolve different versions.

python-dotenv is imported as dotenv. load_dotenv() loads local configuration into environment variables, and os.getenv() reads values. It helps local development avoid embedding passwords in source. Existing environment values are not overridden by default. The website calls it in configuration/database/storage modules. It does not encrypt a .env file; keeping that file out of Git is still necessary. Reference:

<https://bbc2.github.io/python-dotenv/>

psycopg2-binary installs the psycopg2 PostgreSQL adapter used by the SQL service modules. connect() opens a database connection, cursor() creates an object that executes SQL, and fetchone()/fetchall() retrieve rows. RealDictCursor returns named dictionary-like rows instead of positional tuples. SQL values are passed separately with %s placeholders. With psycopg2, leaving a successful connection context commits; an exception causes rollback. That context does not close the connection, which is why the code also calls close() in finally. Reference: <https://www.psycopg.org/docs/usage>

Flask maps requests to Python functions and builds responses. request holds the current HTTP input; session holds signed cookie state; g stores values for the current request only; jsonify creates JSON; render_template runs Jinja; flash stores a message for a later page; redirect sends the browser to another URL. Flask's default session cookie is signed, not encrypted: do not store secrets in it. Reference:

<https://flask.palletsprojects.com/en/stable/quickstart/>

Gunicorn is a production WSGI server for UNIX environments. A typical Linux deployment command uses gunicorn app:app: the first app is the Python module and the second is the Flask object inside it. It is not the database or a notification worker queue. The website's app.run path is the development-server entry point. A normal native Windows shell is not Gunicorn's target environment; use the development server for local work or the deployment's supported Linux/WSL environment. Reference: <https://gunicorn.org/>

Authlib provides the Google OAuth/OpenID Connect integration. OAuth(app) creates the registry; oauth.register describes Google and requested scopes. authorize_redirect begins sign-in; authorize_access_token handles the callback exchange and supplies the user information consumed by app.py. Application code still checks that the returned email is verified and decides administrator membership from its allowlist. Login through Google is not permission to administer Bookshelf. Reference:

<https://docs.authlib.org/en/v1.6.6/client/flask.html>

requests is a Python HTTP client. It is listed in requirements, but the reviewed website modules do not directly call requests.get or requests.post. Authlib's Flask client integration uses an HTTP client layer built around Requests. Do not confuse this dependency with JavaScript fetch(), which runs in the browser and does not require pip. Reference: <https://requests.readthedocs.io/en/latest/>

azure-storage-blob exposes BlobServiceClient, container clients, and blob clients. The website uses the connection string to obtain a service client, chooses a container by resource category, and uploads/deletes blobs. The returned URL is stored in SQL. A blob URL alone does not make a private container publicly

readable; public access or an appropriate access mechanism is a hosting/storage concern. The website currently stores `blob_client.url` rather than issuing short-lived SAS links. Reference:

<https://learn.microsoft.com/en-us/azure/storage/blobs/storage-blob-upload-python>

Jinja2 and Werkzeug are important indirect dependencies supplied with Flask. Jinja renders loops, conditions, macros, and escaped template values. Werkzeug supplies `secure_filename`, request exceptions, and ProxyFix. `secure_filename` produces a conservative filename, not malware detection. ProxyFix trusts a configured number of forwarded headers and must match the real proxy topology. References:

<https://jinja.palletsprojects.com/en/stable/templates/> and

https://werkzeug.palletsprojects.com/en/stable/middleware/proxy_fix/

Python's standard library does not need separate pip installation: `os` reads configuration; `secrets` creates CSRF tokens and compares them; `uuid` creates/validates identifiers; `hashlib` calculates SHA-256; `datetime` handles years and timestamps; `pathlib` inspects paths; `zipfile` checks Office archive structure; `contextlib` implements the transaction context manager; `functools.wraps` preserves decorated function metadata; `re` validates patterns; `logging` records failures; `unittest` and `unittest.mock` support tests. ReportLab, pypdf, pdfplumber, and Poppler are document-building/checking tools used for this PDF, not new website dependencies. PySide6 belongs to the separate upload manager, not the Flask notification feature. Three.js is a JavaScript dependency listed in `package.json`, not a pip module.

06 Reading the syntax without guessing

In Python, `def` creates a function; indented lines form its body. Calling a function executes that body. `return` exits it and supplies a result. `None` means no value; it is not an empty list and not the string `"None"`. `if/elif/else` choose branches. `try/except` handle failures; `finally` runs for cleanup whether the main operation succeeds or fails. `raise` reports failure to the caller. A docstring explains code but does not execute the described operation.

An `import` loads names from a module. Importing `app.py` has side effects: configuration checks and schema initialization run at module level. That is why tests patch startup database functions before importing it. A comment saying "DUMMY LOGIN" is outdated in the current file; the executable code uses Google OAuth. Read executable statements, not only comments.

A decorator wraps a function. The route registers the already-wrapped callable. In the admin mutation pattern, authentication runs before the CSRF-protected body. `@wraps` preserves the original function's name so Flask route registration does not confuse multiple wrappers. `*args` and `**kwargs` forward positional and named arguments, including IDs extracted from a URL.

A dictionary maps keys to values. `data['id']` requires a key; `data.get('id')` can return `None`; `data.get('id', default)` supplies a fallback. A comprehension constructs a collection from another collection. `{int(value) for value in recipients}` is a set comprehension, so duplicate IDs disappear. `sorted(...)` turns that set into a predictable ordered list. A tuple groups SQL parameters; a one-item tuple needs a trailing comma, such as `(user_id,)`.

In SQL, `SELECT` reads, `INSERT` creates, `UPDATE` changes, and `DELETE` removes rows. `WHERE` is the filter that prevents an operation affecting every record. `JOIN` connects related rows. `RETURNING` asks a write to return fields from affected rows. `COUNT(*)` counts rows; `COUNT(read_at)` counts non-NULL read timestamps. `NULL` means missing/unknown, so use `IS NULL` rather than `= NULL`. `COALESCE` selects the first non-NULL value; `NULLIF(x, '')` converts an empty string to `NULL` before fallback selection.

`%s` inside Psycopyg SQL is a parameter marker. It is not Python string interpolation. The separate parameters argument keeps user text as data. Escaping percent and underscore for `ILIKE` is another concern: it makes a search for those characters literal rather than treating them as wildcards. Trusted SQL fragments such as an optional `WHERE` clause can be assembled in code; user values must remain bound parameters.

In JavaScript, `const` prevents reassignment of the binding and `let` permits reassignment. An object or `Map` stored in `const` can still be modified. An arrow function is a callable. An immediately invoked function expression, `(() => { ... })()`, gives the file private local variables. `async/await` makes request code easier to sequence without blocking the browser's whole UI. A promise rejection enters `catch`; an HTTP 503 does not automatically make `fetch` reject, so `response.ok` must also be checked.

`document.querySelector` finds a DOM element; `createElement` constructs one. `textContent` inserts text literally; it should be used for user-written messages instead of `innerHTML`. `append` attaches nodes and `replaceChildren` clears/replaces a list. `classList` changes styling classes. `hidden` toggles visibility. `dataset` reads HTML data-* attributes: `data-search-url` becomes `dataset.searchUrl`. `URL` and `URLSearchParams` safely construct query strings. `?.` is optional chaining, `...` spreads values, and backticks form template strings containing `${expressions}`.

In Jinja, `{{ value }}` outputs a value and `{% ... %}` performs a condition, loop, import, or macro operation. These execute before the browser receives HTML. HTML templates rendered by Flask are normally autoescaped, so a title containing markup is shown as text. That does not make every URL safe, hence the separate link validator. A macro is a reusable rendering function; its arguments let the profile add a styling class without duplicating

the badge.

In CSS, a selector chooses elements and declarations assign property/value pairs. A class selector starts with a dot; an ID selector starts with #. :root holds shared custom properties such as --accent. var(--accent) reads one. Later or more specific rules can override earlier ones. @media enables rules for a screen condition. Width controls the box; padding is inside; margin is outside; gap separates grid/flex children; min-width: 0 permits a child to shrink; overflow-wrap prevents long text escaping its box. Closing braces end groups; blank lines only aid reading.

07 Authentication and permission boundaries

Google login starts by clearing previous session state and remembering whether the visitor chose student or admin login. The callback obtains a verified email from Google's response. `normalize_email` trims whitespace and case-folds it before membership checks. If it is in `ADMIN_EMAILS`, `app.py` stores `is_admin` and `admin_user`. Otherwise it creates or updates a student and stores the student session. The Google picture is accepted only as a bounded HTTPS URL; the dashboard has an initial fallback.

`admin_required` currently checks `session['is_admin']`. It does not accept a role posted by the browser. Signed session cookies make direct tampering detectable if the signing secret is strong and private. This decorator does not re-read the allowlist for every request; revocation of an already-issued admin session needs separate consideration. The review also confirmed that `SESSION_COOKIE_SECURE` is not enabled in current code. For HTTPS production, this flag should be enabled as part of a properly configured deployment.

`csrf_token` creates a random token if the session does not already contain one. Templates include it in hidden form inputs or a data attribute consumed by browser code. `csrf_protected` requires the submitted token and uses `secrets.compare_digest`. This makes a state-changing request prove that it originated from a page that knew the session's token. It is not encryption and does not replace login checks.

The notification `student_required` wrapper obtains the email from `session['user']`, loads the current account, rejects missing or banned accounts, and sets `g.notification_user` to that row's id. That ID exists only for this request. A browser-supplied `user_id` is ignored. Every inbox/read query is scoped to this server-resolved ID. This is the core privacy boundary: knowing another student's ID or a notification ID is insufficient to read their inbox.

Authentication failures have different responses: missing student identity produces 401, a missing/banned account produces 403, malformed input produces 400, and an unavailable database produces 503. Admin authentication redirects to the login page, so the admin JavaScript checks `response.redirected` as well as `response.ok` before treating a response as JSON. Cache-Control: no-store discourages storage of private API responses by browsers and intermediaries.

08 Announcement logic: one editable official bar

The administrator opens GET /admin/communications. admin_page loads both the current banner (including a hidden one) and recent send history. It renders the two forms. The announcement form carries revision, title, body, optional link, and CSRF token. The hidden revision represents the version seen by that editor, not a version the server should blindly overwrite.

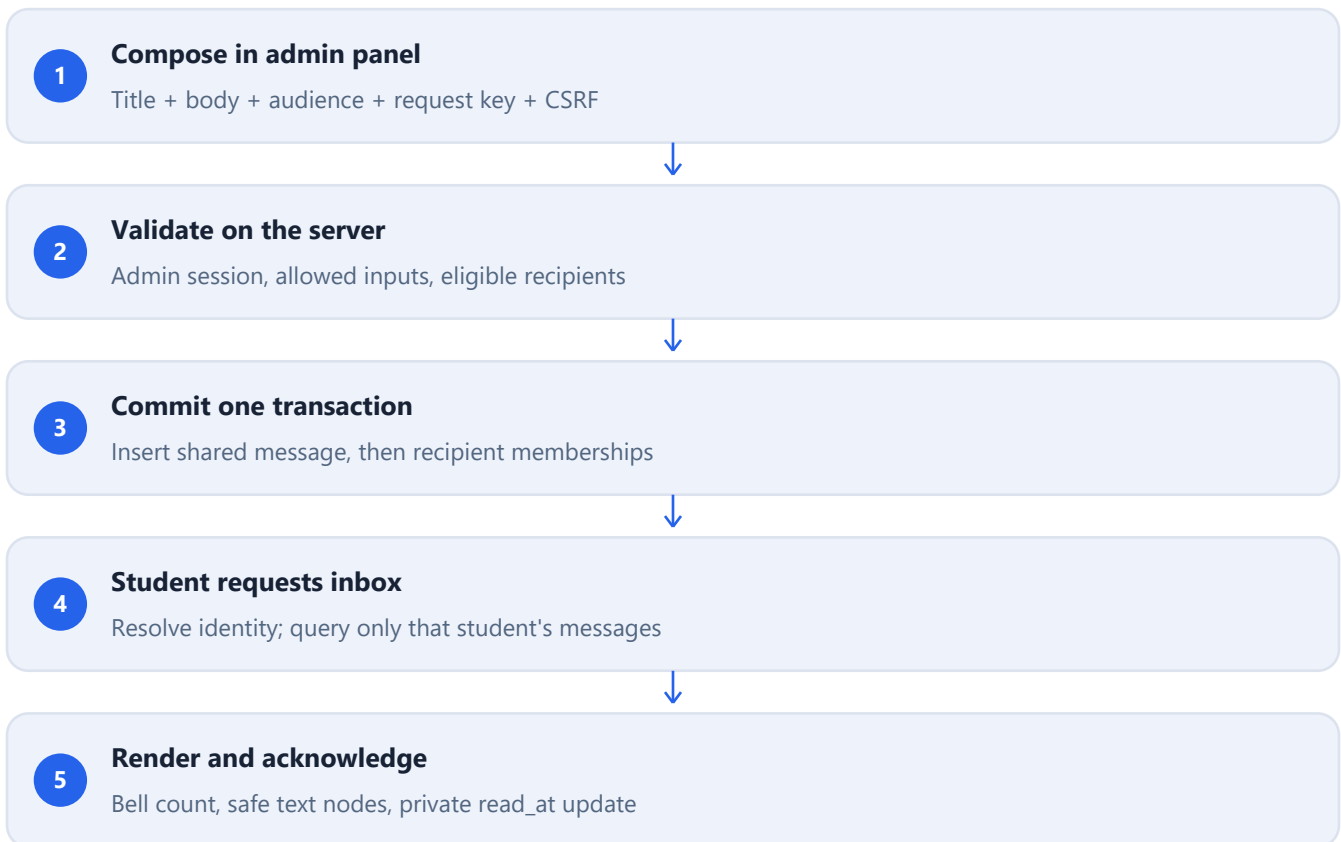
POST /admin/communications/announcement runs administrator and CSRF checks. If action is remove, the service changes active to false. Otherwise it validates the content and updates title, body, link, visibility, audit fields, and revision. SQL includes WHERE id=1 AND revision=%s. If no row is returned, another edit changed the revision; the service asks the administrator to reload instead of overwriting that newer work.

Example: two admin tabs both load revision 8. The first saves and produces revision 9. The second submits revision 8. Its WHERE clause matches no row, so the newer banner survives. This is optimistic concurrency control: edits normally proceed without locking the whole editor session, but the final write checks its assumption. The same idea is used when renaming a PYQ with expected_name.

The student dashboard calls announcement() and renders the section only when an active row exists. It is server-rendered on dashboard load. Banner changes do not stream into an already-open dashboard; the student refreshes or reopens it. The inbox counter has its own polling logic and does not refresh the banner. Publishing a banner does not create a notification row or recipient rows.

content() trims inputs, enforces a 1-160 character title and 1-2,000 character body, and rejects embedded NUL. valid_link() allows an empty link, HTTP(S) URLs with a hostname and no embedded username/password, or an internal path starting with one slash. It rejects control characters, backslashes, and protocol-relative // links. This is syntactic link validation, not a guarantee that an external site's contents are trustworthy.

09 Sending notifications: the complete journey



Imagine an administrator wants to announce "New Data Structures notes" to two students. The browser submits the title, body, optional /notes link, audience=selected, repeated recipients fields, a request_key UUID, and the CSRF token. It does not submit a trusted sender identity. The route derives created_by from the administrator's session.

The request enters admin_send_notification. After access checks, it calls service.send_notification(...) with plain Python values. Validation happens again on the server even if the browser already disabled an invalid button. A malicious client can bypass JavaScript, so the server checks audience, text lengths, URL shape, positive recipient IDs, the 32-bit user-ID range, and the maximum of 500 selected students.

The service normalizes request_key using UUID(...). For selected recipients it converts values to integers, removes duplicates with a set, and sorts them. Thus recipients 7, 9, 7 become [7, 9]. Empty selected lists are rejected. In all mode the supplied recipient list is deliberately ignored: SQL selects all eligible users itself.

The service opens one database transaction. First it inserts the shared message with ON CONFLICT (request_key) DO NOTHING RETURNING id. If a row is returned, it is a new send. Next, INSERT ... SELECT adds membership rows by selecting users who are not banned and are not administrator emails. For selected mode, AND id=ANY(%s) restricts this to the requested IDs.

The service checks cursor.rowcount after inserting memberships. If zero recipients were found, it raises an error. For selected mode, the number must exactly match the deduplicated IDs; otherwise at least one requested student was removed, banned, an administrator, or otherwise unavailable. Raising inside the transaction rolls

back both message and memberships. The system does not silently deliver only to the surviving subset of a selected group.

Only after the transaction exits successfully does the route report the recipient count and redirect to the admin communications page. The redirect implements post/redirect/get: refreshing the resulting page is a GET rather than resubmitting the POST. The browser also disables the send button while submitting, but the database uniqueness rule is the real duplicate-send protection.

For 5,000 eligible students, all mode produces one student_notifications row and 5,000 recipient rows. It is a synchronous database operation in the web request. The code has no worker queue or chunked broadcast service. A future larger deployment may need an outbox/worker design, but this version favors a single straightforward transaction. Its actual capacity has not been load-tested here.

"All active students" in this form means currently registered, non-banned accounts excluding administrators. It does not mean only students online in the last five minutes. New registrations after a send do not receive that older message because membership is captured at send time. The separate presence feature uses a different definition of active.

10 Why retrying a send does not create another message

`request_key` identifies one logical form submission. `admin_page` generates `uuid4()` for a fresh form, but preserves `request.form['request_key']` when re-rendering a failed submission. If a send succeeds and the network loses the response, the administrator may not know whether it committed. Retrying that same form uses the same key.

The unique constraint ensures that concurrent or repeated inserts for that key cannot both create independent message rows. On conflict, the service counts memberships belonging to the existing notification and returns that original count. It does not insert recipients again. The protection is scoped to the key, not to matching message text: intentionally sending the same text from a new form with a new key creates a new notification.

Example: key K sends to students 7 and 9 and commits. The response is lost. The same form posts K again. The insert returns no row; the service looks up the count for K and returns 2. The database still contains one message and two memberships. If the first attempt actually rolled back, no row owns K and the retry can create the send.

Important boundary: if an uncertain send has already committed, editing the body while retaining the old key will still return the original send; it will not replace that message. A corrected message should be a new intentional send with a fresh key. No message-edit feature is implemented for already-sent notifications.

Do not confuse this with a distributed guarantee for Azure uploads. The notification message and its memberships live in one PostgreSQL transaction. Azure and PostgreSQL do not participate in one shared transaction in the current upload/deletion code. That distinction explains why notification retries are safer than the known PYQ deletion failure path.

11 Recipient search and selection in the browser

`static/admin-communications.js` stores selected recipients in a Map keyed by string user ID. A Map prevents duplicate selections and keeps selection independent of the currently visible search results. `updateSelection` recreates hidden recipients inputs and removable name chips, synchronizes checkbox state, updates the count, and changes the send button label. Hidden inputs are how a normal form POST transmits selections.

Search waits 250 milliseconds after typing stops. This debounce avoids a request for every keystroke. An `AbortController` cancels the previous search so an old query is less likely to replace newer results. The request uses the URL stored in `data-search-url` and adds `q` via `searchParams`. A redirect or failed status becomes a visible error rather than attempting to parse a login page as JSON.

Each search result is built with `createElement` and `textContent`. A checkbox adds/removes its ID in the Map. The UI stops selection at 500 and the server enforces the same limit independently. Switching searches preserves existing IDs; selecting someone does not depend on keeping that person's checkbox visible forever. Selected names loaded after a failed form initially appear as `Student #ID` until search results can recover names.

Choosing all hides the picker; the all-mode backend ignores selected IDs that remain in the form. Choosing selected shows it and runs a search. `pageshow` recomputes button state after navigating back from another page. The `unavailable` flag preserves the server's decision to disable sending when the communications database could not be loaded.

12 Student inbox queries and pagination

GET /api/notifications resolves the current student, validates optional before, and calls `inbox(user_id, before)`. The service first counts unread recipient rows for that user. It then joins memberships to messages, applies the same user filter, sorts by notification ID descending, and fetches up to 21 rows.

Why 21 when the page size is 20? The extra row proves another page exists. The response returns the first 20 and supplies `next_before` equal to the twentieth row's ID only when there is an extra row. Loading older messages asks for IDs less than that cursor. This is keyset pagination: it continues from a known item instead of skipping an offset that could shift when new messages arrive.

Example: your visible IDs are 120 down to 101 and a twenty-first row 100 exists. The response sends 20 rows and `next_before=101`. The next request uses `before=101` and returns IDs below 101. A newly inserted message 121 does not shift that older-page boundary. IDs are ordering keys, not gap-free message counts; rollbacks and concurrency can leave gaps, and sequence ID order is not a perfect statement of commit time.

`created_at` and `read_at` become ISO-format strings in the route because JavaScript needs a transport representation for dates. The browser creates Date objects and uses the viewer's local format for display. The admin send history explicitly formats its timestamps in UTC. This is why the same moment may have different visible clock text in different parts of the application.

The JSON response contains `items`, `unread_count`, and `next_before`. An empty inbox is a successful response with an empty list; a database failure is 503 with an error. The UI should not interpret a failure as "no messages." The code keeps these states separate and offers a retry button for an inbox load failure.

13 Bell, unread counter, and read acknowledgements

The dashboard template places a notification-center beside the avatar. Its data-endpoint gives JavaScript the inbox route; data-csrf supplies the current session token. The bell has aria-expanded and aria-controls so assistive technology can understand the panel relationship. The panel starts hidden and includes a message list, status text, retry button, load-older button, and mark-all control.

notifications.js caches those DOM references and uses loading to prevent overlapping loads. load() fetches the newest page; load(true) appends an older page using nextBefore. The unread bubble shows the number or 99+ and hides at zero. The accessible bell label keeps the actual unread number. row(item) creates the heading, body, timestamp, optional link, and mark-read button using safe text nodes.

The interval runs every 30 seconds while the document is visible. It calls load(false, true): countOnly updates the counter without replacing the open list or moving focus while a student is reading. Opening the bell calls a full load. Becoming visible again refreshes the count. Consequently a new message can raise the count without immediately appearing inside an already-open list; closing/reopening or another full load refreshes the list.

Mark as read sends POST /api/notifications/ID/read with the CSRF token. The database update requires BOTH notification_id and user_id. COALESCE(read_at, CURRENT_TIMESTAMP) preserves the first read time if the request is repeated. A valid ID that is not owned by that user produces no updated row and the route returns 404. Opening the bell or following View update does not automatically mark the item read; the explicit control records the acknowledgement.

Mark all read sends through=latestId, where latestId came from the newest item in the last full list load. SQL marks only that user's unread IDs less than or equal to through. This avoids marking a higher-ID message that arrived after the displayed snapshot as read. It is an ID boundary, not an exact list of every item the student physically saw, and lower-ID concurrent commits are a subtle edge case of sequence-based ordering.

After a successful acknowledgement the code reloads the inbox. Failure restores the button and shows an error. Escape and the close button hide the panel and restore focus to the bell; clicking outside also closes it. The profile photo error handler removes a broken image so the initial behind it becomes visible. It handles both a later error event and an image that had already failed before the listener was attached.

A current limitation is that notifications.js does not attach a fetch timeout. Its loading guard remains true until the request settles, so an indefinitely hanging request can delay subsequent loads. Presence requests do use a timeout. This is a concrete future robustness improvement, not a feature already added by writing this guide.

14 Notification worked examples and failure cases

Selected send: users 7 and 9 exist, are not banned, and are not administrators. The service creates message 42 and rows (42,7,NULL) and (42,9,NULL). Student 7 reads it; only (42,7) gains read_at. Student 9 still has one unread message. History reports 2 recipients and 1 read. "Read" here means the explicit acknowledgement was recorded, not proof of comprehension or even a link click.

Invalid selection: users 7 and 9 are chosen, then user 9 is banned before the membership INSERT runs. Only user 7 is eligible in that statement. rowcount is 1 but the requested unique count is 2, so the service raises and the transaction rolls back. The administrator must refresh the selection. No partly created send is intentionally committed.

Unauthorized read: student 7 posts the ID of a message addressed only to student 9. The server still resolves `g.notification_user=7`. `WHERE user_id=7 AND notification_id=...` matches nothing. Changing a client-side hidden ID cannot change the resolved account. Missing CSRF is rejected before the mutation reaches SQL.

Banner conflict: one tab publishes revision 4 to 5; a second tab tries to remove revision 4. The second write matches nothing and fails with a reload instruction. Hidden text remains editable after a successful removal. A sent notification is unaffected because the banner and inbox tables are separate.

Network uncertainty: a send can commit before the browser receives its response. The retry key prevents a second copy for the same submission. This is different from a validation error, where the transaction is not committed. The route preserves form values on error so the administrator does not lose the composed message or accidentally abandon the retry identity.

15 PYQs: metadata, hashes, upload and editing

The filename is a source of metadata, not a reliable unique database key by itself. Multiple papers can share subject code and year. The dedicated importer uses a stable export identity and a content hash for matching. The website's single uploader checks the hash for duplicate contents. It does not search by subject code/year and replace an arbitrary paper.

`validate_metadata` trims the subject code/name, bounds their lengths to 100/255 characters, and converts the year to an integer from 1900 through next calendar year. `fingerprint` checks for a PDF marker in the first 1,024 bytes, then reads chunks of 1 MiB into a SHA-256 hash while enforcing the 50 MiB limit. Streaming avoids reading thousands of whole PDFs into application memory. The file pointer is reset before upload. A signature check is a format sanity check, not a complete PDF parser or antivirus scan.

`import_pdf` first looks up `file_hash`. If it exists, it returns that ID with `created=False`. Otherwise it uploads to Azure using a UUID-prefixed sanitized blob name, then inserts the metadata. The database unique constraint remains essential: two requests can both pass the initial lookup. `ON CONFLICT DO NOTHING` handles that race, and the losing request attempts to delete its extra uploaded blob.

The current website `upload_blob` call does not explicitly pass application/pdf content settings. The independent upload-manager plan included PDF content type, but that should not be assumed to exist in `storage_agent.py`. The current website also has no local durable upload journal. Its cleanup is best-effort; an uncertain database commit can require reconciliation rather than unconditional deletion. These distinctions matter when learning from the code.

`list_pyqs` builds optional literal subject/code search and year filters, counts matches, and returns at most 50 rows ordered by descending year then ID. It creates title by combining `subject_code`, a separator, and `subject_name`. The public PYQ JSON endpoint whitelists `id`, `title`, `subject_code`, `subject_name`, `year`, and `blob_url`; it does not expose uploader or hash fields.

The admin list presents each paper as a row with title, original filename, year, open link, edit-name control, and delete action. `rename_pyq` normalizes whitespace and validates a new subject name. Its `UPDATE` requires both `id` and `expected_name`, so editing an old form does not overwrite another subject-name edit. It updates `updated_at` while preserving ID, code, year, filename, hash, and blob URL. Two administrators changing a name away and back can evade an expected-name check; a monotonic revision would be stronger if that concurrency mattered.

Known failure: `admin_delete_pyq` deletes the blob before deleting the SQL row. If the SQL delete fails, the row points to a missing PDF. A later retry treats the already-missing blob as cloud failure and retains the row. The review reproduced this with mocks. A robust improvement would use a durable deletion job/tombstone, retryable cleanup, and idempotent handling of already-absent blobs, with concurrency/reference checks. This guide records the problem; it has not repaired it.

16 Ordinary resources and dashboard filters

The normal upload route accepts notes, assignment, and book. It requires an administrator and CSRF token. It trims form fields, validates the category/branch/semester allowlists and text lengths, sanitizes the filename, checks file signatures and size, uploads to Azure, then inserts metadata with verified status and the administrator's email. A failed metadata save triggers best-effort blob cleanup.

The file checks distinguish PDF, JPEG, PNG, WebP, old Office compound files, and ZIP-based DOCX/PPTX structure. Checking [Content_Types].xml and the expected word/ or ppt/ folder is more meaningful than trusting the extension alone. However it does not prove that a file is harmless, uncorrupted, or free of embedded active content.

get_resource_catalogue accepts a known category plus optional branch, semester, search, and page. It always adds status='verified'. It executes a count query and a page query using the same filter conditions. It looks up the uploader's preferred/Google/legacy name and current contributor badge, suppressing the awarded badge for banned users. Results are freshly queried when the page loads, so a verified upload appears without editing a static HTML card list.

_resource_catalogue is the route helper behind /notes, /assignments, and /books. It validates URL input and marks a row as an admin upload when normalized uploaded_by is the legacy literal admin or appears in ADMIN_EMAILS. It displays Admin, otherwise the resolved contributor name, otherwise User. The template escapes these labels and renders a consistent resource card with file details and link.

dashboard.js reads the selected department and semester from data attributes. When a category is clicked it constructs a URL using data-url, adds branch and semester for ordinary categories, and navigates to it. PYQs bypass those filters because they are shared across students. A direct visit to a resource section can still use its search/filter form without going through the dashboard flow.

The public legacy /api/resources route currently returns the full get_resources dictionaries, including uploaded_by. That can reveal email addresses to anonymous callers and was reproduced in the review. The HTML catalogue's friendly label does not fix a separate JSON endpoint. The appropriate follow-up is a public field allowlist similar to /api/pyqs. Also, the present /upload route is admin-only: a Contribution page and a contributor badge do not by themselves implement a working student file-submission backend.

17 Trusted authors and profile badges

There are three distinct ideas: who uploaded a resource, whether that resource is verified, and whether an account has an awarded badge. Admin upload identity comes from the allowlist or legacy admin value. Verified is a resource moderation status. Trusted and Best Contributor are manual account awards. Neither award grants administrator permissions and neither is automatically calculated from upload counts.

The users schema adds contributor_badge with allowed values "", trusted, and best_contributor. search_users and the user row conversion include that column, otherwise the template would have no value to render. set_contributor_badge validates the user ID and allowed award, updates the row, and returns whether a user was found. The protected route preserves the search query when returning to user management.

The shared contributor_badge macro renders the selected visual, and nothing for an unrecognized/empty value. Trusted uses a circle/check motif; Best Contributor uses a star. A macro argument adds profile-contributor-badge on the dashboard. The shared resource badge remains subdued; the compound profile selector changes only that instance to violet and adjusts dark mode. Changing CSS does not change the database award.

The mild glow is a box-shadow plus a small inset shadow. Its colour and opacity are restrained. The profile-specific override uses both the profile class and the Best Contributor class, so it does not restyle every resource badge. This is an example of narrowly scoped CSS: selectors should express the precise place you intend to change.

18 Recent presence: what the active count really means

The shared `_student_activity.html` include adds `student-activity.js` only for a signed-in non-admin student session. It appears on the dashboard, contribution page, and shared catalogue pages. A visible page sends a POST heartbeat immediately and about once a minute. It skips hidden documents, concurrent requests, and attempts less than 30 seconds apart, and attaches a ten-second timeout.

The route independently resolves the student and rejects banned accounts. `record_activity` updates `last_seen_at` using the database clock only if the previous timestamp is NULL or older than 30 seconds. The SQL throttle protects against duplicate page/tab reports even when multiple browsers run independent timers. It does not avoid every SELECT or HTTP request, and the current implementation opens database connections rather than using an application-level pool.

`active_students` counts eligible users seen in the past five minutes, then retrieves the latest 50. It excludes banned accounts and administrator emails. The total is not limited to 50. A missing timestamp means no recent heartbeat has been recorded, not that the user never logged in. Closing a page naturally expires presence; there is no guaranteed instant offline signal.

`admin-presence.js` refreshes every 30 seconds while visible, updates both prominent counts, and builds links to search that user's email in the admin panel. It avoids replacing the list while keyboard focus is inside it. Hiding the list stores a browser preference and leaves the count refreshing. It is a display setting, not permission to expose the data publicly. API failures show an unavailable message and a dash rather than falsely showing zero users.

19 Dashboard layout fixes and the admin redesign

The department-card alignment bug came from unequal text heights interacting with space-between. A two-line Computer Science title made its text block taller than Civil. width:100% on the text wrapper makes every badge use the available card width; min-height:2.6em on a heading with line-height:1.3 reserves two lines. The icon stays in its original top region. The change targets .dept-card, so unrelated headings do not gain a blank line of height.

Branch icons are local SVGs with the same line-art language: no filled background, consistent stroke treatment, and scalable shapes. Electronics uses a circuit motif, Electrical a lightning symbol, Mechanical a gear, and Civil a bridge. These are interface icons, not verified institutional logos. SVG viewBox supplies the internal coordinate system; stroke draws outlines, fill='none' avoids filled shapes, and rounded caps/joins keep the appearance consistent.

For mobile, the sidebar becomes a five-column grid. Its nav/footer wrappers use display:contents so the actual links become grid items. Short labels and icons prevent PYQ archive from crowding the row. Bottom safe-area padding keeps content above the floating navigation. The settings popup is positioned relative to the available edge rather than translating from a misleading center. Viewport-specific changes are distinct from desktop navigation.

When a branch changes, the script clears an earlier reveal timer, resets semester selection, hides categories, and updates selected/dimmed classes. After the short reveal delay it shows semesters and scrolls there on the next animation frame. Semester selection records a snapshot so a delayed callback can check that the choice is still current. reduced-motion switches smooth scroll to immediate scroll. Keyboard Enter/Space triggers card selection as well.

The admin redesign replaces green/sage design tokens with slate/charcoal neutrals and blue accents, including related archive/communications stylesheets. The main header uses a clean sans-serif treatment. Helper text is generally increased, student names are 18px, account details are 15px, and important management controls have a 44px minimum height. Green remains in small success/status indicators where it carries meaning, rather than being the dominant theme.

The user table keeps explicit table/row/cell roles and a visually hidden header but renders each student as a grid card. Each cell also has a visible label. At wide widths it has three columns, at intermediate widths two, and at small mobile widths one. min-width:0, wrapped email text, and fluid warning forms prevent the old wide table from forcing sideways scrolling. Resource tables retain their own column layout.

Dark-mode variables change surfaces, borders, text, and accents without changing data. The active-user count retains special emphasis in both themes. The website's student dashboard and its resource badges keep their own palette. Browser checks used populated local fixtures at widths including 1,440, 1,024, 768, 390, and 320 pixels, with expanded warning forms, panel switching, theme switching, badge selection, keyboard focus, and active-list toggling. They were layout/interaction checks, not live moderation actions.

20 The standalone bulk uploader, in context

The separate admin only folder contains PYQ Studio, a PySide6 desktop application. Its own requirements include PySide6 plus python-dotenv, psycpg2-binary, and azure-storage-blob. PySide6 is Qt's Python binding: it supplies the window, controls, model/view table, dialogs, and signals. QAbstractTableModel exposes paper rows without constructing thousands of independent row widgets; QSortFilterProxyModel handles sorting/filtering. A QThread worker keeps scans and other background work away from the UI thread, and signals deliver progress back to the interface. The job engine uses three concurrent file workers.

The manager scans the chosen export root or its pdfs folder. Year folder names provide the exam year. Matching checkpoint download metadata can recover full subject names and stable source identity; filenames are a fallback. A missing name becomes Name unavailable. The scanner reads the local export; it does not follow scraper URLs or alter the original collection. Source selection, year filters, metadata editing, and explicit replacement review happen before an upload job.

classify in catalogue.py compares local papers with database records using import_source_key and hashes. A subject/year pair is not a safe identity because multiple different papers can share it. The manager distinguishes new, unchanged, update, duplicate, and review-needed states. Manual matching is available for older rows without import identity. Add-only mode should not silently replace a record; replacement mode requires explicit review. Metadata-only changes can keep the existing PDF.

jobs.py records task intentions and stages in a local SQLite journal before cloud writes. On resume it checks whether a database commit or blob upload already happened. An uncertain database outcome retains the new blob for reconciliation instead of immediately deleting it. cloud.py uses application/pdf content settings and unique blob names; it updates a replacement transactionally while preserving the row ID. Old-file cleanup happens after the replacement commit and reference checks in both pyqs and resources. Missing files in a local folder never imply permission to delete remote papers.

The operator's selected admin email is attribution within a trusted local tool; it is not Google OAuth authentication. Access to the manager's connection credentials is the real capability to write. Configuration and state stay in its own folder. Moving it outside the repository requires preserving job state and recreating its virtual environment as needed. The website imports none of this code and reads only the resulting shared-table metadata. The source atlas deliberately covers the tracked main website commit, while this chapter explains the separate tool's role and current local modules.

The optional manifest command-line importer is a different execution path: it is add-only and does not participate in the desktop journal's recovery. Its default validation run does not upload; --apply starts writes after validation. Do not assume every recovery guarantee in the Qt job engine also exists in the website's single-upload helper or the manifest CLI. This separation is why the code review still found a website deletion problem even though the desktop manager has a more careful replacement workflow.

21 Animation refinements included in the commit

The latest commit also changes the landing page's animation loops. These changes are included in the code atlas for completeness; they are not dependencies of the notification feature. `createDesktopAnimationLoop` coordinates `requestAnimationFrame` with desktop pointer/media conditions, element visibility, page visibility, and an optional enabled predicate. A frame variable prevents scheduling multiple outstanding callbacks for the same loop.

`grid-bg.js` caches size and `Path2D` grid segments on desktop. Segments outside the mouse-distortion area can replay a cached path rather than recomputing points. Resize marks the cache dirty. The loop stops requesting frames after exact state convergence, then wakes on input or layout changes. It preserves a fallback path when the shared helper is unavailable.

`page-wheel.js` stores previously written style values in a `WeakMap` and avoids repeating identical writes on desktop. It caches container height and asks for layout again when scroll progress, mouse input, or resize changes state. `script.js` similarly caches hero bounds and rope dimensions. `three.js` and `page2-narrative.js` use the scheduler for their rendering/watch loops. The goal is less unnecessary offscreen or idle rendering while retaining the original animation behaviour. There is no production performance benchmark in the recorded checks, so a specific speedup percentage would be an unsupported claim.

22 Testing, review findings, and safe maintenance

The regression run passed 58 tests. Tests use unittest and mocks for database/Azure operations. They exercise authorization, CSRF, selected/all audiences, duplicate-send paths, rollback conditions, inbox ownership, pagination, banner conflicts, badge handling, catalogue filters, and presence. Mock tests prove the tested Python decisions and generated SQL shapes; they do not execute a real PostgreSQL query planner or validate Azure account permissions.

Tracked Python and JavaScript syntax checks passed, and the latest commit passed whitespace/diff checks. A pattern scan of tracked source/config text found no matches for the credential patterns used, and no tracked .env file was found. This is limited evidence, not a complete secret audit of all Git history, binary assets, or hosting configuration. The working tree was clean at review time. The review did not push or alter application code.

Four findings remain relevant. PYQ deletion can leave a record referring to a deleted file after database failure. The anonymous resource JSON API returns uploader emails. Session cookies lack the Secure flag for HTTPS production. Finally, .gitignore excludes tests/, so the local tests are absent from the tracked commit and cannot be reproduced from a fresh clone without restoring them. These findings were reported, not fixed during document creation.

Startup initializes tables from app.py after the users schema succeeds. That requires the database user to have appropriate schema privileges. The code uses additive IF NOT EXISTS operations, but that is not a complete migration framework: it does not reconcile arbitrary incompatible existing column definitions. create_pyq_schema is outside the communications/moderation try blocks, so its failure can prevent application startup. Test migrations against a staging copy and take a backup before a production change; neither occurred in the review.

For local tests, the normal command is `python -m unittest discover -s tests -p "test_*.py"` in an environment with website dependencies. The actual review used an isolated installed dependency directory and mocked startup imports. Existing tests rely on configured Google OAuth values being available during import; a fully self-contained test setup should provide harmless placeholders for those too. Never import app.py casually in a "read-only" helper if it might connect to production on import.

For notification debugging, identify the stage first. A 401/403 points toward session/account checks. A 400 points toward CSRF or validation. A 503 calls for server logs and database/schema availability. A successful send with no message for a later registrant is expected membership-at-send-time behaviour. A counter that changes without the open list changing is count-only polling. Do not start by deleting notification rows or changing SQL constraints.

23 Learning exercises with answers

Exercise 1: one notification is sent to three students; one marks it read. How many rows are involved? Answer: one shared message, three membership rows, and one non-NULL read_at. The unread count is calculated separately for each user.

Exercise 2: why cannot a form field named user_id choose which inbox is loaded? Answer: student_required derives g.notification_user from the authenticated email and the query filters by that server-resolved value. The route does not trust a posted recipient identity for reads.

Exercise 3: why fetch 21 messages? Answer: return 20 and use the extra result to decide whether next_before should exist. The next query uses an ID boundary, so new higher-ID records do not shift the older-page starting point.

Exercise 4: does hiding active students stop activity tracking? Answer: no. It hides the admin list and stores a browser preference; heartbeats and the count refresh continue. Presence is approximate, and all-notification eligibility is a different concept from five-minute activity.

Exercise 5: why does a different violet profile badge not require a database migration? Answer: the stored value is still best_contributor. The macro adds a context class and CSS changes its appearance only at that location.

Exercise 6: a second selected recipient becomes banned during composition. What happens? Answer: the INSERT SELECT includes only eligible accounts; a count mismatch raises inside the transaction, rolling back the send. The administrator refreshes the selection.

Exercise 7: the file uploaded, but metadata saving failed. Is a database transaction sufficient to undo Azure? Answer: no. The services are separate. Best-effort cleanup or a durable reconciliation workflow is required. This is why the deletion issue matters even though notification SQL is transactional.

Exercise 8: how would you add notification delivery to a department? Answer: add a validated audience option and a server-side department condition to recipient selection, preserve administrator/banned filtering and transaction/idempotency rules, then add access/selection tests and UI controls. Merely hiding students in the picker would not enforce a new server policy. This is a design exercise, not implemented code.

24 Glossary and reference shelf

API: an endpoint returning data for code to consume. Route: the URL/method-to-function mapping. Endpoint: Flask's name for a registered view, used by `url_for`. WSGI: the interface between a Python web server and the application. DOM: the browser's tree of page elements. ORM: a database-to-object abstraction; this website uses direct Psycopg SQL rather than an ORM for the reviewed features.

Transaction: related SQL work that commits or rolls back together. Atomic: the transaction's writes are applied together or not applied. Idempotent operation: repeating the same logical request does not create another independent effect. Foreign key: a rule linking a row to an existing parent. Composite key: a key made of multiple columns. Optimistic concurrency: writing only if a version/value still matches the one the editor saw. Cursor pagination: continuing after/before an ordering key.

Authentication: proving who is signed in. Authorization: deciding what that identity may do. CSRF: a cross-site request that tries to use an existing session to perform an unwanted action. XSS: attacker-controlled script executing in a page; escaped templates and `textContent` reduce relevant injection paths. UUID: an identifier used here for logical sends/blob names. Hash: a content fingerprint used for duplicate detection; hashing is not encryption.

Polling: the browser periodically asks for updates. WebSocket: a persistent bidirectional channel, not used here. Push notification: an operating-system/browser delivery mechanism, not implemented by this bell. Debounce: wait briefly until rapid input stops. Throttle: limit how frequently an operation proceeds. Reconciliation: compare stored state across systems after uncertainty and repair inconsistencies. Outbox: a durable record of pending external work; it is a suggested future reliability pattern, not an existing website component.

Repository source is the authority for the behaviour described here. Official package references are included near their explanations for further learning, not as evidence that the deployment runs the newest versions. Package versions are not pinned in `requirements.txt`. The remaining source atlas records exact line numbers from the documented snapshot so the explanatory chapters can be checked against code.

Source atlas - the changed code, with reading notes

This atlas includes every added/replacement line in the 42-file commit and nearby context for edited existing files. It shows new files in full. Source line numbers refer to the current snapshot, not a fabricated example. Very long source lines wrap visually; continuation rows have no new line number. Omitted unchanged regions are explicitly identified. Earlier chapters explain the surrounding existing authentication, upload, profile, and moderation behaviour.

The two Python communications files have a per-line reading table. The two browser communications files have closely grouped line notes for multi-statement lines and callbacks. Other files have purpose notes plus the exact changed code. Use the syntax chapter to decode punctuation, CSS values, and template delimiters. This is a code study atlas, not a claim that every unmodified historical line in the whole repository changed today.

A leading source label such as 125 is an editor line number, not part of the code. Source spelling, comments, and colour literals are retained. Icon characters unsupported by the code font are displayed as explicit Unicode notation such as `\u26A0` or `\U0001F4DA`, solely to prevent missing-glyph squares in the PDF. These represent the original source character, not a code edit. Tabs are expanded for alignment. An old comment or a limited error path should be interpreted alongside the limitations recorded in the teaching chapters.

A01 communications_service.py

The database half of notifications. Read top to bottom: transaction helper, schema, input validation, banner operations, recipient search, atomic send/retry, history, inbox, and acknowledgements.

Source: communications_service.py | 186 total lines | SHA-256 fe0fbf6c3af1cca3...

Source lines 1-186

transaction (lines 12-20): Open a connection, refuse an unavailable database, yield a RealDictCursor inside a transaction, and always close the connection. Success commits and exceptions roll back. The service caller owns the SQL but this helper owns its lifetime.

create_schema (lines 23-57): Create the singleton announcement, shared messages, recipient memberships, and supporting indexes. IF NOT EXISTS supports repeated startup. The initial banner is inactive, so startup itself neither publishes nor sends anything.

valid_link (lines 60-71): Normalize a link and reject unsafe URL shapes before saving. Empty is allowed. Relative paths must have one leading slash; external HTTP(S) links need a hostname and cannot embed credentials.

content (lines 74-78): Normalize and bound message text before SQL, reject NUL, and delegate link checking. This is shared by banner publishing and notification sends, avoiding different validation rules.

announcement (lines 81-85): Fetch the one banner and return it only if visible, unless the admin explicitly asks to include hidden state. This enables remove-without-losing-the-editor-text.

save_announcement (lines 88-95): Validate text, then update the singleton only when its revision matches the form. Increment the revision in the same statement. No returned row means a concurrent edit won and the current request must reload.

remove_announcement (lines 98-104): Hide the banner with the same revision guard. It keeps title/body/link for reuse, records the actor/time, and advances the revision. It does not delete sent messages.

find_students (lines 107-117): Return at most 50 eligible matches. Escape literal search wildcards, exclude banned/admin accounts, choose a display name, and bind all user-supplied search values.

send_notification (lines 120-149): Validate all inputs, normalize/deduplicate selected IDs, then atomically insert the message and memberships. A unique request key makes a retry return the original recipient count. A count mismatch raises inside the transaction and prevents a partial selected send.

history (lines 152-161): Aggregate membership and read counts per message, including zero-recipient rows through a left join if any exist. Return the newest 50 sends and normalize timestamps to UTC for the admin template.

inbox (lines 164-175): Count this student's unread memberships. Join to message content, apply an optional ID cursor, fetch 21 to display 20, and use the extra row to advertise another page.

mark_read (lines 178-186): Use the authenticated user ID in both update variants. A single read preserves its first timestamp; read-all updates only unread memberships at or below the displayed latest-ID boundary.

```

1  """Official announcements and private, persistent student notifications."""
2  from contextlib import contextmanager
3  from datetime import timezone
4  from urllib.parse import urlsplit
5  from uuid import UUID
6
7  from psycopg2.extras import RealDictCursor
8  from database import get_connection
9
10
11 @contextmanager
12 def transaction():
13     conn = get_connection()
14     if conn is None:
15         raise RuntimeError('Communications are temporarily unavailable.')
16     try:
17         with conn, conn.cursor(cursor_factory=RealDictCursor) as cur:
18             yield cur

```

Line	Reading note
1	Continue the surrounding statement or SQL expression. Read with the preceding line and the function purpose above; indentation/parentheses determine its scope.
2	Import the named module/helper. Package and standard-library roles are explained in chapter 5.
3	Import the named module/helper. Package and standard-library roles are explained in chapter 5.
4	Import the named module/helper. Package and standard-library roles are explained in chapter 5.
5	Import the named module/helper. Package and standard-library roles are explained in chapter 5.
7	Import the named module/helper. Package and standard-library roles are explained in chapter 5.
8	Import the named module/helper. Package and standard-library roles are explained in chapter 5.
11	Turn a generator using yield into a with-block context manager.
12	Open a connection, refuse an unavailable database, yield a RealDictCursor inside a transaction, and always close the connection. Success commits and exceptions roll back. The service caller owns the SQL but this helper owns its lifetime.
13	Compute/store this named value for the following steps. Its exact inputs and defaults are visible on this line.
14	Evaluate this condition and run the indented branch only when true. The expression shows the exact validation, result, or state being checked.
15	Signal failure to the caller; inside a transaction this prevents the partial writes from committing.
16	Begin operations whose validation, network, or database failures will be handled below.
17	Enter the resource/transaction context. Exceptions trigger cleanup and, for the connection, rollback.
18	Give the cursor to the calling with block, then resume here when that block finishes.

communications_service.py | continued, lines 19-36

```

19     finally:
20         conn.close()
21
22
23 def create_schema():
24     with transaction() as cur:
25         cur.execute('''
26             CREATE TABLE IF NOT EXISTS official_announcement (
27                 id INTEGER PRIMARY KEY CHECK (id = 1),
28                 title VARCHAR(160) NOT NULL DEFAULT '',
29                 body VARCHAR(2000) NOT NULL DEFAULT '',
30                 link_url VARCHAR(2000) NOT NULL DEFAULT '',
31                 active BOOLEAN NOT NULL DEFAULT FALSE,
32                 revision BIGINT NOT NULL DEFAULT 0,
33                 updated_by VARCHAR(255),
34                 updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP

```

```

35         );
36         INSERT INTO official_announcement (id) VALUES (1) ON CONFLICT DO NOTHING;

```

Line	Reading note
19	Begin cleanup that runs on both success and failure.
20	Release the database connection even after a failed query; the transaction context alone does not close Psycopg2 connections.
23	Create the singleton announcement, shared messages, recipient memberships, and supporting indexes. IF NOT EXISTS supports repeated startup. The initial banner is inactive, so startup itself neither publishes nor sends anything.
24	Enter the resource/transaction context. Exceptions trigger cleanup and, for the connection, rollback.
25	Send SQL and bound parameters to PostgreSQL through the cursor. Execution is inside the surrounding transaction where present.
26	Declare a persistent table if absent; following column lines define its stored data and constraints.
27	Define this column's type and constraints. NOT NULL requires a value; DEFAULT fills omitted values; key/check clauses enforce stored-data rules.
28	Define this column's type and constraints. NOT NULL requires a value; DEFAULT fills omitted values; key/check clauses enforce stored-data rules.
29	Define this column's type and constraints. NOT NULL requires a value; DEFAULT fills omitted values; key/check clauses enforce stored-data rules.
30	Define this column's type and constraints. NOT NULL requires a value; DEFAULT fills omitted values; key/check clauses enforce stored-data rules.
31	Define this column's type and constraints. NOT NULL requires a value; DEFAULT fills omitted values; key/check clauses enforce stored-data rules.
32	Define this column's type and constraints. NOT NULL requires a value; DEFAULT fills omitted values; key/check clauses enforce stored-data rules.
33	Change matching rows. Read the accompanying WHERE clause carefully because it is the permission/concurrency boundary.
34	Change matching rows. Read the accompanying WHERE clause carefully because it is the permission/concurrency boundary.
35	Close the grouped call/collection/statement opened above; this punctuation has no separate business action.
36	Create rows; this may be a new shared message, the singleton seed, or memberships selected from users.

communications_service.py | continued, lines 37-54

```

37         CREATE TABLE IF NOT EXISTS student_notifications (
38             id BIGSERIAL PRIMARY KEY,

```

```

39         title VARCHAR(160) NOT NULL,
40         body VARCHAR(2000) NOT NULL,
41         link_url VARCHAR(2000) NOT NULL DEFAULT '',
42         audience VARCHAR(20) NOT NULL CHECK (audience IN ('all', 'selected')),
43         created_by VARCHAR(255) NOT NULL,
44         created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
45         request_key UUID NOT NULL UNIQUE
46     );
47     CREATE TABLE IF NOT EXISTS student_notification_recipients (
48         notification_id BIGINT NOT NULL REFERENCES student_notifications(id) ON DELETE
CASCADE,
49         user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
50         read_at TIMESTAMPTZ,
51         PRIMARY KEY (notification_id, user_id)
52     );
53     CREATE INDEX IF NOT EXISTS idx_notification_inbox
54     ON student_notification_recipients (user_id, notification_id DESC);

```

Line	Reading note
37	Declare a persistent table if absent; following column lines define its stored data and constraints.
38	Define this column's type and constraints. NOT NULL requires a value; DEFAULT fills omitted values; key/check clauses enforce stored-data rules.
39	Define this column's type and constraints. NOT NULL requires a value; DEFAULT fills omitted values; key/check clauses enforce stored-data rules.
40	Define this column's type and constraints. NOT NULL requires a value; DEFAULT fills omitted values; key/check clauses enforce stored-data rules.
41	Define this column's type and constraints. NOT NULL requires a value; DEFAULT fills omitted values; key/check clauses enforce stored-data rules.
42	Define this column's type and constraints. NOT NULL requires a value; DEFAULT fills omitted values; key/check clauses enforce stored-data rules.
43	Define this column's type and constraints. NOT NULL requires a value; DEFAULT fills omitted values; key/check clauses enforce stored-data rules.
44	Define this column's type and constraints. NOT NULL requires a value; DEFAULT fills omitted values; key/check clauses enforce stored-data rules.
45	Define this column's type and constraints. NOT NULL requires a value; DEFAULT fills omitted values; key/check clauses enforce stored-data rules.
46	Close the grouped call/collection/statement opened above; this punctuation has no separate business action.
47	Declare a persistent table if absent; following column lines define its stored data and constraints.
48	Define this column's type and constraints. NOT NULL requires a value; DEFAULT fills omitted values; key/check clauses enforce stored-data rules.

Line	Reading note
49	Define this column's type and constraints. NOT NULL requires a value; DEFAULT fills omitted values; key/check clauses enforce stored-data rules.
50	Define this column's type and constraints. NOT NULL requires a value; DEFAULT fills omitted values; key/check clauses enforce stored-data rules.
51	Define this column's type and constraints. NOT NULL requires a value; DEFAULT fills omitted values; key/check clauses enforce stored-data rules.
52	Close the grouped call/collection/statement opened above; this punctuation has no separate business action.
53	Create a lookup structure for the repeated filters/order below, rather than scanning every row by default.
54	Define a join match, conflict rule, or index target depending on the statement above.

communications_service.py | continued, lines 55-72

```

55         CREATE INDEX IF NOT EXISTS idx_notification_unread
56         ON student_notification_recipients (user_id) WHERE read_at IS NULL;
57     '')
58
59
60 def valid_link(value):
61     value = str(value or '').strip()
62     if not value:
63         return ''
64     if len(value) > 2000 or any(ord(c) < 32 for c in value) or '\\' in value:
65         raise ValueError('Enter a valid website link.')
66     parts = urlsplit(value)
67     if value.startswith('/') and not value.startswith('///'):
68         return value
69     if parts.scheme in ('https', 'http') and parts.hostname and not parts.username and not
parts.password:
70         return value
71     raise ValueError('Links must start with https://, http://, or a single /.')
72

```

Line	Reading note
55	Create a lookup structure for the repeated filters/order below, rather than scanning every row by default.
56	Define a join match, conflict rule, or index target depending on the statement above.
57	Open/close or provide a documentation/SQL string; it becomes executable SQL only when passed to execute.
60	Normalize a link and reject unsafe URL shapes before saving. Empty is allowed. Relative paths must have one leading slash; external HTTP(S) links need a hostname and cannot embed credentials.
61	Compute/store this named value for the following steps. Its exact inputs and defaults are visible on this line.
62	Evaluate this condition and run the indented branch only when true. The expression shows the exact validation, result, or state being checked.

Line	Reading note
63	End this helper/view and provide its result or HTTP response to the caller. A transaction context exits before the result reaches the caller.
64	Evaluate this condition and run the indented branch only when true. The expression shows the exact validation, result, or state being checked.
65	Signal failure to the caller; inside a transaction this prevents the partial writes from committing.
66	Compute/store this named value for the following steps. Its exact inputs and defaults are visible on this line.
67	Evaluate this condition and run the indented branch only when true. The expression shows the exact validation, result, or state being checked.
68	End this helper/view and provide its result or HTTP response to the caller. A transaction context exits before the result reaches the caller.
69	Evaluate this condition and run the indented branch only when true. The expression shows the exact validation, result, or state being checked.
70	End this helper/view and provide its result or HTTP response to the caller. A transaction context exits before the result reaches the caller.
71	Signal failure to the caller; inside a transaction this prevents the partial writes from committing.

communications_service.py | continued, lines 73-90

```

73
74 def content(title, body, link_url):
75     title, body = str(title or '').strip(), str(body or '').strip()
76     if not 1 <= len(title) <= 160 or not 1 <= len(body) <= 2000 or '\x00' in title + body:
77         raise ValueError('Enter a title (up to 160 characters) and a message (up to 2,000
characters).')
78     return title, body, valid_link(link_url)
79
80
81 def announcement(include_hidden=False):
82     with transaction() as cur:
83         cur.execute('SELECT title, body, link_url, active, revision FROM official_announcement
WHERE id = 1')
84         row = cur.fetchone()
85         return dict(row) if row and (include_hidden or row['active']) else None
86
87
88 def save_announcement(title, body, link_url, actor, revision):
89     title, body, link_url = content(title, body, link_url)
90     with transaction() as cur:

```

Line	Reading note
74	Normalize and bound message text before SQL, reject NUL, and delegate link checking. This is shared by banner publishing and notification sends, avoiding different validation rules.

Line	Reading note
75	Compute/store this named value for the following steps. Its exact inputs and defaults are visible on this line.
76	Evaluate this condition and run the indented branch only when true. The expression shows the exact validation, result, or state being checked.
77	Signal failure to the caller; inside a transaction this prevents the partial writes from committing.
78	End this helper/view and provide its result or HTTP response to the caller. A transaction context exits before the result reaches the caller.
81	Fetch the one banner and return it only if visible, unless the admin explicitly asks to include hidden state. This enables remove-without-losing-the-editor-text.
82	Enter the resource/transaction context. Exceptions trigger cleanup and, for the connection, rollback.
83	Send SQL and bound parameters to PostgreSQL through the cursor. Execution is inside the surrounding transaction where present.
84	Read one result row, possibly None. Dictionary cursors let subsequent code access fields by name.
85	End this helper/view and provide its result or HTTP response to the caller. A transaction context exits before the result reaches the caller.
88	Validate text, then update the singleton only when its revision matches the form. Increment the revision in the same statement. No returned row means a concurrent edit won and the current request must reload.
89	Compute/store this named value for the following steps. Its exact inputs and defaults are visible on this line.
90	Enter the resource/transaction context. Exceptions trigger cleanup and, for the connection, rollback.

communications_service.py | continued, lines 91-108

```

91         cur.execute('''UPDATE official_announcement SET title=%s, body=%s, link_url=%s,
92             active=TRUE, updated_by=%s, updated_at=CURRENT_TIMESTAMP, revision=revision+1
93             WHERE id=1 AND revision=%s RETURNING id''', (title, body, link_url, actor,
int(revision)))
94         if not cur.fetchone():
95             raise ValueError('The announcement changed. Reload this page before saving.')
96
97
98 def remove_announcement(actor, revision):
99     with transaction() as cur:
100         cur.execute('''UPDATE official_announcement SET active=FALSE, revision=revision+1,
101             updated_by=%s, updated_at=CURRENT_TIMESTAMP WHERE id=1 AND revision=%s RETURNING
id''',
102             (actor, int(revision)))
103         if not cur.fetchone():
104             raise ValueError('The announcement changed. Reload this page before removing it.')
105
106
107 def find_students(term, admin_emails):
108     term = str(term or '').strip()[:200]

```

Line	Reading note
91	Send SQL and bound parameters to PostgreSQL through the cursor. Execution is inside the surrounding transaction where present.
92	Compute/store this named value for the following steps. Its exact inputs and defaults are visible on this line.
93	Restrict affected/returned rows. For inbox/read operations, this must include the server-resolved user.
94	Evaluate this condition and run the indented branch only when true. The expression shows the exact validation, result, or state being checked.
95	Signal failure to the caller; inside a transaction this prevents the partial writes from committing.
98	Hide the banner with the same revision guard. It keeps title/body/link for reuse, records the actor/time, and advances the revision. It does not delete sent messages.
99	Enter the resource/transaction context. Exceptions trigger cleanup and, for the connection, rollback.
100	Send SQL and bound parameters to PostgreSQL through the cursor. Execution is inside the surrounding transaction where present.
101	Change matching rows. Read the accompanying WHERE clause carefully because it is the permission/concurrency boundary.
102	Continue the surrounding statement or SQL expression. Read with the preceding line and the function purpose above; indentation/parentheses determine its scope.
103	Evaluate this condition and run the indented branch only when true. The expression shows the exact validation, result, or state being checked.
104	Signal failure to the caller; inside a transaction this prevents the partial writes from committing.
107	Return at most 50 eligible matches. Escape literal search wildcards, exclude banned/admin accounts, choose a display name, and bind all user-supplied search values.
108	Compute/store this named value for the following steps. Its exact inputs and defaults are visible on this line.

communications_service.py | continued, lines 109-126

```

109     pattern = '%' + term.replace('\\', '\\\\').replace('%', '\\%').replace('_', '\\_') + '%'
110     with transaction() as cur:
111         cur.execute('''SELECT id, email, COALESCE(NULLIF(preferred_name, ''), google_name, name,
'Student') AS name,
112             department, semester_no FROM users WHERE is_banned=FALSE
113             AND NOT (LOWER(email)=ANY(%s))
114             AND (email ILIKE %s OR preferred_name ILIKE %s OR google_name ILIKE %s
115                 OR name ILIKE %s OR urn ILIKE %s OR crn ILIKE %s)
116             ORDER BY id DESC LIMIT 50''', (list(admin_emails), *([pattern] * 6)))
117         return [dict(row) for row in cur.fetchall()]
118
119
120 def send_notification(title, body, link_url, audience, recipients, actor, request_key,
    admin_emails):

```

```

121     title, body, link_url = content(title, body, link_url)
122     key = str(UUID(str(request_key)))
123     if audience not in ('all', 'selected'):
124         raise ValueError('Choose all students or selected students.')
125     try:
126         ids = sorted({int(value) for value in recipients}) if audience == 'selected' else []

```

Line	Reading note
109	Build a contains-search pattern while escaping literal wildcard characters; SQL values are still passed separately.
110	Enter the resource/transaction context. Exceptions trigger cleanup and, for the connection, rollback.
111	Send SQL and bound parameters to PostgreSQL through the cursor. Execution is inside the surrounding transaction where present.
112	Compute/store this named value for the following steps. Its exact inputs and defaults are visible on this line.
113	Add another condition that must also hold, such as eligibility, ownership, or a cursor boundary.
114	Add another condition that must also hold, such as eligibility, ownership, or a cursor boundary.
115	Continue the surrounding statement or SQL expression. Read with the preceding line and the function purpose above; indentation/parentheses determine its scope.
116	Make result order explicit; LIMIT bounds the displayed page rather than the underlying total.
117	End this helper/view and provide its result or HTTP response to the caller. A transaction context exits before the result reaches the caller.
120	Validate all inputs, normalize/deduplicate selected IDs, then atomically insert the message and memberships. A unique request key makes a retry return the original recipient count. A count mismatch raises inside the transaction and prevents a partial selected send.
121	Compute/store this named value for the following steps. Its exact inputs and defaults are visible on this line.
122	Parse and normalize the submitted logical-send key; malformed UUID input raises validation failure.
123	Evaluate this condition and run the indented branch only when true. The expression shows the exact validation, result, or state being checked.
124	Signal failure to the caller; inside a transaction this prevents the partial writes from committing.
125	Begin operations whose validation, network, or database failures will be handled below.
126	Convert selected IDs to integers, deduplicate with a set, and sort for deterministic bounds/count checks.

communications_service.py | continued, lines 127-144

```

127     except (TypeError, ValueError):
128         raise ValueError('Select valid students.') from None
129     if audience == 'selected' and (not ids or ids[0] <= 0 or ids[-1] > 2147483647 or len(ids) >
130 500):
131         raise ValueError('Select between 1 and 500 students.')

```

```

131     with transaction() as cur:
132         # A repeated form submission returns the original send; it cannot send twice.
133         cur.execute(''INSERT INTO student_notifications (title, body, link_url, audience,
created_by, request_key)
134             VALUES (%s,%s,%s,%s,%s,%s) ON CONFLICT (request_key) DO NOTHING RETURNING id'',
135             (title, body, link_url, audience, actor, key))
136         inserted = cur.fetchone()
137         if not inserted:
138             cur.execute(''SELECT COUNT(*) AS total FROM student_notification_recipients r
139             JOIN student_notifications n ON n.id=r.notification_id WHERE
n.request_key=%s'', (key,))
140             return cur.fetchone()['total']
141         notification_id = inserted['id']
142         clause = ' AND id=ANY(%s)' if audience == 'selected' else ''
143         cur.execute(''INSERT INTO student_notification_recipients (notification_id, user_id)
144             SELECT %s, id FROM users WHERE is_banned=FALSE AND NOT (LOWER(email)=ANY(%s))'' +
clause,

```

Line	Reading note
127	Handle the named failure category rather than returning a false success; inspect the following response or raised error.
128	Signal failure to the caller; inside a transaction this prevents the partial writes from committing.
129	Evaluate this condition and run the indented branch only when true. The expression shows the exact validation, result, or state being checked.
130	Signal failure to the caller; inside a transaction this prevents the partial writes from committing.
131	Enter the resource/transaction context. Exceptions trigger cleanup and, for the connection, rollback.
132	Comment: A repeated form submission returns the original send; it cannot send twice.
133	Send SQL and bound parameters to PostgreSQL through the cursor. Execution is inside the surrounding transaction where present.
134	Supply bound values for a newly inserted row; placeholders are filled by the separate parameter tuple.
135	Continue the surrounding statement or SQL expression. Read with the preceding line and the function purpose above; indentation/parentheses determine its scope.
136	Read one result row, possibly None. Dictionary cursors let subsequent code access fields by name.
137	Evaluate this condition and run the indented branch only when true. The expression shows the exact validation, result, or state being checked.
138	Send SQL and bound parameters to PostgreSQL through the cursor. Execution is inside the surrounding transaction where present.
139	Link records through their IDs so shared content can be combined with private membership state.
140	End this helper/view and provide its result or HTTP response to the caller. A transaction context exits before the result reaches the caller.

Line	Reading note
141	Compute/store this named value for the following steps. Its exact inputs and defaults are visible on this line.
142	Choose a trusted SQL fragment for the selected-recipient mode; user IDs remain bound parameters.
143	Send SQL and bound parameters to PostgreSQL through the cursor. Execution is inside the surrounding transaction where present.
144	Choose returned columns or an aggregate; values stay in the database until fetchone/fetchall reads the result.

communications_service.py | continued, lines 145-162

```

145         (notification_id, list(admin_emails), ids) if ids else (notification_id,
list(admin_emails)))
146         count = cur.rowcount
147         if not count or (audience == 'selected' and count != len(ids)):
148             raise ValueError('One or more recipients are unavailable. Refresh your selection and
try again.')
149         return count
150
151
152 def history():
153     with transaction() as cur:
154         cur.execute('''SELECT n.id, n.title, n.body, n.audience, n.created_at,
COUNT(r.user_id) AS recipients, COUNT(r.read_at) AS read_count
155         FROM student_notifications n LEFT JOIN student_notification_recipients r ON
156         r.notification_id=n.id
157         GROUP BY n.id ORDER BY n.id DESC LIMIT 50''')
158         rows = [dict(row) for row in cur.fetchall()]
159         for row in rows:
160             row['created_at'] = row['created_at'].astimezone(timezone.utc)
161         return rows
162

```

Line	Reading note
145	Continue the surrounding statement or SQL expression. Read with the preceding line and the function purpose above; indentation/parentheses determine its scope.
146	Read how many rows the preceding write affected; compare this with intended recipient membership.
147	Evaluate this condition and run the indented branch only when true. The expression shows the exact validation, result, or state being checked.
148	Signal failure to the caller; inside a transaction this prevents the partial writes from committing.
149	End this helper/view and provide its result or HTTP response to the caller. A transaction context exits before the result reaches the caller.
152	Aggregate membership and read counts per message, including zero-recipient rows through a left join if any exist. Return the newest 50 sends and normalize timestamps to UTC for the admin template.
153	Enter the resource/transaction context. Exceptions trigger cleanup and, for the connection, rollback.

Line	Reading note
154	Send SQL and bound parameters to PostgreSQL through the cursor. Execution is inside the surrounding transaction where present.
155	Continue the surrounding statement or SQL expression. Read with the preceding line and the function purpose above; indentation/parentheses determine its scope.
156	Choose the source table for a read or INSERT SELECT membership snapshot.
157	Combine related records for per-message or per-year aggregates.
158	Read the result rows and make them available to Python for formatting or paging.
159	Repeat the indented work for each value; the loop target names the current item.
160	Represent the same instant in UTC for the administrator history display.
161	End this helper/view and provide its result or HTTP response to the caller. A transaction context exits before the result reaches the caller.

communications_service.py | continued, lines 163-180

```

163
164 def inbox(user_id, before=None):
165     with transaction() as cur:
166         cur.execute('''SELECT COUNT(*) AS total FROM student_notification_recipients
167             WHERE user_id=%s AND read_at IS NULL''', (user_id,))
168         unread = cur.fetchone()['total']
169         cur.execute('''SELECT n.id, n.title, n.body, n.link_url, n.created_at, r.read_at
170             FROM student_notifications n JOIN student_notification_recipients r ON
171             r.notification_id=n.id
172             WHERE r.user_id=%s''' + (' AND n.id < %s' if before else '') + ' ORDER BY n.id DESC
173             LIMIT 21',
174             (user_id, before) if before else (user_id,))
175         rows = [dict(row) for row in cur.fetchall()]
176         return {'items': rows[:20], 'unread_count': unread,
177             'next_before': rows[19]['id'] if len(rows) > 20 else None}
178
179 def mark_read(user_id, notification_id=None, through=None):
180     with transaction() as cur:
181         if notification_id is not None:

```

Line	Reading note
164	Count this student's unread memberships. Join to message content, apply an optional ID cursor, fetch 21 to display 20, and use the extra row to advertise another page.
165	Enter the resource/transaction context. Exceptions trigger cleanup and, for the connection, rollback.
166	Send SQL and bound parameters to PostgreSQL through the cursor. Execution is inside the surrounding transaction where present.

Line	Reading note
167	Restrict affected/returned rows. For inbox/read operations, this must include the server-resolved user.
168	Read one result row, possibly None. Dictionary cursors let subsequent code access fields by name.
169	Send SQL and bound parameters to PostgreSQL through the cursor. Execution is inside the surrounding transaction where present.
170	Choose the source table for a read or INSERT SELECT membership snapshot.
171	Restrict affected/returned rows. For inbox/read operations, this must include the server-resolved user.
172	Continue the surrounding statement or SQL expression. Read with the preceding line and the function purpose above; indentation/parentheses determine its scope.
173	Read the result rows and make them available to Python for formatting or paging.
174	End this helper/view and provide its result or HTTP response to the caller. A transaction context exits before the result reaches the caller.
175	Continue the surrounding statement or SQL expression. Read with the preceding line and the function purpose above; indentation/parentheses determine its scope.
178	Use the authenticated user ID in both update variants. A single read preserves its first timestamp; read-all updates only unread memberships at or below the displayed latest-ID boundary.
179	Enter the resource/transaction context. Exceptions trigger cleanup and, for the connection, rollback.
180	Evaluate this condition and run the indented branch only when true. The expression shows the exact validation, result, or state being checked.

communications_service.py | continued, lines 181-186

```

181         cur.execute('''UPDATE student_notification_recipients SET
read_at=COALESCE(read_at,CURRENT_TIMESTAMP)
182         WHERE user_id=%s AND notification_id=%s RETURNING notification_id''', (user_id,
notification_id))
183         return bool(cur.fetchone())
184         cur.execute('''UPDATE student_notification_recipients SET read_at=CURRENT_TIMESTAMP
185         WHERE user_id=%s AND notification_id<=%s AND read_at IS NULL''', (user_id, through))
186         return True

```

Line	Reading note
181	Send SQL and bound parameters to PostgreSQL through the cursor. Execution is inside the surrounding transaction where present.
182	Restrict affected/returned rows. For inbox/read operations, this must include the server-resolved user.
183	End this helper/view and provide its result or HTTP response to the caller. A transaction context exits before the result reaches the caller.

Line	Reading note
184	Send SQL and bound parameters to PostgreSQL through the cursor. Execution is inside the surrounding transaction where present.
185	Restrict affected/returned rows. For inbox/read operations, this must include the server-resolved user.
186	End this helper/view and provide its result or HTTP response to the caller. A transaction context exits before the result reaches the caller.

A02 communications_routes.py

The HTTP half. Helpers close over the existing app/decorators/allowlist, and each route translates form/query/session input into service calls. The student identity comes from the server-side profile lookup.

Source: communications_routes.py | 166 total lines | SHA-256 5f5a39bc571c7e6e...

Source lines 1-166

register_routes (lines 11-166): Register the communications routes on the existing Flask app using its supplied admin/CSRF decorators and allowlist. Nested helpers capture those dependencies; no second Flask app or server is created.

student_required (lines 12-26): Resolve the current student from the signed-in session and a fresh user row, reject missing/banned accounts, and attach its numeric ID to request-local g. This prevents arbitrary recipient selection by the client.

wrapped (lines 14-25): Execute the enclosing authentication or CSRF guard, then call the original view with forwarded arguments. functools.wraps retains the original view identity used by Flask.

admin_page (lines 28-39): Load banner/history and render the communications workspace. Preserve submitted fields and the send key after errors. A service outage disables publishing and returns 503 instead of a success-shaped empty state.

admin_communications (lines 43-44): Serve the administrator-only communications workspace by delegating rendering to admin_page.

student_activity (lines 49-55): Record approximate presence only after student/CSRF checks. Return 204 without a body on success and 503 on tracking failure.

admin_active_students (lines 59-66): Return recent presence as a private no-store JSON response. On failure, return an unavailable response so the UI does not confuse an outage with zero active users.

admin_announcement (lines 71-86): Choose publish or remove from the submitted action, derive the actor from the session, pass the revision, convert validation/service failures into an editor response, and redirect with feedback on success.

notification_students (lines 90-97): Expose the bounded recipient search only to admins, with no-store caching and a generic failure response.

admin_send_notification (lines 102-113): Translate a normal form POST into service inputs. Repeated recipients fields require getlist. The sender comes from the session. Validation errors preserve the form; uncertain service failures preserve the retry key.

student_notifications (lines 117-134): Validate the optional cursor as a positive signed-64-bit value, query this student's inbox, convert dates to ISO strings, and return no-store JSON. Do not return another user's data on malformed input.

read_notification (lines 139-149): Reject out-of-range notification IDs, invoke the user-scoped read update, return 404 for no owned membership, and JSON success otherwise.

read_all_notifications (lines 154-166): Require a valid through boundary from the form and mark only the current user's unread memberships through that boundary. CSRF applies to this mutation too.

```

1  """Authenticated admin publishing and student inbox endpoints."""
2  from uuid import uuid4
3  from functools import wraps
4
5  from flask import abort, flash, jsonify, render_template, redirect, request, session, url_for, g
6  from database import get_user_by_email
7  import communications_service as service
8  import presence_service
9
10
11 def register_routes(app, admin_required, csrf_protected, admin_emails):
12     def student_required(view):
13         @wraps(view)
14         def wrapped(*args, **kwargs):
15             email = session.get('user', {}).get('email')
16             if not email:
17                 abort(401)
18             try:

```

Line	Reading note
1	Continue the surrounding statement or SQL expression. Read with the preceding line and the function purpose above; indentation/parentheses determine its scope.
2	Import the named module/helper. Package and standard-library roles are explained in chapter 5.
3	Import the named module/helper. Package and standard-library roles are explained in chapter 5.
5	Import the named module/helper. Package and standard-library roles are explained in chapter 5.
6	Import the named module/helper. Package and standard-library roles are explained in chapter 5.
7	Import the named module/helper. Package and standard-library roles are explained in chapter 5.
8	Import the named module/helper. Package and standard-library roles are explained in chapter 5.
11	Register the communications routes on the existing Flask app using its supplied admin/CSRF decorators and allowlist. Nested helpers capture those dependencies; no second Flask app or server is created.
12	Resolve the current student from the signed-in session and a fresh user row, reject missing/banned accounts, and attach its numeric ID to request-local g. This prevents arbitrary recipient selection by the client.
13	Preserve the original view metadata while adding a guard wrapper.
14	Execute the enclosing authentication or CSRF guard, then call the original view with forwarded arguments. funtools.wraps retains the original view identity used by Flask.
15	Read authenticated session context with a safe fallback if a key is absent.
16	Evaluate this condition and run the indented branch only when true. The expression shows the exact validation, result, or state being checked.
17	Stop the request with the given HTTP error status; do not continue into the protected operation.
18	Begin operations whose validation, network, or database failures will be handled below.

communications_routes.py | continued, lines 19-36

```

19         profile = get_user_by_email(email)
20     except Exception:
21         return jsonify(error='Notifications are temporarily unavailable.'), 503
22     if not profile or profile.get('is_banned'):
23         abort(403)
24     g.notification_user = profile['id']
25     return view(*args, **kwargs)
26     return wrapped
27
28 def admin_page(error=None, status=200):
29     notice, sent = None, []
30     unavailable = False
31     try:
32         notice, sent = service.announcement(include_hidden=True), service.history()
33     except Exception:
```

```

34         app.logger.exception('Could not load communications workspace')
35         error, status = 'Communications are temporarily unavailable. Please try again.', 503
36         unavailable = True

```

Line	Reading note
19	Compute/store this named value for the following steps. Its exact inputs and defaults are visible on this line.
20	Handle the named failure category rather than returning a false success; inspect the following response or raised error.
21	End this helper/view and provide its result or HTTP response to the caller. A transaction context exits before the result reaches the caller.
22	Evaluate this condition and run the indented branch only when true. The expression shows the exact validation, result, or state being checked.
23	Stop the request with the given HTTP error status; do not continue into the protected operation.
24	Use the account ID resolved for this request, rather than trusting a browser-provided user ID.
25	End this helper/view and provide its result or HTTP response to the caller. A transaction context exits before the result reaches the caller.
26	End this helper/view and provide its result or HTTP response to the caller. A transaction context exits before the result reaches the caller.
28	Load banner/history and render the communications workspace. Preserve submitted fields and the send key after errors. A service outage disables publishing and returns 503 instead of a success-shaped empty state.
29	Compute/store this named value for the following steps. Its exact inputs and defaults are visible on this line.
30	Compute/store this named value for the following steps. Its exact inputs and defaults are visible on this line.
31	Begin operations whose validation, network, or database failures will be handled below.
32	Compute/store this named value for the following steps. Its exact inputs and defaults are visible on this line.
33	Handle the named failure category rather than returning a false success; inspect the following response or raised error.
34	Record a traceback on the server while the client receives a controlled message.
35	Compute/store this named value for the following steps. Its exact inputs and defaults are visible on this line.
36	Compute/store this named value for the following steps. Its exact inputs and defaults are visible on this line.

communications_routes.py | continued, lines 37-54

```

37         return render_template('admin-communications.html', announcement=notice, sent=sent,
38                                error=error, request_key=request.form.get('request_key') or str(uuid4()),
39                                submitted=request.form, unavailable=unavailable), status
40

```

```

41     @app.get('/admin/communications')
42     @admin_required
43     def admin_communications():
44         return admin_page()
45
46     @app.post('/api/activity')
47     @student_required
48     @csrf_protected
49     def student_activity():
50         try:
51             presence_service.record_activity(g.notification_user, admin_emails)
52         except Exception:
53             app.logger.exception('Could not record student activity')
54             return jsonify(error='Activity tracking is temporarily unavailable.'), 503

```

Line	Reading note
37	End this helper/view and provide its result or HTTP response to the caller. A transaction context exits before the result reaches the caller.
38	Read submitted form or query-string input; subsequent validation and bound parameters determine how it may be used.
39	Read submitted form or query-string input; subsequent validation and bound parameters determine how it may be used.
41	Register the function below for this HTTP method/path; decorator arguments describe routing.
42	Wrap this view with the existing administrator-session guard.
43	Serve the administrator-only communications workspace by delegating rendering to admin_page.
44	End this helper/view and provide its result or HTTP response to the caller. A transaction context exits before the result reaches the caller.
46	Register the function below for this HTTP method/path; decorator arguments describe routing.
47	Resolve and authorize the student account before allowing this endpoint.
48	Wrap this mutation with CSRF-token validation before its body runs.
49	Record approximate presence only after student/CSRF checks. Return 204 without a body on success and 503 on tracking failure.
50	Begin operations whose validation, network, or database failures will be handled below.
51	Use the account ID resolved for this request, rather than trusting a browser-provided user ID.
52	Handle the named failure category rather than returning a false success; inspect the following response or raised error.
53	Record a traceback on the server while the client receives a controlled message.

Line	Reading note
54	End this helper/view and provide its result or HTTP response to the caller. A transaction context exits before the result reaches the caller.

communications_routes.py | continued, lines 55-72

```

55         return '', 204
56
57     @app.get('/api/admin/active-students')
58     @admin_required
59     def admin_active_students():
60         try:
61             response = jsonify(presence_service.active_students(admin_emails))
62             response.headers['Cache-Control'] = 'no-store'
63             return response
64         except Exception:
65             app.logger.exception('Could not load active students')
66             return jsonify(error='Active student information is temporarily unavailable.'), 503
67
68     @app.post('/admin/communications/announcement')
69     @admin_required
70     @csrf_protected
71     def admin_announcement():
72         try:

```

Line	Reading note
55	End this helper/view and provide its result or HTTP response to the caller. A transaction context exits before the result reaches the caller.
57	Register the function below for this HTTP method/path; decorator arguments describe routing.
58	Wrap this view with the existing administrator-session guard.
59	Return recent presence as a private no-store JSON response. On failure, return an unavailable response so the UI does not confuse an outage with zero active users.
60	Begin operations whose validation, network, or database failures will be handled below.
61	Compute/store this named value for the following steps. Its exact inputs and defaults are visible on this line.
62	Mark this response no-store so private inbox/student data is not deliberately cached.
63	End this helper/view and provide its result or HTTP response to the caller. A transaction context exits before the result reaches the caller.
64	Handle the named failure category rather than returning a false success; inspect the following response or raised error.
65	Record a traceback on the server while the client receives a controlled message.
66	End this helper/view and provide its result or HTTP response to the caller. A transaction context exits before the result reaches the caller.

Line	Reading note
68	Register the function below for this HTTP method/path; decorator arguments describe routing.
69	Wrap this view with the existing administrator-session guard.
70	Wrap this mutation with CSRF-token validation before its body runs.
71	Choose publish or remove from the submitted action, derive the actor from the session, pass the revision, convert validation/service failures into an editor response, and redirect with feedback on success.
72	Begin operations whose validation, network, or database failures will be handled below.

communications_routes.py | continued, lines 73-90

```

73         actor = session.get('admin_user', {}).get('email', 'admin')
74         revision = request.form.get('revision', '')
75         if request.form.get('action') == 'remove':
76             service.remove_announcement(actor, revision)
77         else:
78             service.save_announcement(request.form.get('notice_title'),
request.form.get('notice_body'),
79                                     request.form.get('notice_link'), actor, revision)
80     except ValueError as exc:
81         return admin_page(str(exc), 400)
82     except Exception:
83         app.logger.exception('Could not update official announcement')
84         return admin_page('The announcement could not be saved. Please try again.', 503)
85     flash('Announcement removed.' if request.form.get('action') == 'remove' else 'Official
announcement published.', 'success')
86     return redirect(url_for('admin_communications'))
87
88 @app.get('/admin/communications/students')
89 @admin_required
90 def notification_students():

```

Line	Reading note
73	Read authenticated session context with a safe fallback if a key is absent.
74	Read submitted form or query-string input; subsequent validation and bound parameters determine how it may be used.
75	Evaluate this condition and run the indented branch only when true. The expression shows the exact validation, result, or state being checked.
76	Continue the surrounding statement or SQL expression. Read with the preceding line and the function purpose above; indentation/parentheses determine its scope.
77	Use the alternate path when the preceding condition did not match.
78	Read submitted form or query-string input; subsequent validation and bound parameters determine how it may be used.

Line	Reading note
79	Read submitted form or query-string input; subsequent validation and bound parameters determine how it may be used.
80	Handle the named failure category rather than returning a false success; inspect the following response or raised error.
81	End this helper/view and provide its result or HTTP response to the caller. A transaction context exits before the result reaches the caller.
82	Handle the named failure category rather than returning a false success; inspect the following response or raised error.
83	Record a traceback on the server while the client receives a controlled message.
84	End this helper/view and provide its result or HTTP response to the caller. A transaction context exits before the result reaches the caller.
85	Store a user-facing feedback message for the page reached after a redirect.
86	End this helper/view and provide its result or HTTP response to the caller. A transaction context exits before the result reaches the caller.
88	Register the function below for this HTTP method/path; decorator arguments describe routing.
89	Wrap this view with the existing administrator-session guard.
90	Expose the bounded recipient search only to admins, with no-store caching and a generic failure response.

communications_routes.py | continued, lines 91-108

```

91         try:
92             response = jsonify(items=service.find_students(request.args.get('q'), admin_emails))
93             response.headers['Cache-Control'] = 'no-store'
94             return response
95         except Exception:
96             app.logger.exception('Could not search notification recipients')
97             return jsonify(error='Student search is temporarily unavailable.'), 503
98
99     @app.post('/admin/communications/send')
100     @admin_required
101     @csrf_protected
102     def admin_send_notification():
103         try:
104             count = service.send_notification(request.form.get('title'),
105 request.form.get('body'),
106 request.form.get('link_url'), request.form.get('audience'),
107 request.form.getlist('recipients'),
108 session.get('admin_user', {}).get('email', 'admin'),
109 request.form.get('request_key'), admin_emails)
110         except (ValueError, TypeError) as exc:
111             return admin_page(str(exc), 400)

```

Line	Reading note
91	Begin operations whose validation, network, or database failures will be handled below.
92	Read submitted form or query-string input; subsequent validation and bound parameters determine how it may be used.
93	Mark this response no-store so private inbox/student data is not deliberately cached.
94	End this helper/view and provide its result or HTTP response to the caller. A transaction context exits before the result reaches the caller.
95	Handle the named failure category rather than returning a false success; inspect the following response or raised error.
96	Record a traceback on the server while the client receives a controlled message.
97	End this helper/view and provide its result or HTTP response to the caller. A transaction context exits before the result reaches the caller.
99	Register the function below for this HTTP method/path; decorator arguments describe routing.
100	Wrap this view with the existing administrator-session guard.
101	Wrap this mutation with CSRF-token validation before its body runs.
102	Translate a normal form POST into service inputs. Repeated recipients fields require getlist. The sender comes from the session. Validation errors preserve the form; uncertain service failures preserve the retry key.
103	Begin operations whose validation, network, or database failures will be handled below.
104	Read submitted form or query-string input; subsequent validation and bound parameters determine how it may be used.
105	Read repeated form values for recipients, not only the first submitted value.
106	Read submitted form or query-string input; subsequent validation and bound parameters determine how it may be used.
107	Handle the named failure category rather than returning a false success; inspect the following response or raised error.
108	End this helper/view and provide its result or HTTP response to the caller. A transaction context exits before the result reaches the caller.

communications_routes.py | continued, lines 109-126

```

109         except Exception:
110             app.logger.exception('Could not send student notifications')
111             return admin_page('The send could not be confirmed. Retry this form safely; it will
not send twice.', 503)
112             flash(f'Notification sent to {count} student(s).', 'success')
113             return redirect(url_for('admin_communications'))

```

```

114
115     @app.get('/api/notifications')
116     @student_required
117     def student_notifications():
118         try:
119             before = int(request.args['before']) if 'before' in request.args else None
120             if before is not None and not 0 < before <= 9223372036854775807:
121                 raise ValueError()
122         except ValueError:
123             abort(400)
124         try:
125             payload = service.inbox(g.notification_user, before)
126             for item in payload['items']:

```

Line	Reading note
109	Handle the named failure category rather than returning a false success; inspect the following response or raised error.
110	Record a traceback on the server while the client receives a controlled message.
111	End this helper/view and provide its result or HTTP response to the caller. A transaction context exits before the result reaches the caller.
112	Store a user-facing feedback message for the page reached after a redirect.
113	End this helper/view and provide its result or HTTP response to the caller. A transaction context exits before the result reaches the caller.
115	Register the function below for this HTTP method/path; decorator arguments describe routing.
116	Resolve and authorize the student account before allowing this endpoint.
117	Validate the optional cursor as a positive signed-64-bit value, query this student's inbox, convert dates to ISO strings, and return no-store JSON. Do not return another user's data on malformed input.
118	Begin operations whose validation, network, or database failures will be handled below.
119	Read submitted form or query-string input; subsequent validation and bound parameters determine how it may be used.
120	Evaluate this condition and run the indented branch only when true. The expression shows the exact validation, result, or state being checked.
121	Signal failure to the caller; inside a transaction this prevents the partial writes from committing.
122	Handle the named failure category rather than returning a false success; inspect the following response or raised error.
123	Stop the request with the given HTTP error status; do not continue into the protected operation.
124	Begin operations whose validation, network, or database failures will be handled below.

Line	Reading note
125	Use the account ID resolved for this request, rather than trusting a browser-provided user ID.
126	Repeat the indented work for each value; the loop target names the current item.

communications_routes.py | continued, lines 127-144

```

127         item['created_at'] = item['created_at'].isoformat()
128         item['read_at'] = item['read_at'].isoformat() if item['read_at'] else None
129         response = jsonify(payload)
130         response.headers['Cache-Control'] = 'no-store'
131         return response
132     except Exception:
133         app.logger.exception('Could not load student inbox')
134         return jsonify(error='Notifications are temporarily unavailable. Please try
again.'), 503
135
136 @app.post('/api/notifications/<int:notification_id>/read')
137 @student_required
138 @csrf_protected
139 def read_notification(notification_id):
140     if not 0 < notification_id <= 9223372036854775807:
141         abort(400)
142     try:
143         found = service.mark_read(g.notification_user, notification_id=notification_id)
144     except Exception:

```

Line	Reading note
127	Convert a timestamp to a machine-readable string for JSON and browser date handling.
128	Convert a timestamp to a machine-readable string for JSON and browser date handling.
129	Compute/store this named value for the following steps. Its exact inputs and defaults are visible on this line.
130	Mark this response no-store so private inbox/student data is not deliberately cached.
131	End this helper/view and provide its result or HTTP response to the caller. A transaction context exits before the result reaches the caller.
132	Handle the named failure category rather than returning a false success; inspect the following response or raised error.
133	Record a traceback on the server while the client receives a controlled message.
134	End this helper/view and provide its result or HTTP response to the caller. A transaction context exits before the result reaches the caller.
136	Register the function below for this HTTP method/path; decorator arguments describe routing.
137	Resolve and authorize the student account before allowing this endpoint.
138	Wrap this mutation with CSRF-token validation before its body runs.

Line	Reading note
139	Reject out-of-range notification IDs, invoke the user-scoped read update, return 404 for no owned membership, and JSON success otherwise.
140	Evaluate this condition and run the indented branch only when true. The expression shows the exact validation, result, or state being checked.
141	Stop the request with the given HTTP error status; do not continue into the protected operation.
142	Begin operations whose validation, network, or database failures will be handled below.
143	Use the account ID resolved for this request, rather than trusting a browser-provided user ID.
144	Handle the named failure category rather than returning a false success; inspect the following response or raised error.

communications_routes.py | continued, lines 145-162

```

145         app.logger.exception('Could not mark notification read')
146         return jsonify(error='Could not update this notification.'), 503
147     if not found:
148         abort(404)
149     return jsonify(ok=True)
150
151 @app.post('/api/notifications/read-all')
152 @student_required
153 @csrf_protected
154 def read_all_notifications():
155     try:
156         through = int(request.form.get('through', ''))
157         if not 0 < through <= 9223372036854775807:
158             raise ValueError()
159     except ValueError:
160         abort(400)
161     try:
162         service.mark_read(g.notification_user, through=through)

```

Line	Reading note
145	Record a traceback on the server while the client receives a controlled message.
146	End this helper/view and provide its result or HTTP response to the caller. A transaction context exits before the result reaches the caller.
147	Evaluate this condition and run the indented branch only when true. The expression shows the exact validation, result, or state being checked.
148	Stop the request with the given HTTP error status; do not continue into the protected operation.
149	End this helper/view and provide its result or HTTP response to the caller. A transaction context exits before the result reaches the caller.
151	Register the function below for this HTTP method/path; decorator arguments describe routing.

Line	Reading note
152	Resolve and authorize the student account before allowing this endpoint.
153	Wrap this mutation with CSRF-token validation before its body runs.
154	Require a valid through boundary from the form and mark only the current user's unread memberships through that boundary. CSRF applies to this mutation too.
155	Begin operations whose validation, network, or database failures will be handled below.
156	Read submitted form or query-string input; subsequent validation and bound parameters determine how it may be used.
157	Evaluate this condition and run the indented branch only when true. The expression shows the exact validation, result, or state being checked.
158	Signal failure to the caller; inside a transaction this prevents the partial writes from committing.
159	Handle the named failure category rather than returning a false success; inspect the following response or raised error.
160	Stop the request with the given HTTP error status; do not continue into the protected operation.
161	Begin operations whose validation, network, or database failures will be handled below.
162	Use the account ID resolved for this request, rather than trusting a browser-provided user ID.

communications_routes.py | continued, lines 163-166

```

163         except Exception:
164             app.logger.exception('Could not mark notifications read')
165             return jsonify(error='Could not update your notifications.'), 503
166             return jsonify(ok=True)

```

Line	Reading note
163	Handle the named failure category rather than returning a false success; inspect the following response or raised error.
164	Record a traceback on the server while the client receives a controlled message.
165	End this helper/view and provide its result or HTTP response to the caller. A transaction context exits before the result reaches the caller.
166	End this helper/view and provide its result or HTTP response to the caller. A transaction context exits before the result reaches the caller.

A03 static/notifications.js

Student inbox controller. DOM references -> safe rendering -> fetch/pagination -> open/close events -> read actions -> visible-page polling. The line-by-line notes below explain every executable line.

Source: static/notifications.js | 87 total lines | SHA-256 3cbdb321852006e6...

Source lines 1-87

```

1  (() => {
2    const root = document.querySelector('.notification-center');
3    if (!root) return;
4    const bell = document.getElementById('notificationBell');
5    const panel = document.getElementById('notificationPanel');
6    const count = document.getElementById('notificationCount');
7    const list = document.getElementById('notificationList');
8    const status = document.getElementById('notificationStatus');
9    const more = document.getElementById('moreNotifications');
10   const retry = document.getElementById('retryNotifications');
11   const readAll = document.getElementById('readAllNotifications');
12   let nextBefore = null, latestId = null, loading = false;
13
14   const photo = document.querySelector('.profile-photo');
15   if (photo) {
16     photo.addEventListener('error', () => photo.remove());
17     if (photo.complete && !photo.naturalWidth) photo.remove();
18   }

```

Lines	Reading note
1-3	Create a private scope, locate the notification root, and exit harmlessly on a page without the feature.
4-11	Cache the bell, panel, count, list, status, paging, retry, and read-all controls. IDs must agree with dashboard.html.
12-12	nextBefore remembers the older-page cursor; latestId bounds read-all; loading prevents simultaneous list requests.
14-18	Find the Google image. Remove it on an error, including an error that completed before the listener was attached, revealing the initial fallback.

static/notifications.js | continued, lines 19-36

```

19   function close() { panel.hidden = true; bell.setAttribute('aria-expanded', 'false'); }
20   async function markRead(path, values = {}) {
21     const response = await fetch(path, { method: 'POST', body: new URLSearchParams({
22       _csrf_token: root.dataset.csrf, ...values }) });
23     if (!response.ok) throw new Error();
24   }
25   function row(item) {
26     const li = document.createElement('li'); li.className = `notification-item${item.read_at
27     ? '' : ' unread'}`;
28     const title = document.createElement('h3'); title.textContent = item.title;
29     const body = document.createElement('p'); body.textContent = item.body;
30     const time = document.createElement('time'); time.dateTime = item.created_at;
31     time.textContent = new Date(item.created_at).toLocaleString(); li.append(title, body,
32     time);
33     if (item.link_url) {
34       const link = document.createElement('a'); link.href = item.link_url;
35       link.textContent = 'View update →'; li.append(link);

```

```

32     }
33     if (!item.read_at) {
34         const button = document.createElement('button'); button.type = 'button';
        button.textContent = 'Mark as read';
35         button.addEventListener('click', async () => {
36             button.disabled = true;

```

Lines	Reading note
19-19	Close the panel and update its accessible expanded state together.
20-23	POST an acknowledgement using form-encoded fields and CSRF. Await completion and reject non-success HTTP statuses.
24-29	Create one list item, mark it unread if read_at is absent, insert title/body as text, and display a machine-readable and localized timestamp.
30-32	Add an optional link from the server-validated stored URL. Clicking it is distinct from the read action.
33-36	For unread messages only, create an acknowledgement button. Disable it during POST, reload on success, or restore it with feedback on failure.

static/notifications.js | continued, lines 37-54

```

37         try { await markRead(`${root.dataset.endpoint}/${item.id}/read`); await load();
    }
38         catch (_) { status.textContent = 'Could not mark this notification as read.
Please try again.'; button.disabled = false; }
39         });
40         li.append(button);
41     }
42     return li;
43 }
44 async function load(append = false, countOnly = false) {
45     if (loading) return;
46     loading = true; more.disabled = true; retry.hidden = true;
47     try {
48         const url = new URL(root.dataset.endpoint, location.href);
49         if (append && nextBefore) url.searchParams.set('before', nextBefore);
50         const response = await fetch(url, { cache: 'no-store' });
51         if (!response.ok) throw new Error();
52         const data = await response.json();
53         count.textContent = data.unread_count > 99 ? '99+' : String(data.unread_count);
54         count.hidden = !data.unread_count;

```

Lines	Reading note
37-40	For unread messages only, create an acknowledgement button. Disable it during POST, reload on success, or restore it with feedback on failure.
41-43	Return the completed DOM row to the list builder; end the row factory.
44-46	Start a page load unless another is active; temporarily disable paging and hide the previous retry control.
47-52	Construct a fresh endpoint URL, add before only for older-page requests, fetch without caching, check status, and decode JSON.

Lines	Reading note
53-54	Update the visual 99+ bubble, hide it at zero, and set the accessible label with the full unread count.

static/notifications.js | continued, lines 55-72

```

55         bell.setAttribute('aria-label', `Notifications, ${data.unread_count} unread`);
56         if (!countOnly) {
57             if (!append) { list.replaceChildren(); latestId = data.items[0]?.id || null; }
58             for (const item of data.items) list.append(row(item));
59             nextBefore = data.next_before; more.hidden = !nextBefore;
60             readAll.disabled = !data.unread_count || !latestId;
61             status.textContent = list.children.length ? '' : 'You're all caught up. New
official updates will appear here.';
62         }
63         bell.removeAttribute('title');
64     } catch (_) {
65         bell.title = 'Notifications are temporarily unavailable';
66         if (!countOnly) { status.textContent = 'Could not load notifications. Please try
again.'; retry.hidden = false; }
67     } finally { loading = false; more.disabled = false; }
68 }
69 bell.addEventListener('click', () => {
70     panel.hidden = !panel.hidden; bell.setAttribute('aria-expanded', String(!panel.hidden));
71     if (!panel.hidden) load();
72 });

```

Lines	Reading note
55-55	Update the visual 99+ bubble, hide it at zero, and set the accessible label with the full unread count.
56-57	Skip list changes during count-only polling. A full replacement clears old rows and captures the newest displayed ID.
58-61	Build rows, store the next cursor, decide whether older/read-all controls are usable, and show an empty-state message when appropriate.
62-63	Finish the full-list branch and clear a prior bell outage tooltip after any successful load.
64-68	On failure show an unavailable tooltip; full loads also show retry feedback. Always release the loading flag and paging button.
69-72	Toggle the panel on bell click, synchronize aria-expanded, and load a fresh list when opening.

static/notifications.js | continued, lines 73-87

```

73     document.getElementById('closeNotifications').addEventListener('click', () => { close();
bell.focus(); });
74     document.addEventListener('click', event => { if (!root.contains(event.target)) close(); });
75     document.addEventListener('keydown', event => { if (event.key === 'Escape' && !panel.hidden)
{ close(); bell.focus(); } });
76     more.addEventListener('click', () => load(true));
77     retry.addEventListener('click', () => load());
78     readAll.addEventListener('click', async () => {
79         readAll.disabled = true;
80         try { await markRead(`${root.dataset.endpoint}/read-all`, { through: latestId }); await
load(); }

```

```
81         catch (_) { status.textContent = 'Could not update your notifications. Please try
again.'; readAll.disabled = false; }
82     });
83     // Refresh the count without moving focus or replacing an open inbox while it is being read.
84     setInterval(() => { if (!document.hidden) load(false, true); }, 30000);
85     document.addEventListener('visibilitychange', () => { if (!document.hidden) load(false,
true); });
86     load();
87 }());
```

Lines	Reading note
73-75	Wire close, outside click, and Escape. Explicit dismissal returns keyboard focus to the bell.
76-77	Older uses append mode; retry requests a fresh newest page.
78-82	Disable read-all, POST the displayed latest-ID boundary, reload on success, or restore the control on failure.
83-87	Explain count-only polling, run it every 30 seconds only while visible, refresh on visibility return, perform the initial load, and close the private scope.

A04 static/admin-communications.js

Recipient picker and send-form state. A Map preserves selections across searches, hidden inputs serialize them, debounce/abort coordinates requests, and the backend still validates all submitted IDs.

Source: static/admin-communications.js | 80 total lines | SHA-256 1e40fe8f651096b8...

Source lines 1-80

```

1  (() => {
2    const form = document.getElementById('sendNotificationForm');
3    const audience = document.getElementById('notificationAudience');
4    const picker = document.getElementById('recipientPicker');
5    const results = document.getElementById('recipientResults');
6    const selectedBox = document.getElementById('selectedRecipients');
7    const status = document.getElementById('recipientStatus');
8    const sendButton = document.getElementById('sendNotificationButton');
9    const unavailable = sendButton.disabled;
10   const selected = new Map([...selectedBox.querySelectorAll('input')].map(input =>
    [input.value, `Student ${input.value}`]));
11   let searchController, timer;
12
13   function updateSelection() {
14     selectedBox.replaceChildren();
15     for (const [id, name] of selected) {
16       const input = document.createElement('input');
17       input.type = 'hidden'; input.name = 'recipients'; input.value = id;
18       const remove = document.createElement('button');
```

Lines	Reading note
1-8	Create a private scope and cache the form, audience, picker, result list, selected-chip box, status, and send button.
9-11	Remember server-side unavailable state, restore selected IDs from hidden inputs into a Map, and reserve abort/debounce handles.
13-14	Rebuild the selected-recipient presentation from the Map, which is the local source of selection state.
15-18	For each unique ID create a hidden form field and a removable chip. Its click removes the ID and refreshes all dependent controls.

static/admin-communications.js | continued, lines 19-36

```

19     remove.type = 'button'; remove.textContent = `${name} ×`;
20     remove.setAttribute('aria-label', `Remove ${name}`);
21     remove.addEventListener('click', () => { selected.delete(id); updateSelection(); });
22     selectedBox.append(input, remove);
23   }
24   results.querySelectorAll('input').forEach(input => { input.checked =
selected.has(input.value); });
25   document.getElementById('recipientCount').textContent = `${selected.size} students
selected`;
26   sendButton.textContent = audience.value === 'all' ? 'Send to all students' : `Send to
${selected.size} selected students`;
27   sendButton.disabled = unavailable || (audience.value === 'selected' && !selected.size);
28   }
29
30   async function search() {
```

```

31     searchController?.abort();
32     searchController = new AbortController();
33     status.textContent = 'Finding students...';
34     try {
35         const url = new URL(picker.dataset.searchUrl, location.href);
36         url.searchParams.set('q', document.getElementById('recipientSearch').value);

```

Lines	Reading note
19-22	For each unique ID create a hidden form field and a removable chip. Its click removes the ID and refreshes all dependent controls.
23-24	Finish chip generation and synchronize visible checkboxes with selections that may span multiple searches.
25-28	Show the selection count and correct send label; disable selected-mode sends with no recipients and preserve service-unavailable disabling.
30-33	Cancel the previous request, create a new abort signal, and announce the start of recipient search.
34-36	Build the server-provided search URL, attach the current q value, fetch with cancellation/no-store, reject redirects/errors, and parse results.

static/admin-communications.js | continued, lines 37-54

```

37     const response = await fetch(url, { signal: searchController.signal, cache:
    'no-store' });
38     if (!response.ok || response.redirected) throw new Error();
39     const data = await response.json();
40     results.replaceChildren();
41     for (const student of data.items) {
42         const id = String(student.id);
43         const label = document.createElement('label');
44         const checkbox = document.createElement('input');
45         checkbox.type = 'checkbox'; checkbox.value = id; checkbox.checked =
    selected.has(id);
46         checkbox.addEventListener('change', () => {
47             if (checkbox.checked && selected.size >= 500) {
48                 checkbox.checked = false; status.textContent = 'You can select up to 500
    students.'; return;
49             }
50             if (checkbox.checked) selected.set(id, student.name); else
    selected.delete(id);
51             updateSelection();
52         });
53         const copy = document.createElement('span');
54         copy.textContent = student.name;

```

Lines	Reading note
37-39	Build the server-provided search URL, attach the current q value, fetch with cancellation/no-store, reject redirects/errors, and parse results.
40-45	Clear only the search results, keeping the Map. Convert IDs to strings to match form keys and create a labeled checkbox per student.

Lines	Reading note
46-52	On changes enforce the 500-person UI limit, update the Map, and rebuild selection state. The backend has independent validation.
53-54	Build safe text nodes for display name, email, and department, attach them, and recover names for previously restored selected IDs.

static/admin-communications.js | continued, lines 55-72

```

55         const detail = document.createElement('small');
56         detail.textContent = `${student.email} · ${student.department} || 'Department not
    set'`;
57         copy.append(detail); label.append(checkbox, copy); results.append(label);
58         if (selected.has(id)) selected.set(id, student.name);
59     }
60     updateSelection();
61     status.textContent = data.items.length ? `Showing ${data.items.length} students.
    Search to find others.` : 'No matching active students.';
62     } catch (error) {
63         if (error.name !== 'AbortError') status.textContent = 'Could not load students.
    Search again or reload this page.';
64     }
65 }
66 audience.addEventListener('change', () => {
67     picker.hidden = audience.value !== 'selected';
68     document.getElementById('audienceHint').textContent = picker.hidden
69         ? 'All currently registered, active students. Administrators are excluded.'
70         : 'Choose students below. Selections remain when you change your search.';
71     updateSelection();
72     if (!picker.hidden) search();

```

Lines	Reading note
55-58	Build safe text nodes for display name, email, and department, attach them, and recover names for previously restored selected IDs.
59-61	Finish the result loop, refresh chips/buttons, and report the bounded result count or empty search.
62-65	Ignore deliberate cancellation errors; show feedback for real search failures and close the search function.
66-72	Switch between all and selected audiences, update explanatory text and button state, and search when the picker becomes visible.

static/admin-communications.js | continued, lines 73-80

```

73     });
74     document.getElementById('recipientSearch').addEventListener('input', () => {
75         clearTimeout(timer); searchController?.abort(); timer = setTimeout(search, 250);
76     });
77     form.addEventListener('submit', () => { sendButton.disabled = true; sendButton.textContent =
    'Sending...'; });
78     window.addEventListener('pageshow', updateSelection);
79     audience.dispatchEvent(new Event('change'));
80 }());

```

Lines	Reading note
73-73	Switch between all and selected audiences, update explanatory text and button state, and search when the picker becomes visible.
74-76	Debounce typing for 250ms and abort a stale in-flight request as soon as the input changes.
77-80	Disable the send button during submission, restore state on browser pageshow, initialize by dispatching the audience change, and end the scope.

A05 presence_service.py

Two database operations: throttled last_seen_at update, and a five-minute count/list query. Recent presence is intentionally separate from notification eligibility.

Source: presence_service.py | 25 total lines | SHA-256 66d706fd5a1fbddc...

Source lines 1-25

record_activity (lines 5-10): Use a database timestamp and a 30-second SQL throttle, excluding banned/admin users even if a client posts frequent heartbeats.

active_students (lines 13-25): Use a shared five-minute eligibility predicate for the total and newest 50 students. Convert timestamps for JSON and distinguish total membership from the bounded displayed list.

```

1  """Approximate signed-in student presence from visible website pages."""
2  from communications_service import transaction
3
4
5  def record_activity(user_id, admin_emails):
6      with transaction() as cur:
7          cur.execute('''UPDATE users SET last_seen_at=CURRENT_TIMESTAMP
8                          WHERE id=%s AND is_banned=FALSE AND NOT (LOWER(email)=ANY(%s))
9                          AND (last_seen_at IS NULL OR last_seen_at < CURRENT_TIMESTAMP - INTERVAL '30
seconds')''',
10                     (user_id, list(admin_emails)))
11
12
13  def active_students(admin_emails):
14      with transaction() as cur:
15          eligible = ''' FROM users WHERE is_banned=FALSE AND NOT (LOWER(email)=ANY(%s))
16                          AND last_seen_at >= CURRENT_TIMESTAMP - INTERVAL '5 minutes' '''
17          cur.execute('SELECT COUNT(*) AS total' + eligible, (list(admin_emails),))
18          total = cur.fetchone()['total']

```

presence_service.py | continued, lines 19-25

```

19      cur.execute('''SELECT id, email, COALESCE(NULLIF(preferred_name, ''),
NULLIF(google_name, ''), name, 'Student') AS name,
20      department, semester_no, last_seen_at''' + eligible + 'ORDER BY last_seen_at DESC,
id DESC LIMIT 50',
21      (list(admin_emails),))
22      items = [dict(row) for row in cur.fetchall()]
23      for item in items:
24          item['last_seen_at'] = item['last_seen_at'].isoformat()
25      return {'total': total, 'items': items, 'window_minutes': 5}

```

A06 static/student-activity.js

Small visible-page heartbeat. Capture the current script's data attributes, throttle attempts, post CSRF-protected activity, and ignore best-effort tracking failures so studying can continue.

Source: static/student-activity.js | 20 total lines | SHA-256 6ed3dfbf352eff55...

Source lines 1-20

```
1  (() => {
2      const script = document.currentScript;
3      let busy = false, lastAttempt = 0;
4      async function heartbeat() {
5          if (document.hidden || busy || Date.now() - lastAttempt < 30000) return;
6          busy = true; lastAttempt = Date.now();
7          try {
8              await fetch(script.dataset.endpoint, {
9                  method: 'POST', body: new URLSearchParams({ _csrf_token: script.dataset.csrf }),
10                 signal: AbortSignal.timeout(10000)
11             });
12         } catch (_) {
13             // Presence is best effort; a tracking outage must not interrupt studying.
14         } finally { busy = false; }
15     }
16     heartbeat();
17     setInterval(heartbeat, 60000);
18     document.addEventListener('visibilitychange', heartbeat);
```

static/student-activity.js | continued, lines 19-20

```
19     window.addEventListener('pageshow', heartbeat);
20 }());
```

A07 static/admin-presence.js

Admin presence display. Two count elements receive the same total; list visibility is persisted independently. A focused list is not rebuilt mid-navigation; timeouts and errors expose unavailable state.

Source: static/admin-presence.js | 65 total lines | SHA-256 f80af89138d74429...

Source lines 1-65

```

1  (() => {
2      const root = document.getElementById('activeStudents');
3      const count = document.getElementById('activeStudentCount');
4      const list = document.getElementById('activeStudentList');
5      const status = document.getElementById('activeStudentStatus');
6      const refresh = document.getElementById('refreshActiveStudents');
7      const toggle = document.getElementById('toggleActiveStudents');
8      const panelCount = document.getElementById('activeStudentPanelCount');
9      let latest = null;
10     let busy = false;
11     function updateSummary() {
12         if (!latest) return;
13         status.textContent = latest.total
14         ? `${latest.total} active students${list.hidden ? ' · student list hidden' :
latest.total > latest.items.length ? ' · showing the latest ${latest.items.length}` : ''}`
15         : 'No students have been active in the last 5 minutes.';
16     }
17     function setListHidden(hidden) {
18         list.hidden = hidden;

```

static/admin-presence.js | continued, lines 19-36

```

19         toggle.textContent = hidden ? 'Show students' : 'Hide students';
20         toggle.setAttribute('aria-expanded', String(!hidden));
21         updateSummary();
22     }
23     try { setListHidden(localStorage.getItem('bookshelf_hide_active_students') === 'true'); }
24     catch (_) { setListHidden(false); }
25     toggle.addEventListener('click', () => {
26         setListHidden(!list.hidden);
27         try { localStorage.setItem('bookshelf_hide_active_students', String(list.hidden)); }
28         catch (_) { /* The visibility control also works without browser storage. */ }
29     });
30     async function load() {
31         if (busy || document.hidden) return;
32         busy = true; refresh.disabled = true;
33         try {
34             const response = await fetch(root.dataset.endpoint, { cache: 'no-store', signal:
AbortSignal.timeout(10000) });
35             if (!response.ok || response.redirected) throw new Error();
36             const data = await response.json();

```

static/admin-presence.js | continued, lines 37-54

```

37         latest = data;
38         count.textContent = panelCount.textContent = data.total.toLocaleString();
39         updateSummary();
40         if (!list.contains(document.activeElement)) {
41             list.replaceChildren();
42             for (const student of data.items) {
43                 const item = document.createElement('li');
44                 const link = document.createElement('a');
45                 const url = new URL(root.dataset.userUrl, location.href);
url.searchParams.set('q', student.email);

```

```

46             link.href = url.href;
47             const name = document.createElement('strong'); name.textContent =
student.name;
48             const detail = document.createElement('span');
49             detail.textContent = `${student.department || 'Department not
set'}${student.semester_no ? ` · Semester ${student.semester_no}` : ''}`;
50             const time = document.createElement('time'); time.dateTime =
student.last_seen_at;
51             time.textContent = `Last seen ${new
Date(student.last_seen_at).toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' })}`;
52             link.append(name, detail, time); item.append(link); list.append(item);
53         }
54     }

```

static/admin-presence.js | continued, lines 55-65

```

55         } catch (_) {
56             latest = null;
57             count.textContent = panelCount.textContent = '-'; list.replaceChildren();
58             status.textContent = 'Active student information is temporarily unavailable. Try
refreshing.';
59         } finally { busy = false; refresh.disabled = false; }
60     }
61     refresh.addEventListener('click', load);
62     setInterval(load, 30000);
63     document.addEventListener('visibilitychange', () => { if (!document.hidden) load(); });
64     load();
65 })();

```

A08 pyq_service.py

Shared-paper metadata and file identity. No semester or branch filter exists here; pages are shared across students. Read the upload cleanup and delete-review limitations before using this as a reliability template.

Source: pyq_service.py | 139 total lines | SHA-256 a6a8bd5991b02adf...

Source lines 1-139

query (lines 9-20): Run parameterized SQL with dictionary rows in a short connection/transaction lifetime. one selects fetchone versus fetchall when the statement returns rows. Errors propagate after connection cleanup.

create_pyq_schema (lines 23-40): Create shared-paper metadata, uniqueness on content hash, year/code indexes, and the website-owned updated_at extension. It does not add the importer-only identity column.

validate_metadata (lines 43-54): Trim code/name, validate their lengths, and bound the year. Return normalized values for insertion rather than the original unvalidated form strings.

fingerprint (lines 57-71): Check the initial PDF marker, hash the stream in chunks while enforcing 50 MiB, reset the stream, and return hash plus byte count. Hashing identifies content equality, not subject identity.

import_pdf (lines 74-102): Look for an existing hash, upload a unique blob, and insert complete metadata. The unique SQL constraint catches concurrent duplicate uploads; failed/losing inserts attempt blob cleanup. Azure/SQL uncertainty remains a reliability limitation.

list_pyqs (lines 105-119): Combine allowed search/year predicates, count matches, and return a 50-row page ordered by year/id. Add a computed display title while preserving separate code/name fields.

get_pyq_summary (lines 122-128): Aggregate the shared table by year and derive the archive total and oldest/newest year. This includes rows created by the independent manager.

rename_pyq (lines 131-139): Normalize the subject name and update only the requested ID whose previous name matches the form. Preserve identity/file metadata and update the audit timestamp.

```

1  """Shared PYQ catalogue in the application's existing PostgreSQL database."""
2  import hashlib
3  from datetime import date
4
5  from psycopg2.extras import RealDictCursor
6  from database import get_connection
7
8
9  def query(sql, parameters=(), one=False):
10     conn = get_connection()
11     if conn is None:
12         raise RuntimeError("The PYQ database is unavailable.")
13     try:
14         with conn:
15             with conn.cursor(cursor_factory=RealDictCursor) as cur:
16                 cur.execute(sql, parameters)
17                 if cur.description:
18                     return cur.fetchone() if one else cur.fetchall()

```

pyq_service.py | continued, lines 19-36

```

19     finally:
20         conn.close()
21
22
23  def create_pyq_schema():
24     query("""
25         CREATE TABLE IF NOT EXISTS pyqs (
26             id BIGSERIAL PRIMARY KEY,
27             subject_code VARCHAR(100) NOT NULL,
28             subject_name VARCHAR(255) NOT NULL,

```

```

29         year INTEGER NOT NULL CHECK (year BETWEEN 1900 AND 9999),
30         blob_url TEXT NOT NULL,
31         original_filename TEXT NOT NULL,
32         file_size BIGINT NOT NULL CHECK (file_size > 0),
33         file_hash CHAR(64) NOT NULL UNIQUE,
34         uploaded_by VARCHAR(255) NOT NULL,
35         created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
36     );

```

pyq_service.py | continued, lines 37-54

```

37         CREATE INDEX IF NOT EXISTS idx_pyqs_year_id ON pyqs(year DESC, id DESC);
38         CREATE INDEX IF NOT EXISTS idx_pyqs_subject_code ON pyqs(subject_code);
39         ALTER TABLE pyqs ADD COLUMN IF NOT EXISTS updated_at TIMESTAMPTZ DEFAULT
CURRENT_TIMESTAMP;
40     """)
41
42
43 def validate_metadata(subject_code, subject_name, year):
44     code = str(subject_code or '').strip()
45     name = str(subject_name or '').strip()
46     try:
47         year = int(year)
48     except (ValueError, TypeError):
49         raise ValueError("Enter a valid exam year.")
50     if not 1900 <= year <= date.today().year + 1:
51         raise ValueError("Enter a valid exam year.")
52     if not code or len(code) > 100 or not name or len(name) > 255:
53         raise ValueError("Subject code and subject name are required (maximum 100 and 255
characters).")
54     return code, name, year

```

pyq_service.py | continued, lines 55-72

```

55
56
57 def fingerprint(stream):
58     stream.seek(0)
59     digest = hashlib.sha256()
60     size = 0
61     header = stream.read(1024)
62     if b'%PDF-' not in header:
63         raise ValueError("Only PDF files are accepted.")
64     stream.seek(0)
65     for chunk in iter(lambda: stream.read(1024 * 1024), b''):
66         size += len(chunk)
67         if size > 50 * 1024 * 1024:
68             raise ValueError("PDFs must be 50 MB or smaller.")
69         digest.update(chunk)
70     stream.seek(0)
71     return digest.hexdigest(), size
72

```

pyq_service.py | continued, lines 73-90

```

73
74 def import_pdf(stream, filename, subject_code, subject_name, year, actor):
75     from storage_agent import upload_file, delete_file
76     code, name, year = validate_metadata(subject_code, subject_name, year)
77     if not filename.lower().endswith('.pdf'):
78         raise ValueError("Only PDF files are accepted.")
79     digest, size = fingerprint(stream)
80     existing = query('SELECT id FROM pyqs WHERE file_hash = %s', (digest,), one=True)
81     if existing:

```

```

82         return existing['id'], False
83     result = upload_file(stream, filename, 'pyq')
84     if not result or not result.get('blob_url'):
85         raise RuntimeError("The PDF could not be uploaded to Azure.")
86     url = result['blob_url']
87     try:
88         row = query("""INSERT INTO pyqs
89                     (subject_code, subject_name, year, blob_url, original_filename,
90                      file_size, file_hash, uploaded_by)

```

pyq_service.py | continued, lines 91-108

```

91         VALUES (%s,%s,%s,%s,%s,%s,%s,%s)
92         ON CONFLICT (file_hash) DO NOTHING RETURNING id""",
93         (code, name, year, url, filename, size, digest, actor), one=True)
94     except Exception:
95         if not delete_file(url):
96             raise RuntimeError("Database save failed; Azure cleanup also failed for " + url)
97         raise
98     if row:
99         return row['id'], True
100     if not delete_file(url):
101         raise RuntimeError("Duplicate detected; Azure cleanup failed for " + url)
102     return None, False
103
104
105 def list_pyqs(search='', year=None, page=1):
106     conditions, params = [], []
107     if search:
108         pattern = '%' + search.replace('\', '\\').replace('%', '\\%').replace('_', '\\_') +
109         '%'

```

pyq_service.py | continued, lines 109-126

```

109         conditions.append('(subject_code ILIKE %s OR subject_name ILIKE %s)')
110         params.extend([pattern, pattern])
111     if year:
112         conditions.append('year = %s')
113         params.append(year)
114     where = ' WHERE ' + ' AND '.join(conditions) if conditions else ''
115     total = query('SELECT COUNT(*) AS total FROM pyqs' + where, params, one=True)['total']
116     rows = query('SELECT *, subject_code || \' - \' || subject_name AS title FROM pyqs' +
117                 where + ' ORDER BY year DESC, id DESC LIMIT 50 OFFSET %s',
118                 [*params, (page - 1) * 50])
119     return rows, total
120
121
122 def get_pyq_summary():
123     """Read the shared archive, including papers saved by the independent uploader."""
124     rows = query('SELECT year, COUNT(*) AS total FROM pyqs GROUP BY year ORDER BY year DESC')
125     years = [row['year'] for row in rows]
126     return {'total': sum(row['total'] for row in rows), 'years': years,

```

pyq_service.py | continued, lines 127-139

```

127         'oldest_year': min(years) if years else None,
128         'newest_year': max(years) if years else None}
129
130
131 def rename_pyq(paper_id, subject_name, expected_name):
132     """Rename one paper without changing its file, code, year or import identity."""
133     name = ' '.join(str(subject_name or '').split())
134     if not name or len(name) > 255 or any(ord(character) < 32 for character in name):
135         raise ValueError('Enter a subject name between 1 and 255 characters.')

```

```
136     row = query('''UPDATE pyqs SET subject_name=%s, updated_at=CURRENT_TIMESTAMP
137         WHERE id=%s AND subject_name=%s RETURNING id''',
138         (name, paper_id, expected_name), one=True)
139     return bool(row)
```

A09 app.py

Main request orchestration. Changed regions connect the existing login/admin infrastructure to PYQs, resource catalogues, awards, announcements, and notification route registration. Existing access/upload functions are explained in the teaching chapters.

Source: app.py | 845 total lines | SHA-256 7a964bb136b7d6cc...

Source lines 1-8

```
1 import os
2 import secrets
3 import zipfile
4 import communications_service
5 from communications_routes import register_routes as register_communications_routes
6 from collections.abc import Mapping
7 from functools import wraps
8 from pathlib import Path
```

Unchanged source lines 9-28 omitted.

Source lines 29-42

```
29     list_warnings,
30     list_warnings_for_users,
31     search_users,
32     set_contributor_badge,
33     unban_user,
34 )
35 from storage_agent import upload_file, delete_file
36 from pyq_service import create_pyq_schema, import_pdf, list_pyqs, get_pyq_summary, rename_pyq,
37     query as pyq_query
38 from db_agent import (
39     add_resource, get_resources, get_resource_by_id,
40     update_resource_status, delete_resource, get_resource_stats, get_resource_catalogue
41 )
42 from werkzeug.exceptions import RequestEntityTooLarge
43 from werkzeug.middleware.proxy_fix import ProxyFix
```

Unchanged source lines 43-65 omitted.

Source lines 66-72

```
66     ".pdf", ".doc", ".docx", ".ppt", ".pptx",
67     ".png", ".jpg", ".jpeg", ".webp",
68 }
69 ALLOWED_CATEGORIES = {"assignment", "book", "notes"}
70 ALLOWED_BRANCHES = {"cse", "it", "ece", "ee", "me", "ce"}
71 ALLOWED_SEMESTERS = {str(value) for value in range(1, 9)}
72
```

Unchanged source lines 73-134 omitted.

Source lines 135-145

```

135 # Verify database connection and initialize tables on startup
136 check_connection()
137 if create_tables():
138     create_pyq_schema()
139     try:
140         communications_service.create_schema()
141     except Exception:
142         app.logger.exception('Could not initialize announcements and notifications')
143     try:
144         create_admin_user_schema()
145     except AdminUserServiceError:

```

Unchanged source lines 146-233 omitted.

Source lines 234-242

callback (lines 186-240): Consume Google's OAuth response, normalize and verify the email, choose admin versus student session state, and retain a bounded HTTPS picture URL. Earlier authentication scaffolding is included as context.

```

234         flash("This account has been banned. Contact an administrator for help.", "error")
235         return redirect(url_for('login'))
236
237     picture = str(user_info.get('picture') or '')
238     session['user'] = {'email': email, 'name': name,
239                        'picture': picture if picture.startswith('https://') and len(picture) <=
2000 else ''}
240     return redirect(url_for('dashboard'))
241
242 @app.route('/dashboard')

```

Unchanged source lines 243-260 omitted.

Source lines 261-274

dashboard (lines 243-272): Require a student account, reject bans, load warnings and the active banner, and render the dashboard with both session/photo and profile/badge data.

```

261     except (AdminUserServiceError, AdminUserValidationError):
262         app.logger.exception("Could not load warnings for user %s", profile['id'])
263
264     announcement = None
265     try:
266         announcement = communications_service.announcement()
267     except Exception:
268         app.logger.exception('Could not load official announcement')
269     return render_template(
270         'dashboard.html', user=user, profile=profile, user_warnings=warnings,
271         announcement=announcement,
272     )
273
274 @app.route('/logout')

```

Unchanged source lines 275-369 omitted.

Source lines 370-387

admin_dashboard (lines 369-405): Combine ordinary resource statistics with shared PYQ totals, load bounded user matches and warning histories, and render management panels. Each optional service failure gets its own visible feedback.

```

370 # Fetch all resources and stats for the dashboard
371 resources = get_resources()
372 stats = get_resource_stats()
373 pyq_summary = None
374 try:

```

```

375         pyq_summary = get_pyq_summary()
376         pyq_count = pyq_summary['total']
377         stats['total'] = stats.get('total', 0) + pyq_count
378         counts = stats.setdefault('by_category', {})
379         counts['pyq'] = counts.get('pyq', 0) + pyq_count
380         statuses = stats.setdefault('by_status', {})
381         statuses['verified'] = statuses.get('verified', 0) + pyq_count
382     except Exception:
383         app.logger.exception('Could not load PYQ statistics')
384         flash('PYQ statistics are temporarily unavailable.', 'error')
385     try:
386         users = search_users(request.args.get('q', ''))
387     except AdminUserValidationError as error:

```

Unchanged source lines 388-400 omitted.

Source lines 401-407

admin_dashboard (lines 369-405): Combine ordinary resource statistics with shared PYQ totals, load bounded user matches and warning histories, and render management panels. Each optional service failure gets its own visible feedback.

```

401         flash("Warning histories could not be loaded.", "error")
402     return render_template(
403         'admin-dashboard.html', resources=resources, stats=stats, users=users,
404         warnings_by_user=warnings_by_user, pyq_summary=pyq_summary,
405     )
406
407

```

Unchanged source lines 408-424 omitted.

Source lines 425-446

admin_contributor_badge (lines 431-441): Keep the admin's search context, validate/apply the award through the service, report not-found or errors, and redirect to user management. The badge does not grant admin access.

```

425     return _moderation_redirect()
426
427
428 @app.route('/admin/users/<int:user_id>/badge', methods=['POST'])
429 @admin_required
430 @csrf_protected
431 def admin_contributor_badge(user_id):
432     destination = url_for('admin_dashboard', panel='users', q=request.form.get('q', '')[ :200])
433     try:
434         updated = set_contributor_badge(user_id, request.form.get('badge'))
435     except (AdminUserValidationError, AdminUserServiceError) as error:
436         if isinstance(error, AdminUserServiceError):
437             app.logger.exception('Could not set contributor badge for user %s', user_id)
438             flash(str(error), 'error')
439             return redirect(destination)
440         flash('Contributor badge saved.' if updated else 'User not found.', 'success' if updated
441             else 'error')
442     return redirect(destination)
443
444

```

app.py | continued, lines 443-446

```

443
444 @app.route('/admin/users/<int:user_id>/unban', methods=['POST'])
445 @admin_required
446 @csrf_protected

```

Unchanged source lines 447-657 omitted.

Source lines 658-665

```

658     Only returns verified resources to students.
659     """
660     category = request.args.get('category')
661     if category == 'pyq':
662         return api_pyqs()
663     branch = request.args.get('branch')
664     semester = request.args.get('semester')
665

```

Unchanged source lines 666-675 omitted.

Source lines 676-795

api_pyqs (lines 683-702): Validate page/year, call the service, and expose a public field allowlist. Internal hash/uploader fields remain excluded from this JSON endpoint.

_pyq_catalogue (lines 705-721): Load papers and year summary for either the public or admin template; validate navigation inputs and return a service-unavailable response on failure.

admin_pyqs (lines 726-727): Require administrator access before calling the shared catalogue helper with the admin template flag.

admin_rename_pyq (lines 733-750): Call the optimistic subject-name update, report conflicts, and preserve sanitized search/year/page filters in the redirect.

admin_upload_pyq (lines 756-772): Accept one PDF, strip directory components from its display filename, derive the sender from the admin session, and delegate validation/upload/deduplication to import_pdf.

admin_delete_pyq (lines 778-792): Current implementation: fetch blob URL, delete the Azure blob, then delete the SQL row. The review reproduced a broken-row failure if SQL fails after blob deletion; do not treat this as a recommended deletion pattern.

```

676
677 @app.route('/pyqs')
678 def pyqs():
679     return _pyq_catalogue(False)
680
681
682 @app.route('/api/pyqs')
683 def api_pyqs():
684     try:
685         page = int(request.args.get('page', '1'))
686         year = int(request.args.get('year')) if request.args.get('year') else None
687         if not 1 <= page <= 1000000 or (year is not None and not 1900 <= year <= 9999):
688             raise ValueError()
689     except ValueError:
690         return jsonify(error='Invalid page or year.'), 400
691     try:
692         papers, total = list_pyqs(request.args.get('q', '').strip()[:200], year, page)
693         # Public results omit internal uploader and file hash fields.

```

app.py | continued, lines 694-711

```

694         items = [{key: paper[key] for key in
695                     ('id', 'title', 'subject_code', 'subject_name', 'year', 'blob_url')}
696                  for paper in papers]
697         for item in items:
698             item['category'] = 'pyq'
699         return jsonify(items=items, total=total, page=page, page_size=50)
700     except Exception:
701         app.logger.exception('Could not load PYQ API')
702         return jsonify(error='The PYQ catalogue is temporarily unavailable.'), 503
703
704
705 def _pyq_catalogue(admin):
706     search = request.args.get('q', '').strip()[:200]

```

```

707     try:
708         page = max(1, int(request.args.get('page', '1')))
709         year = int(request.args['year']) if request.args.get('year') else None
710         if page > 1000000 or (year is not None and not 1900 <= year <= 9999):
711             raise ValueError()

```

app.py | continued, lines 712-729

```

712     except ValueError:
713         abort(400, 'Invalid page or year.')
714     try:
715         papers, total = list_pyqs(search, year, page)
716         archive = get_pyq_summary()
717     except Exception:
718         app.logger.exception('Could not load PYQs')
719         return 'The PYQ catalogue is temporarily unavailable.', 503
720     return render_template('admin-pyqs.html' if admin else 'pyqs.html', papers=papers,
total=total, page=page,
721                             search=search, year=year, admin=admin, archive=archive)
722
723
724 @app.route('/admin/pyqs')
725 @admin_required
726 def admin_pyqs():
727     return _pyq_catalogue(True)
728
729

```

app.py | continued, lines 730-747

```

730 @app.route('/admin/pyqs/<int:paper_id>/rename', methods=['POST'])
731 @admin_required
732 @csrf_protected
733 def admin_rename_pyq(paper_id):
734     try:
735         saved = rename_pyq(paper_id, request.form.get('subject_name'),
request.form.get('expected_name'))
736         flash('Subject name updated.' if saved else 'This paper was removed or changed since you
opened it. Refresh and try again.',
737             'success' if saved else 'error')
738     except ValueError as error:
739         flash(str(error), 'error')
740     except Exception:
741         app.logger.exception('Could not rename PYQ %s', paper_id)
742         flash('The name could not be saved. Please try again.', 'error')
743     try:
744         page = max(1, min(1000000, int(request.form.get('page', '1'))))
745         year = int(request.form['filter_year']) if request.form.get('filter_year') else ''
746         if year and not 1900 <= year <= 9999:
747             year = ''

```

app.py | continued, lines 748-765

```

748     except ValueError:
749         page, year = 1, ''
750     return redirect(url_for('admin_pyqs', q=request.form.get('q', '')[:200], year=year,
page=page))
751
752
753 @app.route('/admin/pyqs/upload', methods=['POST'])
754 @admin_required
755 @csrf_protected
756 def admin_upload_pyq():
757     file = request.files.get('file_upload')

```

```

758     try:
759         if not file or not file.filename:
760             raise ValueError('Choose a PDF file.')
761         # Retain the filename for display; Azure gets a generated safe blob name.
762         filename = file.filename.replace('\\', '/').rsplit('/', 1)[-1]
763         _, created = import_pdf(file.stream, filename,
764                                request.form.get('subject_code'), request.form.get('subject_name'),
765                                request.form.get('year'), session['admin_user']['email'])

```

app.py | continued, lines 766-783

```

766         flash('PYQ uploaded.' if created else 'This PDF is already in the catalogue.',
767               'success')
768     except ValueError as error:
769         flash(str(error), 'error')
770     except Exception:
771         app.logger.exception('PYQ upload failed')
772         flash('PYQ upload failed. Please retry or check the server log.', 'error')
773     return redirect(url_for('admin_pyqs'))
774
775 @app.route('/admin/pyqs/<int:paper_id>/delete', methods=['POST'])
776 @admin_required
777 @csrf_protected
778 def admin_delete_pyq(paper_id):
779     try:
780         paper = pyq_query('SELECT blob_url FROM pyqs WHERE id = %s', (paper_id,), one=True)
781         if not paper:
782             flash('PYQ not found.', 'error')
783             return redirect(url_for('admin_pyqs'))

```

app.py | continued, lines 784-795

```

784         if not delete_file(paper['blob_url']):
785             flash('Cloud deletion failed; the record was kept for retry.', 'error')
786         else:
787             pyq_query('DELETE FROM pyqs WHERE id = %s', (paper_id,))
788             flash('PYQ deleted.', 'success')
789     except Exception:
790         app.logger.exception('PYQ deletion failed')
791         flash('PYQ deletion failed. Please retry.', 'error')
792     return redirect(url_for('admin_pyqs'))
793
794
795 @app.route('/contribute')

```

Unchanged source lines 796-797 omitted.

Source lines 798-844

books (lines 800-801): Map the public digital-books URL to the book category and common resource catalogue helper.

notes (lines 804-805): Map the public notes URL to the notes category and common resource catalogue helper.

assignments (lines 808-809): Map the public assignments URL to the assignment category and common resource catalogue helper.

_resource_catalogue (lines 812-838): Validate request filters, call the shared resource service, derive safe presentation labels for admin/contributor authorship, and render a consistent category page with an explicit outage state.

```

798
799 @app.route('/books')
800 def books():
801     return _resource_catalogue('book', 'Digital Books', 'books')
802
803 @app.route('/notes')
804 def notes():

```

```

805     return _resource_catalogue('notes', 'Notes', 'notes')
806
807 @app.route('/assignments')
808 def assignments():
809     return _resource_catalogue('assignment', 'Assignments', 'assignments')
810
811
812 def _resource_catalogue(category, title, endpoint):
813     branch = request.args.get('branch', '').strip().lower()
814     semester = request.args.get('semester', '').strip()
815     search = request.args.get('q', '').strip()[:200]

```

app.py | continued, lines 816-833

```

816     try:
817         page = int(request.args.get('page', '1'))
818         if not 1 <= page <= 1000000 or (branch and branch not in ALLOWED_BRANCHES) or (semester
and semester not in ALLOWED_SEMESTERS):
819             raise ValueError()
820     except ValueError:
821         abort(400, 'Invalid department, semester or page.')
822     error = None
823     try:
824         resources, total = get_resource_catalogue(category, branch or None, semester or None,
search, page)
825         for resource in resources:
826             uploader = normalize_email(resource.get('uploaded_by'))
827             resource['is_admin_upload'] = uploader == 'admin' or uploader in ADMIN_EMAILS
828             resource['uploader_label'] = (
829                 'Admin' if resource['is_admin_upload']
830                 else resource.get('contributor_name') or 'User'
831             )
832     except Exception:
833         app.logger.exception('Could not load the %s catalogue', category)

```

app.py | continued, lines 834-844

```

834         resources, total = [], 0
835         error = 'This library is temporarily unavailable. Please try again shortly.'
836     return render_template('resource-catalogue.html', title=title, endpoint=endpoint,
837         resources=resources, total=total, branch=branch, semester=semester, search=search,
838         page=page, error=error), 503 if error else 200
839
840
841 register_communications_routes(app, admin_required, csrf_protected, ADMIN_EMAILS)
842
843
844 if __name__ == '__main__':

```

Unchanged source lines 845-845 omitted.

A10 database.py

The schema adds a constrained contributor badge and nullable recent-activity timestamp/index. The selected-column list and tuple-to-dictionary key list must remain aligned, or a value could be assigned to the wrong field.

Source: database.py | 330 total lines | SHA-256 a92e0652b153aef8...

Unchanged source lines 1-23 omitted.

Source lines 24-35

```

24
25 USER_SELECT_COLUMNS = ""
26     id, email, name, google_name, preferred_name, department, urn, crn,
27     semester_no, profile_completed, is_banned, banned_at, created_at, updated_at,
    contributor_badge
28 ""
29 USER_RESULT_COLUMNS = (
30     "id", "email", "name", "google_name", "preferred_name", "department",
31     "urn", "crn", "semester_no", "profile_completed", "is_banned",
32     "banned_at", "created_at", "updated_at", "contributor_badge",
33 )
34
35

```

Unchanged source lines 36-97 omitted.

Source lines 98-108

```

98         ALTER TABLE users ADD COLUMN IF NOT EXISTS is_banned BOOLEAN NOT NULL DEFAULT FALSE;
99         ALTER TABLE users ADD COLUMN IF NOT EXISTS banned_at TIMESTAMPTZ;
100         ALTER TABLE users ADD COLUMN IF NOT EXISTS updated_at TIMESTAMPTZ DEFAULT
    CURRENT_TIMESTAMP;
101         ALTER TABLE users ADD COLUMN IF NOT EXISTS contributor_badge VARCHAR(20) NOT NULL
    DEFAULT ''
102         CHECK (contributor_badge IN ('', 'trusted', 'best_contributor'));
103         ALTER TABLE users ADD COLUMN IF NOT EXISTS last_seen_at TIMESTAMPTZ;
104         CREATE INDEX IF NOT EXISTS idx_users_last_seen ON users (last_seen_at DESC)
105         WHERE last_seen_at IS NOT NULL;
106
107         UPDATE users
108         SET google_name = COALESCE(NULLIF(google_name, ''), NULLIF(name, '')),
            split_part(email, '@', 1)),

```

Unchanged source lines 109-330 omitted.

A11 db_agent.py

Adds the shared, paginated verified resource catalogue and resolves contributor names/awards from users. Preserve bound SQL values and the verified predicate when changing filters.

Source: db_agent.py | 385 total lines | SHA-256 63867a2ac3def9f3...

Unchanged source lines 1-6 omitted.

Source lines 7-13

```

7 import psycpg2 as p
8 from dotenv import load_dotenv
9 import os
10 from psycpg2.extras import RealDictCursor
11
12 load_dotenv()
13

```

Unchanged source lines 14-33 omitted.

Source lines 34-83

get_resource_catalogue (lines 37-78): Filter verified ordinary resources, use parameterized literal search, paginate, and resolve contributor display name/current badge. Category and field filters select metadata rather than Azure folders.

```

34         return None
35
36
37 def get_resource_catalogue(category, branch=None, semester=None, search='', page=1):
38     """Read a fresh, paginated student catalogue from verified uploads."""
39     if category not in {'notes', 'assignment', 'book'}:
40         raise ValueError('Invalid resource category.')
41     where = ['category = %s', 'status = %s']
42     params = [category, 'verified']
43     if branch:
44         where.append('branch = %s')
45         params.append(branch)
46     if semester:
47         where.append('semester = %s')
48         params.append(int(semester))
49     if search:
50         term = '%' + search.replace('\'', '\\\'').replace('%', '\\%').replace('_', '\\_') + '%'
51         where.append('(title ILIKE %s OR subject_name ILIKE %s)')

```

db_agent.py | continued, lines 52-69

```

52         params.extend([term, term])
53         clause = ' WHERE ' + ' AND '.join(where)
54         conn = get_connection()
55         if conn is None:
56             raise RuntimeError('The resource library is unavailable.')
57         try:
58             with conn, conn.cursor(cursor_factory=RealDictCursor) as cur:
59                 cur.execute('SELECT COUNT(*) AS total FROM resources' + clause, params)
60                 total = cur.fetchone()['total']
61                 cur.execute('''SELECT id, title, category, branch, semester, subject_name,
62                               blob_url, file_name, file_size, created_at, uploaded_by,
63                               (SELECT COALESCE(NULLIF(BTRIM(u.preferred_name), ''),
64                                NULLIF(BTRIM(u.google_name), ''),
65                                NULLIF(BTRIM(u.name), ''))
66                               FROM users u
67                               WHERE LOWER(BTRIM(u.email)) = LOWER(BTRIM(resources.uploaded_by))
68                               ORDER BY u.id LIMIT 1) AS contributor_name,

```

```
69             (SELECT u.contributor_badge FROM users u
70                 WHERE LOWER(BTRIM(u.email)) = LOWER(BTRIM(resources.uploaded_by))
71                 AND u.is_banned=FALSE
72                 ORDER BY u.id LIMIT 1) AS contributor_badge
73             FROM resources''' + clause +
74             ' ORDER BY created_at DESC, id DESC LIMIT 50 OFFSET %s',
75             [*params, (page - 1) * 50])
76         return [dict(row) for row in cur.fetchall()], total
77     finally:
78         conn.close()
79
80
81 def create_resources_table():
82     """Creates the resources table if it does not exist."""
83     conn = get_connection()
```

Unchanged source lines 84-385 omitted.

A12 admin_user_service.py

Adds contributor_badge to admin search results and an explicit service operation for changing the award. The service remains independent of Flask responses and sessions.

Source: admin_user_service.py | 398 total lines | SHA-256 defca745bcd31234...

Unchanged source lines 1-161 omitted.

Source lines 162-168

```

162             u.created_at,
163             COALESCE(u.is_banned, FALSE) AS is_banned,
164             u.banned_at,
165             u.contributor_badge,
166             COUNT(w.id)::INTEGER AS warning_count,
167             MAX(w.created_at) AS latest_warning_at
168         FROM users AS u

```

Unchanged source lines 169-182 omitted.

Source lines 183-205

set_contributor_badge (lines 186-200): Validate a positive user ID and the limited award vocabulary; perform a parameterized UPDATE with updated_at, commit on success, roll back on error, and close resources.

```

183         connection.close()
184
185
186 def set_contributor_badge(user_id: Any, badge: str) -> bool:
187     """Set one admin-assigned recognition badge; an empty value removes it."""
188     user_id = _positive_user_id(user_id)
189     if badge not in ('', 'trusted', 'best_contributor'):
190         raise AdminUserValidationError('Choose No badge, Trusted, or Best Contributor.')
191     connection = _open_connection()
192     try:
193         with connection, connection.cursor() as cursor:
194             cursor.execute('UPDATE users SET contributor_badge=%s,
updated_at=CURRENT_TIMESTAMP
195                             WHERE id=%s RETURNING id'', (badge, user_id))
196             return cursor.fetchone() is not None
197     except Exception as exc:
198         raise AdminUserServiceError('Could not update the contributor badge.') from exc
199     finally:
200         connection.close()

```

admin_user_service.py | continued, lines 201-205

```

201
202
203 def set_user_banned(user_id: Any, is_banned: bool) -> bool:
204     """Set a user's ban state, returning ``False`` when the user does not exist."""
205

```

Unchanged source lines 206-398 omitted.

A13 storage_agent.py

Sanitizes the blob filename and adds a UUID prefix. It preserves the original filename separately in the PYQ metadata path. The existing upload call has no explicit PDF content type and deletion has the cross-service limitations described earlier.

Source: storage_agent.py | 153 total lines | SHA-256 a068d8aa8c438db1...

Unchanged source lines 1-4 omitted.

Source lines 5-11

```
5
6 import os
7 import uuid
8 from werkzeug.utils import secure_filename
9 from azure.storage.blob import BlobServiceClient
10 from dotenv import load_dotenv
11
```

Unchanged source lines 12-54 omitted.

Source lines 55-62

```
55
56     try:
57         # Generate a unique blob name to prevent collisions
58         unique_prefix = uuid.uuid4().hex
59         safe_filename = secure_filename(original_filename) or 'document.pdf'
60         blob_name = f"{unique_prefix}_{safe_filename}"
61
62         # Get the container client and upload
```

Unchanged source lines 63-153 omitted.

A14 templates/admin-communications.html

Normal POST forms carry CSRF, banner revision, and a persistent send key. The picker submits repeated recipients fields. The page renders service errors and disables unavailable actions.

Source: templates/admin-communications.html | 59 total lines | SHA-256 920292a735bb1601...

Source lines 1-59

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8"><meta name="viewport" content="width=device-width, initial-scale=1.0">
5      <title>Notices & notifications · Bookshelf Admin</title>
6      <link rel="stylesheet" href="{{ url_for('static', filename='admin-pyqs.css') }}">
7      <link rel="stylesheet" href="{{ url_for('static', filename='admin-communications.css') }}">
8  </head>
9  <body>
10 <main class="archive-shell">
11     <nav class="archive-nav"><a class="brand" href="{{ url_for('admin_dashboard') }}"><span
class="brand-mark">B</span> Bookshelf <span class="admin-label">ADMIN</span></a><a href="{{
url_for('admin_dashboard') }}">< Admin dashboard</a></nav>
12     <header class="archive-heading"><div><p class="eyebrow">STUDENT
COMMUNICATIONS</p><h1>Notices & notifications</h1><p class="muted">Keep students informed with
official updates from Bookshelf.</p></div></header>
13     {% for category, message in get_flashed_messages(with_categories=true) %}<p class="notice {{
category }}" role="status">{{ message }}</p>{% endfor %}
14     {% if error %}<p class="notice error" role="alert">{{ error }}</p>{% endif %}
15     <div class="communications-grid">
16         <section class="communications-card">
17             <span class="eyebrow">DASHBOARD BANNER</span><h2>Official announcement</h2>
18             <p class="muted">One announcement for everyone. Publishing replaces the current bar;
removing it hides the bar.</p>

```

templates/admin-communications.html | continued, lines 19-36

```

19         <p class="publication-state">{{ 'Published on the dashboard' if announcement and
announcement.active else 'No announcement is published' }}</p>
20         <form action="{{ url_for('admin_announcement') }}" method="post"
class="communications-form">
21             <input type="hidden" name="_csrf_token" value="{{ csrf_token() }}">
22             <input type="hidden" name="revision" value="{{ announcement.revision if
announcement else 0 }}">
23             <label>Title<input name="notice_title" maxlength="160" required value="{{
submitted.get('notice_title', announcement.title if announcement else '') }}" placeholder="e.g.
Library updates for this week"></label>
24             <label>Message<textarea name="notice_body" maxlength="2000" rows="5" required>{{
submitted.get('notice_body', announcement.body if announcement else '') }}</textarea></label>
25             <label>Link <span class="muted">(optional)</span><input name="notice_link"
maxlength="2000" value="{{ submitted.get('notice_link', announcement.link_url if announcement
else '') }}" placeholder="https://... or /pyqs"></label>
26             <p class="muted">This updates the bar only. Use the notification form to send an
inbox update.</p>
27             <div class="communications-actions"><button class="primary" name="action"
value="publish" {% if unavailable %}disabled{% endif %}>{{ 'Save announcement' if announcement
and announcement.active else 'Publish announcement' }}</button>
28             {% if announcement and announcement.active %}<button name="action"
value="remove" formnovalidate {% if unavailable %}disabled{% endif %}>Remove from
dashboard</button>{% endif %}</div>
29         </form>
30     </section>
31     <section class="communications-card">
32         <span class="eyebrow">STUDENT INBOX</span><h2>Send a notification</h2>

```

```

33         <p class="muted">Deliver an official message to the bell on each recipient's
dashboard.</p>
34         <form action="{{ url_for('admin_send_notification') }}" method="post"
class="communications-form" id="sendNotificationForm">
35             <input type="hidden" name="_csrf_token" value="{{ csrf_token() }}"><input
type="hidden" name="request_key" value="{{ request_key }}">
36             <label>Title<input name="title" required maxlength="160" value="{{
submitted.get('title', '') }}" placeholder="e.g. New study materials are available"></label>

```

templates/admin-communications.html | continued, lines 37-54

```

37         <label>Message<textarea name="body" required maxlength="2000" rows="5">{{
submitted.get('body', '') }}</textarea></label>
38         <label>Link <span class="muted">(optional)</span><input name="link_url"
maxlength="2000" value="{{ submitted.get('link_url', '') }}" placeholder="https://... or
/notes"></label>
39         <label>Send to<select name="audience" id="notificationAudience"><option
value="all">All students</option><option value="selected" {% if submitted.get('audience') ==
'selected' %}>Selected students</option></select></label>
40         <p class="muted" id="audienceHint">All currently registered, active students.
Administrators are excluded.</p>
41         <div id="recipientPicker" data-search-url="{{ url_for('notification_students')
}}" hidden>
42             <label>Find students<input type="search" id="recipientSearch"
maxlength="200" placeholder="Name, email, URN or CRN" autocomplete="off"></label>
43             <p id="recipientStatus" role="status"></p><div id="recipientResults"
class="recipient-results"></div>
44             <p><strong id="recipientCount">0 students selected</strong></p><div
id="selectedRecipients" class="selected-recipients">
45                 {% for recipient in submitted.getlist('recipients') %}<input
type="hidden" name="recipients" value="{{ recipient }}">{% endfor %}
46             </div>
47         </div>
48         <button class="primary" id="sendNotificationButton" {% if unavailable
%}>disabled{% endif %}>Send to all students</button>
49         <p class="muted">Notifications stay in the student's inbox. No email or browser
push is sent.</p>
50     </form>
51 </section>
52 </div>
53 <section class="communications-card sent-history"><h2>Sent notifications</h2><p
class="muted">Your latest 50 sends. Read counts show how many recipients marked the message as
read.</p>
54 <ul>{% for item in sent %}<li><div><strong>{{ item.title }}</strong><p
class="sent-message">{{ item.body }}</p><small>{{ item.created_at.strftime('%d %b %Y, %H:%M
UTC') }} · {{ 'All students' if item.audience == 'all' else 'Selected students'
}}</small></div><span>{{ item.recipients }} recipients · {{ item.read_count }}
read</span></li>{% else %}<li>No notifications sent yet.</li>{% endfor %}</ul>

```

templates/admin-communications.html | continued, lines 55-59

```

55     </section>
56 </main>
57 <script src="{{ url_for('static', filename='admin-communications.js') }}"></script>
58 </body>
59 </html>

```

A15 templates/dashboard.html

Imports the shared badge macro, adds the bell/photo and official banner, replaces navigation labels/icons, and wires branch/category controls plus the heartbeat include.

Source: templates/dashboard.html | 343 total lines | SHA-256 86c57f1fbf423916...

Source lines 1-5

```
1 <!DOCTYPE html>
2 {% from '_contributor_badge.html' import contributor_badge %}
3 <html lang="en">
4 <head>
5
```

Unchanged source lines 6-8 omitted.

Source lines 9-16

```
9
10 <link href="https://fonts.googleapis.com/css2?family=Canela:ital,wght@0,400;0,500;1,400&family=Inter:wght@400;500;600&display=swap" rel="stylesheet">
11 <link rel="stylesheet" href="{{ url_for('static', filename='dashboard.css') }}">
12 <link rel="stylesheet" href="{{ url_for('static', filename='contributor-badges.css') }}">
13 <link rel="stylesheet" href="{{ url_for('static', filename='notifications.css') }}">
14 </head>
15
16 {% set dashboard_user = profile if profile is defined and profile else user %}
```

Unchanged source lines 17-107 omitted.

Source lines 108-134

```
108 
109 </div>
110
111 <nav class="sidebar-nav" aria-label="Dashboard navigation">
112 <a href="#" class="nav-item active">
113 <svg class="dashboard-nav-icon" width="18" height="18" viewBox="0 0 24 24"
    fill="none" stroke="currentColor" stroke-width="1.7" stroke-linecap="round"
    stroke-linejoin="round" aria-hidden="true"><path d="m3 10 9-7 9 7v10H3V10Zm6 10v-7h6v7"/></svg>
114 <span>Overview</span>
115 </a>
116 <a href="{{ url_for('pyqs') }}" class="nav-item" aria-label="PYQ archive">
117 <svg class="dashboard-nav-icon" width="18" height="18" viewBox="0 0 24 24"
    fill="none" stroke="currentColor" stroke-width="1.7" stroke-linecap="round"
    stroke-linejoin="round" aria-hidden="true"><path d="M12 5v16M3 3c4 0 7 1 9 3 2-2 5-3 9-3v15c-4
    0-7 1-9 3-2-2-5-3-9-3V3Z"/></svg>
118 <span class="nav-label-desktop">PYQ archive</span><span
    class="nav-label-mobile" aria-hidden="true">PYQs</span>
119 </a>
120 <a href="{{ url_for('contribute') }}" class="nav-item"
    aria-label="Contribution">
121 <svg class="dashboard-nav-icon" width="18" height="18" viewBox="0 0 24 24"
    fill="none" stroke="currentColor" stroke-width="1.7" stroke-linecap="round"
    stroke-linejoin="round" aria-hidden="true"><rect x="3" y="3" width="18" height="18"
    rx="3"/><path d="M12 7v10M7 12h10"/></svg>
122 <span class="nav-label-desktop">Contribution</span><span
    class="nav-label-mobile" aria-hidden="true">Contribute</span>
123 </a>
124 </nav>
125
```

templates/dashboard.html | continued, lines 126-134

```

126         <div class="sidebar-footer">
127             <div style="position: relative;">
128                 <a href="#" class="nav-item" id="settingsBtn" aria-expanded="false"
129                 aria-controls="settingsPopup">
130                     <svg class="dashboard-nav-icon" width="18" height="18" viewBox="0 0 24
131                     24" fill="none" stroke="currentColor" stroke-width="1.7" stroke-linecap="round"
132                     stroke-linejoin="round" aria-hidden="true"><path d="m10 3 4 0 1 3 3 1 3 3v4l-3 3-3 1-1
133                     3h-4l-1-3-3-1-3-3v-4l3-3 3-1 1-3Z"/><circle cx="12" cy="12" r="3"/></svg>
134                     <span>Settings</span>
135                 </a>
136             </div>
137             <div id="settingsPopup" class="settings-popup hidden">
138                 <a href="mailto:support@bookshelf.com" class="settings-link">

```

Unchanged source lines 135-150 omitted.

Source lines 151-160

```

151 </div>
152 </div>
153
154 <a href="/logout" class="nav-item text-danger">
155     <svg class="dashboard-nav-icon" width="18" height="18" viewBox="0 0 24 24"
156     fill="none" stroke="currentColor" stroke-width="1.7" stroke-linecap="round"
157     stroke-linejoin="round" aria-hidden="true"><path d="M9 3H3v18h6M9 12h12m-5-5 5 5-5 5"/></svg>
158     <span>Log Out</span>
159 </a>
160

```

Unchanged source lines 161-165 omitted.

Source lines 166-190

```

166             <p class="subtitle" id="instruction-text">Select your department to
continue.</p>
167         </div>
168         <div class="user-profile">
169             <div class="notification-center" data-endpoint="{{
url_for('student_notifications') }}" data-csrf="{{ csrf_token() }}">
170                 <button type="button" id="notificationBell" class="notification-bell"
aria-label="Notifications" aria-expanded="false" aria-controls="notificationPanel">
171                     <svg width="22" height="22" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="1.7" stroke-linecap="round" stroke-linejoin="round"
aria-hidden="true"><path d="M18 8a6 6 0 0 0-12 0c0 7-3 7-3 9h18c0-2-3-2-3-9M10 21h4"/></svg>
172                     <span id="notificationCount" class="notification-count"
hidden></span>
173                 </button>
174                 <section id="notificationPanel" class="notification-panel"
aria-label="Notifications" hidden>
175                     <div class="notification-heading"><h2>Notifications</h2><button
type="button" id="closeNotifications" aria-label="Close notifications">x</button></div>
176                     <div class="notification-toolbar"><span>Official
updates</span><button type="button" id="readAllNotifications" disabled>Mark all
read</button></div>
177                     <p id="notificationStatus" role="status">Loading notifications...</p>
178                     <button type="button" id="retryNotifications" hidden>Try
again</button>
179                     <ul id="notificationList"></ul>
180                     <button type="button" id="moreNotifications" hidden>Load older
notifications</button>
181                 </section>
182             </div>
183             <div class="avatar" aria-label="{{ display_name }}">

```

templates/dashboard.html | continued, lines 184-190

```

184             <span>{{ display_name[0]|upper }}</span>
185             {% if user is defined and user.picture %}{%
endif %}
186         </div>
187         {{ contributor_badge(profile.contributor_badge if profile is defined and
profile else '', 'profile-contributor-badge') }}
188     </div>
189 </header>
190

```

Unchanged source lines 191-209 omitted.

Source lines 210-232

```

210             </section>
211             {% endif %}
212
213             {% if announcement %}
214             <section class="pyq-archive-card official-announcement"
215             aria-labelledby="officialAnnouncementTitle">
216                 <div>
217                     <span class="pyq-archive-label">Official announcement</span>
218                     <h2 id="officialAnnouncementTitle">{{ announcement.title }}</h2>
219                     <p>{{ announcement.body }}</p>
220                 </div>
221                 {% if announcement.link_url %}<a class="pyq-archive-action" href="{{
222                 announcement.link_url }}">View notice <span aria-hidden="true"></span></a>{% endif %}
223             </section>
224             {% endif %}
225
226             <div class="dashboard-flow">
227                 <div class="selection-step" id="step-dept">
228                     <div class="dept-squares">

```

templates/dashboard.html | continued, lines 228-232

```

228
229                 <div class="dept-card" data-dept="cse" role="button" tabindex="0">
230                     
232                     <div>
233                         <div class="card-badge" style="margin-bottom: 8px;">CSE</div>

```

Unchanged source lines 233-233 omitted.

Source lines 234-240

```

234                 </div>
235             </div>
236
237             <div class="dept-card" data-dept="it" role="button" tabindex="0">
238                 
240                 <div>
241                     <div class="card-badge" style="margin-bottom: 8px;">IT</div>

```

Unchanged source lines 241-241 omitted.

Source lines 242-265

```

242                 </div>
243             </div>
244
245             {% for code, name in [('ece', 'Electronics'), ('ee', 'Electrical'),
('me', 'Mechanical'), ('ce', 'Civil')] %}
246             <div class="dept-card" data-dept="{{ code }}" role="button"
tabindex="0">
247                 
248                 <div>
249                     <div class="card-badge" style="margin-bottom: 8px;">{{
code|upper }}</div>
250                     <h3>{{ name }}</h3>
251                 </div>
252             </div>
253             {% endfor %}
254         </div>
255     </div>
256
257     <div class="selection-step hidden" id="step-sem">
258         <h4 class="step-label">Select Semester</h4>
259         <div class="semester-grid">

```

templates/dashboard.html | continued, lines 260-265

```

260             {% for value in range(1, 9) %}
261             <button class="sem-card" type="button" data-semester="{{ value }}"><span
class="sem-prefix">Sem</span> {{ value }}</button>
262             {% endfor %}
263         </div>
264     </div>
265

```

Unchanged source lines 266-266 omitted.

Source lines 267-273

```

267         <h4 class="step-label">What are you looking for?</h4>
268         <div class="category-squares">
269
270             <div class="category-card" data-category="pyqs" data-url="{{
url_for('pyqs') }}" role="link" tabindex="0">
271                 
272                 <div>
273                     <div class="card-badge" style="margin-bottom: 8px;">01</div>

```

Unchanged source lines 274-274 omitted.

Source lines 275-289

```

275                 </div>
276             </div>
277
278                 <div class="category-card" data-category="books" data-url="{{
url_for('books') }}" role="link" tabindex="0">
279                     
280                     <div>
281                         <div class="card-badge" style="margin-bottom: 8px;">02</div>
282                         <h3>Digital Books</h3>
283                     </div>
284                 </div>
285
286                 <div class="category-card" data-category="assignments" data-url="{{
url_for('assignments') }}" role="link" tabindex="0">
287                     
288                     <div>
289                         <div class="card-badge" style="margin-bottom: 8px;">03</div>

```

Unchanged source lines 290-290 omitted.

Source lines 291-297

```

291                 </div>
292             </div>
293
294                 <div class="category-card" data-category="notes" data-url="{{
url_for('notes') }}" role="link" tabindex="0">
295                     
296                     <div>
297                         <div class="card-badge" style="margin-bottom: 8px;">04</div>

```

Unchanged source lines 298-308 omitted.

Source lines 309-315

```

309         </div>
310
311         <script src="{{ url_for('static', filename='dashboard.js') }}"></script>
312         <script src="{{ url_for('static', filename='notifications.js') }}"></script>
313
314         <script>
315             console.log("1. Script section reached!");

```

Unchanged source lines 316-334 omitted.

Source lines 335-343

```

335
336         }, 4000);
337     });
338 </script>
339
340
341 {% include '_student_activity.html' %}
342 </body>
343 </html>

```

A16 static/dashboard.js

Department/semester/category selection becomes navigation state. Reset stale choices, cancel earlier timers, reveal before scrolling, preserve reduced-motion and keyboard activation, then construct filter URLs.

Source: static/dashboard.js | 182 total lines | SHA-256 7e448a552311a648...

Unchanged source lines 1-34 omitted.

Source lines 35-61

```
35 const semStep = document.getElementById('step-sem');
36 const categoryStep = document.getElementById('step-category');
37 const instructionText = document.getElementById('instruction-text');
38 let selectedDepartment = '';
39 let selectedSemester = '';
40 let departmentRevealTimer;
41
42 function scrollToSelection(step) {
43   requestAnimationFrame(() => {
44     step.scrollIntoView({
45       behavior: window.matchMedia('(prefers-reduced-motion: reduce)').matches ? 'auto' :
46       'smooth',
47       block: 'start'
48     });
49   });
50 }
51 deptCards.forEach(card => {
52   card.addEventListener('click', () => {
```

static/dashboard.js | continued, lines 53-61

```
53     clearTimeout(departmentRevealTimer);
54     selectedDepartment = card.dataset.dept;
55     selectedSemester = '';
56     semCards.forEach(item => item.classList.remove('selected', 'dimmed'));
57     categoryStep.classList.add('hidden');
58     categoryStep.classList.remove('visible');
59     deptCards.forEach(c => {
60       c.classList.remove('selected');
61       c.classList.add('dimmed');
```

Unchanged source lines 62-64 omitted.

Source lines 65-86

```
65     card.classList.add('selected');
66
67     instructionText.style.opacity = 0;
68     departmentRevealTimer = setTimeout(() => {
69         instructionText.innerText = "Department selected. Now choose your semester.";
70         instructionText.style.opacity = 1;
71         semStep.classList.add('visible');
72         semStep.classList.remove('hidden');
73         scrollToSelection(semStep);
74     }, 300);
75 });
76 });
77
78 semCards.forEach(card => {
79     card.addEventListener('click', () => {
80         clearTimeout(departmentRevealTimer);
81         selectedSemester = card.dataset.semester;
82         const department = selectedDepartment;
```

static/dashboard.js | continued, lines 83-86

```
83     const semester = selectedSemester;
84     semCards.forEach(c => {
85         c.classList.remove('selected');
86         c.classList.add('dimmed');
```

Unchanged source lines 87-90 omitted.

Source lines 91-103

```
91
92     instructionText.style.opacity = 0;
93     setTimeout(() => {
94         if (selectedDepartment !== department || selectedSemester !== semester) return;
95         instructionText.innerText = "Semester selected. What do you need?";
96         instructionText.style.opacity = 1;
97         categoryStep.classList.add('visible');
98         categoryStep.classList.remove('hidden');
99
100         scrollToSelection(categoryStep);
101     }, 300);
102 });
103 });
```

Unchanged source lines 104-116 omitted.

Source lines 117-137

```
117         instructionText.innerText = "Loading your resources...";
118         instructionText.style.opacity = 1;
119     }, 300);
120     const target = new URL(card.dataset.url, window.location.href);
121     if (card.dataset.category !== 'pyqs') {
122         if (selectedDepartment) target.searchParams.set('branch', selectedDepartment);
123         if (selectedSemester) target.searchParams.set('semester', selectedSemester);
124     }
125     window.location.assign(target.href);
126 });
127 });
128
129 document.querySelectorAll('.dept-card, .category-card').forEach(card => {
130     card.addEventListener('keydown', event => {
131         if (event.key === 'Enter' || event.key === ' ') {
132             event.preventDefault();
133             card.click();
134         }
135     });
136 });
137
```

static/dashboard.js | continued, lines 135-137

```
135     });
136 });
137
```

Unchanged source lines 138-152 omitted.

Source lines 153-177

```
153 const settingsPopup = document.getElementById('settingsPopup');
154 const darkModeToggle = document.getElementById('darkModeToggle');
155
156 settingsBtn?.addEventListener('click', (e) => {
157     e.preventDefault();
158     settingsPopup.classList.toggle('hidden');
159     settingsBtn.setAttribute('aria-expanded',
160 String(!settingsPopup.classList.contains('hidden')));
161 });
162
163 document.addEventListener('click', (e) => {
164     if (settingsBtn && settingsPopup && !settingsBtn.contains(e.target) &&
165 !settingsPopup.contains(e.target)) {
166         settingsPopup.classList.add('hidden');
167         settingsBtn.setAttribute('aria-expanded', 'false');
168     }
169 });
170
171 if (localStorage.getItem('bookshelf_dark_mode') === 'true') {
172     document.body.classList.add('dark-mode');
173 }
174
```

static/dashboard.js | continued, lines 171-177

```
171     if (darkModeToggle) darkModeToggle.checked = true;
172 }
173
174 darkModeToggle?.addEventListener('change', (e) => {
175     if (e.target.checked) {
176         document.body.classList.add('dark-mode');
177         localStorage.setItem('bookshelf_dark_mode', 'true');
178     }
179 }
180
```

Unchanged source lines 178-179 omitted.

Source lines 180-182

```
180         localStorage.setItem('bookshelf_dark_mode', 'false');  
181     }  
182 });
```

A17 templates/_student_activity.html

Conditionally loads the heartbeat for a non-admin signed-in student and passes its route/CSRF through data attributes. The endpoint still authenticates independently.

Source: templates/_student_activity.html | 3 total lines | SHA-256 7f73b455d1a32fdf...

Source lines 1-3

```
1 {% if session is defined and session.get('user') and not session.get('is_admin') %}
2 <script src="{{ url_for('static', filename='student-activity.js') }}" data-endpoint="{{
  url_for('student_activity') }}" data-csrf="{{ csrf_token() }}"></script>
3 {% endif %}
```

A18 templates/_contributor_badge.html

Jinja macro returns award markup only for recognized values. SVG is decorative; text names the award. The extra class allows contextual profile styling without duplicated markup.

Source: templates/_contributor_badge.html | 13 total lines | SHA-256 e8b5837bf9bc7bfa...

Source lines 1-13

```

1 {% macro contributor_badge(badge, extra_class='') -%}
2 {% if badge == 'trusted' %}
3 <span class="contributor-badge contributor-badge-trusted {{ extra_class }}" title="Contributor
  recognised by Bookshelf administrators.">
4     <svg width="13" height="13" viewBox="0 0 24 24" fill="none" stroke="currentColor"
      stroke-width="1.7" stroke-linecap="round" stroke-linejoin="round" aria-hidden="true"
      focusable="false"><circle cx="12" cy="12" r="9"/><path d="m8 12 2.5 2.5L16 9"/></svg>
5     <span>Trusted</span>
6 </span>
7 {% elif badge == 'best_contributor' %}
8 <span class="contributor-badge contributor-badge-best {{ extra_class }}" title="Recognised by
  Bookshelf administrators for helpful contributions.">
9     <svg width="13" height="13" viewBox="0 0 24 24" fill="none" stroke="currentColor"
      stroke-width="1.7" stroke-linecap="round" stroke-linejoin="round" aria-hidden="true"
      focusable="false"><path d="m12 3 2.8 5.7 6.2 9-4.5 4.4 1.1 6.2-5.6-3-5.6 3 1.1-6.2L3
      9.6l6.2-.9L12 3Z"/></svg>
10    <span>Best Contributor</span>
11 </span>
12 {% endif %}
13 {%- endmacro %}
```

A19 static/contributor-badges.css

Base badge geometry, Trusted colours, and subdued Best Contributor glow. Only these badges use these classes; official Admin resource styling is defined separately.

Source: static/contributor-badges.css | 4 total lines | SHA-256 719b6a490875fcef...

Source lines 1-4

```
1 .contributor-badge { display: inline-flex; align-items: center; align-self: flex-start; gap:
  6px; max-width: 100%; box-sizing: border-box; padding: 5px 9px; border: 1px solid;
  border-radius: 999px; font-size: 10px; font-weight: 500; line-height: 1.4; letter-spacing:
  .01em; }
2 .contributor-badge svg { flex-shrink: 0; }
3 .contributor-badge-trusted { background: #24292c; border-color: #414b50; color: #abb9c1; }
4 .contributor-badge-best { background: #2c271e; border-color: #514735; color: #c1b08c;
  box-shadow: 0 0 10px rgba(201, 164, 88, .2), inset 0 0 5px rgba(201, 164, 88, .08); }
```

A20 static/notifications.css

Positions the bell/panel/photo and styles inbox states, banner wrapping, and profile-only award overrides. On mobile, the notification panel is fixed within viewport margins.

Source: static/notifications.css | 34 total lines | SHA-256 71d70487763d51f3...

Source lines 1-34

```

1 .user-profile { display: grid; grid-template-columns: auto auto; align-items: center; gap: 8px
  16px; flex-shrink: 0; }
2 .user-profile .avatar { justify-self: end; }
3 .profile-contributor-badge { grid-column: 1 / -1; justify-self: end; }
4 body:not(.dark-mode) .profile-contributor-badge.contributor-badge-trusted { background: #edf1f3;
  border-color: #cad3d8; color: #596d79; }
5 .profile-contributor-badge.contributor-badge-best { background: #eee8f7; border-color: #bba5d4;
  color: #614282; box-shadow: 0 0 10px rgba(134, 99, 177, .18), inset 0 0 5px rgba(134, 99, 177,
  .06); }
6 body.dark-mode .profile-contributor-badge.contributor-badge-best { background: #332741;
  border-color: #79618e; color: #ddc8f1; }
7 .user-profile .avatar { position: relative; overflow: hidden; flex-shrink: 0; }
8 .profile-photo { position: absolute; inset: 0; width: 100%; height: 100%; object-fit: cover; }
9 .notification-center { position: relative; }
10 .notification-bell { position: relative; width: 42px; height: 42px; display: grid; place-items:
  center; border: 1px solid #dfd4bf; border-radius: 50%; background: #fffaf0; color: #473922;
  cursor: pointer; }
11 .notification-bell:hover, .notification-bell[aria-expanded="true"] { background: #f9edce; }
12 .notification-count { position: absolute; top: -4px; right: -6px; min-width: 18px; padding: 2px
  4px; border: 2px solid #fff; border-radius: 20px; background: #a33d31; color: #fff; font: 600
  10px/1.3 sans-serif; }
13 .notification-panel { position: absolute; top: 54px; right: 0; width: min(390px, calc(100vw -
  32px)); max-height: min(600px, 75vh); overflow-y: auto; z-index: 50; padding: 20px; border: 1px
  solid #e3d9c7; border-radius: 18px; background: #fffdf8; color: #2c241a; box-shadow: 0 16px 50px
  #35281826; }
14 .notification-heading, .notification-toolbar { display: flex; justify-content: space-between;
  align-items: center; gap: 12px; }
15 .notification-heading h2 { font: 600 20px/1.4 sans-serif; margin: 0; }
16 .notification-panel button { font: inherit; font-size: 12px; background: #f5f0e5; color:
  #5e481f; padding: 7px 10px; border: 1px solid #e1d8c7; border-radius: 7px; cursor: pointer; }
17 .notification-panel button:disabled { opacity: .5; cursor: default; }
18 .notification-toolbar { margin: 18px 0; font-size: 12px; color: #746a5a; }

```

static/notifications.css | continued, lines 19-34

```

19 #notificationStatus { font-size: 13px; margin: 12px 0; }
20 #notificationList { padding: 0; margin: 0; list-style: none; }
21 .notification-item { padding: 16px 0; border-top: 1px solid #ece4d5; }
22 .notification-item.unread { border-left: 3px solid #aa812e; padding-left: 12px; }
23 .notification-item h3 { font-size: 14px; line-height: 1.5; overflow-wrap: anywhere; }
24 .notification-item p { font-size: 13px; margin: 8px 0; line-height: 1.6; white-space: pre-wrap;
  overflow-wrap: anywhere; }
25 .notification-item time { display: block; font-size: 11px; color: #7b705f; margin: 10px 0; }
26 .notification-item a { font-size: 13px; color: #795716; display: inline-block; margin-right:
  12px; }
27 .official-announcement { cursor: default; }
28 .official-announcement > div { min-width: 0; }
29 .official-announcement h2, .official-announcement p { overflow-wrap: anywhere; }
30 .official-announcement p { white-space: pre-wrap; }
31 .official-announcement .pyq-archive-action { color: inherit; text-decoration: none; }
32 .official-announcement .pyq-archive-action:hover { text-decoration: underline; }
33 .notification-center [hidden] { display: none !important; }
34 @media (max-width: 820px) { .user-profile { gap: 10px; } .top-header { gap: 12px; align-items:
  flex-start; } .top-header > div:first-child { min-width: 0; } .notification-panel { position:
  fixed; top: 85px; left: 16px; right: 16px; width: auto; max-height: calc(100dvh - 180px); } }

```

A21 templates/admin-dashboard.html

Links new admin sections, adds archive/presence statistics, award forms, and labeled responsive student cells. Existing ban/warn/search controls retain their IDs, form actions, and permission checks.

Source: templates/admin-dashboard.html | 671 total lines | SHA-256 10b4af8d90179e23...

Unchanged source lines 1-5 omitted.

Source lines 6-12

```

6      <title>Bookshelf Admin – Control Room</title>
7      <link rel="preconnect" href="https://fonts.googleapis.com">
8      <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
9      <link
    href="https://fonts.googleapis.com/css2?family=DM+Sans:wght@400;500;600;700&display=swap"
    rel="stylesheet">
10     <script>
11         (() => {
12             try {

```

Unchanged source lines 13-45 omitted.

Source lines 46-60

```

46         <button class="nav-btn" type="button" data-panel="library"
    onclick="switchPanel('library')">
47             <span class="nav-icon" aria-hidden="true">\u25A4</span><span>Resource
    library</span>
48         </button>
49         <a class="nav-btn pyq-nav-link" href="{{ url_for('admin_pyqs') }}">
50             <span class="nav-icon" aria-hidden="true">\u25A5</span><span>PYQ library</span>
51         </a>
52         <button class="nav-btn" type="button" data-panel="users"
    onclick="switchPanel('users')">
53             <span class="nav-icon" aria-hidden="true">\u25CE</span><span>User
    database</span>
54         </button>
55         <a class="nav-btn pyq-nav-link" href="{{ url_for('admin_communications') }}">
56             <span class="nav-icon" aria-hidden="true"><svg width="20" height="20" viewBox="0
    0 24 24" fill="none" stroke="currentColor" stroke-width="1.7" stroke-linecap="round"
    stroke-linejoin="round"><path d="M18 8a6 6 0 0 0-12 0c0 7-3 7-3 9h18c0-2-3-2-3-9M10
    21h4"/></svg></span><span>Notices & notifications</span>
57         </a>
58         <button class="nav-btn nav-btn-disabled" type="button" disabled>
59             <span class="nav-icon"
    aria-hidden="true">\u2197</span><span>Contributions</span><span class="badge">Soon</span>
60         </button>

```

Unchanged source lines 61-110 omitted.

Source lines 111-121

```

111         <span class="stat-number">{{ stats.get('total', 0) }}</span>
112         <span class="stat-note">Across the full library</span>
113     </div>
114     <a class="stat-card pyq-stat-link" href="{{ url_for('admin_pyqs') }}">
115         <span class="stat-label">PYQs</span>
116         <span class="stat-number">{{ '{:,}'.format(pyq_summary.total) if pyq_summary is
not none else '-' }}</span>
117         <span class="stat-note">Open shared exam archive →</span>
118     </a>
119     <div class="stat-card">
120         <span class="stat-label">Assignments</span>
121         <span class="stat-number">{{ stats.get('by_category', {}).get('assignment', 0)
}}</span>

```

Unchanged source lines 122-130 omitted.

Source lines 131-141

```

131         <span class="stat-number">{{ stats.get('by_status', {}).get('verified', 0)
}}</span>
132         <span class="stat-note">Ready for students</span>
133     </div>
134     <a class="stat-card active-students-stat" href="{{ url_for('admin_dashboard',
panel='users') }}"#activeStudents" title="Signed-in students seen in the last five minutes">
135         <span class="stat-label"><span class="presence-dot" aria-hidden="true"></span>
Active now</span>
136         <span class="stat-number" id="activeStudentCount">—</span>
137         <span class="stat-note">Students · last 5 minutes</span>
138     </a>
139 </section>
140
141 <!-- == PANEL 1: Upload Resources == -->

```

Unchanged source lines 142-153 omitted.

Source lines 154-160

```

154         <span class="card-icon" aria-hidden="true">↑</span>
155     <div>
156         <h3>Resource details</h3>
157         <p>Add an assignment, note, or book directly to the library. <a href="{{
url_for('admin_pyqs') }}">Upload and manage shared PYQs →</a></p>
158     </div>
159 </div>
160

```

Unchanged source lines 161-165 omitted.

Source lines 166-171

```

166         <label>Category</label>
167         <select name="category" required>
168             <option value="" disabled selected>Select Type...</option>
169             <option value="assignment">Assignment / Lab File</option>
170             <option value="book">Digital Book</option>
171             <option value="notes">Notes</option>

```

Unchanged source lines 172-239 omitted.

Source lines 240-246

```

240         <div>
241             <span class="section-kicker">Content operations</span>
242             <h2 class="header-title">Resource library</h2>
243             <p>Review availability, verification status, and course placement. <a
href="{{ url_for('admin_pyqs') }}">Manage shared PYQs </a></p>
244         </div>
245         <span class="panel-step">02</span>
246     </div>

```

Unchanged source lines 247-338 omitted.

Source lines 339-356

```

339         <span class="panel-step">03</span>
340     </div>
341
342     <section class="active-students-panel" id="activeStudents" data-endpoint="{{
url_for('admin_active_students') }}" data-user-url="{{ url_for('admin_dashboard', panel='users')
 }}">
343         <div class="active-students-heading">
344             <div class="active-students-overview"><strong class="presence-total"
id="activeStudentPanelCount"></strong><div><h3>Active students</h3><p>Seen in the last 5
minutes · updates every 30 seconds.</p></div></div>
345             <div class="active-students-controls">
346                 <button type="button" id="toggleActiveStudents" aria-expanded="true"
aria-controls="activeStudentList">Hide students</button>
347                 <button type="button" id="refreshActiveStudents">Refresh</button>
348             </div>
349         </div>
350         <p id="activeStudentStatus" role="status">Loading active students...</p>
351         <ul id="activeStudentList"></ul>
352     </section>
353
354     <div class="user-toolbar">
355         <form action="/admin-dashboard" method="GET" class="user-search-form"
role="search">
356             <label for="userSearch" class="sr-only">Search registered users</label>

```

Unchanged source lines 357-374 omitted.

Source lines 375-416

```

375
376     {% if users|default([],true) %}
377     <div class="table-wrapper user-table-wrapper">
378         <table class="resource-table user-table" role="table" aria-label="Registered
students">
379             <thead role="rowgroup">
380                 <tr role="row">
381                     <th role="columnheader" scope="col">Names</th>
382                     <th role="columnheader" scope="col">Account</th>
383                     <th role="columnheader" scope="col">Academic details</th>
384                     <th role="columnheader" scope="col">Warnings</th>
385                     <th role="columnheader" scope="col">Status</th>
386                     <th role="columnheader" scope="col">Actions</th>
387                 </tr>
388             </thead>
389             <tbody role="rowgroup">
390                 {% for managed_user in users %}
391                 <tr role="row">
392                     <td class="user-name-cell" role="cell"><span class="user-cell-label"
aria-hidden="true">Names</span>

```

templates/admin-dashboard.html | continued, lines 393-410

```

393             <span class="preferred-name">{{ managed_user.preferred_name or
'Not provided' }}</span>
394             <span class="name-caption">Entered name</span>
395             <span class="google-name">{{ managed_user.google_name or 'Not
provided' }}</span>
396             <span class="name-caption">Google name</span>
397             <form action="{{ url_for('admin_contributor_badge',
user_id=managed_user.id) }}" method="post" class="contributor-badge-form">
398                 <input type="hidden" name="_csrf_token" value="{{
csrf_token() }}">
399                 <input type="hidden" name="q" value="{{
request.args.get('q', '') }}">
400                 <label for="badge-{{ managed_user.id }}">Contributor
badge</label>
401                 <select name="badge" id="badge-{{ managed_user.id }}">
402                     <option value="">No badge</option>
403                     <option value="trusted"%> if
managed_user.contributor_badge == 'trusted' %> selected{% endif %}>Trusted</option>
404                     <option value="best_contributor"%> if
managed_user.contributor_badge == 'best_contributor' %> selected{% endif %}>Best
Contributor</option>
405                 </select>
406                 <button type="submit" class="moderation-btn">Save
badge</button>
407             </form>
408         </td>
409         <td role="cell"><span class="user-cell-label"
aria-hidden="true">Account</span>
410         <span class="user-email">{{ managed_user.email }}</span>

```

templates/admin-dashboard.html | continued, lines 411-416

```

411             <span class="user-meta">Joined {{
managed_user.created_at.strftime('%d %b %Y') if managed_user.created_at else '-' }}</span>
412         </td>
413         <td role="cell"><span class="user-cell-label"
aria-hidden="true">Academic details</span>
414             <span class="academic-primary">
415                 {{ managed_user.department|upper if managed_user.department
else 'Department -' }}
416                 {% if managed_user.semester_no %} · Sem {{
managed_user.semester_no }}{% endif %}

```

Unchanged source lines 417-417 omitted.

Source lines 418-424

```

418             <span class="user-meta">URN {{ managed_user.urn or '-' }}</span>
419             <span class="user-meta">CRN {{ managed_user.crn or '-' }}</span>
420         </td>
421         <td role="cell"><span class="user-cell-label"
aria-hidden="true">Warnings</span>
422             <span class="warning-count{% if managed_user.warning_count %}
has-warnings{% endif %}">
423                 {{ managed_user.warning_count or 0 }} warning{% if
managed_user.warning_count != 1 %}>s{% endif %}
424             </span>

```

Unchanged source lines 425-450 omitted.

Source lines 451-457

```

451                 </details>
452                 {% endif %}
453             </td>
454             <td role="cell"><span class="user-cell-label"
aria-hidden="true">Status</span>
455                 {% if managed_user.is_banned %}
456                 <span class="user-status user-status-banned">Banned</span>
457                 {% if managed_user.banned_at %}

```

Unchanged source lines 458-460 omitted.

Source lines 461-467

```

461                 <span class="user-status user-status-active">Active</span>
462                 {% endif %}
463             </td>
464             <td class="user-actions" role="cell"><span class="user-cell-label"
aria-hidden="true">Actions</span>
465                 <div class="user-action-row">
466                     {% if managed_user.is_banned %}
467                     <form action="/admin/users/{{ managed_user.id }}/unban"
method="POST" onsubmit="return confirm('Restore access for this user?');">

```

Unchanged source lines 468-527 omitted.

Source lines 528-538

```

528             if (panel) panel.classList.add('active');
529
530             // Highlight the correct nav button
531             const activeButton = Array.from(document.querySelectorAll('.sidebar
.nav-btn[data-panel]'))
532             .find(button => button.dataset.panel === panelName);
533             if (activeButton) {
534                 activeButton.classList.add('active');
535                 activeButton.setAttribute('aria-current', 'page');
536             }
537
538             document.body.classList.remove('sidebar-open');

```

Unchanged source lines 539-665 omitted.

Source lines 666-671

```

666         }
667     </script>
668
669     <script src="{{ url_for('static', filename='admin-presence.js') }}"></script>
670 </body>
671 </html>

```

A22 static/admin-dashboard.css

Admin-only design tokens, navigation, panels, controls, dark mode, and responsive user cards. New slate/blue colours replace the green-dominant theme. The source remains CSS: values affect appearance, not authorization.

Source: static/admin-dashboard.css | 550 total lines | SHA-256 fd47013c4ef120b0...

Source lines 1-48

```

1  :root {
2      --ink-950: #111827;
3      --ink-900: #182235;
4      --ink-800: #243247;
5      --ink-700: #45546b;
6      --slate-600: #536680;
7      --slate-500: #71829b;
8      --slate-300: #c4cedd;
9      --slate-200: #dce3ee;
10     --slate-100: #edf1f7;
11     --paper: #f4f6fa;
12     --surface: #ffffff;
13     --surface-muted: #f8fafc;
14     --line: #e0e6ef;
15     --line-strong: #c4cedc;
16     --text: #202c40;
17     --muted: #596a80;
18     --faint: #64748b;

```

static/admin-dashboard.css | continued, lines 19-36

```

19     --accent: #2563eb;
20     --accent-deep: #1d4ed8;
21     --accent-soft: #eaf1ff;
22     --success: #367558;
23     --success-soft: #dcebe2;
24     --warning: #a56d18;
25     --warning-soft: #f6ead0;
26     --danger: #a5443e;
27     --danger-soft: #f4ddda;
28     --info: #315f9e;
29     --info-soft: #e8f0fc;
30     --shadow-sm: 0 1px 2px rgba(15, 23, 42, 0.04), 0 5px 18px rgba(15, 23, 42, 0.04);
31     --shadow-md: 0 18px 45px rgba(15, 23, 42, 0.1);
32     --radius-sm: 9px;
33     --radius-md: 16px;
34     --radius-lg: 24px;
35     --sidebar-width: 278px;
36 }

```

static/admin-dashboard.css | continued, lines 37-48

```

37
38 .pyq-nav-link, .pyq-stat-link { text-decoration: none; color: inherit; }
39 .pyq-stat-link:hover, .pyq-stat-link:focus-visible { outline: 2px solid var(--slate-600);
40     outline-offset: 2px; }
41 * { box-sizing: border-box; margin: 0; padding: 0; }
42 html { background: var(--paper); }
43 body {
44     min-height: 100vh;
45     background: radial-gradient(circle at 91% 2%, rgba(37, 99, 235, 0.09), transparent 24rem),
46     var(--paper);
47     color: var(--text);
48     font-family: "DM Sans", system-ui, sans-serif;
49     -webkit-font-smoothing: antialiased;

```

Unchanged source lines 49-49 omitted.

Source lines 50-56

```
50 button, input, select, textarea { font: inherit; }
51 button, a, input, select, textarea, summary { -webkit-tap-highlight-color: transparent; }
52 button:focus-visible, a:focus-visible, input:focus-visible, select:focus-visible,
  textarea:focus-visible, summary:focus-visible {
53     outline: 3px solid rgba(37, 99, 235, 0.28);
54     outline-offset: 2px;
55 }
56 .app-shell { min-height: 100vh; }
```

Unchanged source lines 57-66 omitted.

Source lines 67-73

```
67     overflow-y: auto;
68     background: radial-gradient(circle at 0 0, rgba(255,255,255,.08), transparent 17rem),
  var(--ink-950);
69     color: #f5f0e6;
70     box-shadow: 12px 0 40px rgba(15, 23, 42, 0.09);
71 }
72 .brand-lockup { display: flex; align-items: center; gap: 13px; padding: 0 6px 34px; }
73 .brand-mark {
```

Unchanged source lines 74-79 omitted.

Source lines 80-94

```
80     border-radius: 13px 13px 13px 5px;
81     background: var(--accent);
82     color: #fff9ef;
83     font-family: "DM Sans", system-ui, sans-serif;
84     font-size: 27px;
85     font-weight: 600;
86     box-shadow: 0 10px 24px rgba(0,0,0,.18);
87 }
88 .brand-name, .brand-subtitle { display: block; }
89 .brand-name { font-family: "DM Sans", system-ui, sans-serif; font-size: 22px; line-height: 1;
  letter-spacing: .01em; }
90 .brand-subtitle { margin-top: 5px; color: rgba(235,241,237,.55); font-size: 12px; font-weight:
  700; letter-spacing: .16em; text-transform: uppercase; }
91 .nav-label { margin: 0 12px 11px; color: rgba(235,241,237,.38); font-size: 12px; font-weight:
  700; letter-spacing: .16em; text-transform: uppercase; }
92 .sidebar-nav { display: grid; gap: 7px; }
93 .nav-btn {
94     position: relative;
```

Unchanged source lines 95-103 omitted.

Source lines 104-110

```
104     background: transparent;
105     color: rgba(241,245,242,.68);
106     cursor: pointer;
107     font-size: 15px;
108     font-weight: 600;
109     text-align: left;
110     transition: background 180ms ease, color 180ms ease, transform 180ms ease;
```

Unchanged source lines 111-133 omitted.

Source lines 134-142

```

134     color: rgba(255,255,255,.8);
135     font-size: 15px;
136 }
137 .nav-btn.active .nav-icon { border-color: rgba(37,99,235,.5); background: rgba(37,99,235,.17);
    color: #93b8ff; }
138 .nav-btn-disabled { cursor: not-allowed; opacity: .42; }
139 .badge { padding: 3px 7px; border: 1px solid rgba(255,255,255,.12); border-radius: 999px; color:
    rgba(255,255,255,.65); font-size: 12px; letter-spacing: .06em; text-transform: uppercase; }
140 .sidebar-footer { display: grid; gap: 11px; margin-top: auto; padding-top: 28px; }
141 .theme-toggle {
142     display: grid;

```

Unchanged source lines 143-149 omitted.

Source lines 150-276

```

150     background: rgba(255,255,255,.04);
151     color: rgba(241,245,242,.72);
152     cursor: pointer;
153     font-size: 13px;
154     font-weight: 600;
155     text-align: left;
156     transition: background 180ms ease, color 180ms ease;
157 }
158 .theme-toggle:hover { background: rgba(255,255,255,.075); color: #fff; }
159 .theme-toggle-icon { display: grid; width: 24px; height: 24px; place-items: center; color:
    #93b8ff; font-size: 16px; }
160 .theme-toggle-track { position: relative; width: 32px; height: 18px; border-radius: 999px;
    background: rgba(255,255,255,.13); transition: background 180ms ease; }
161 .theme-toggle-track span { position: absolute; top: 3px; left: 3px; width: 12px; height: 12px;
    border-radius: 50%; background: rgba(255,255,255,.72); transition: transform 180ms ease,
    background 180ms ease; }
162 .theme-toggle[aria-pressed="true"] .theme-toggle-track { background: var(--accent); }
163 .theme-toggle[aria-pressed="true"] .theme-toggle-track span { background: #fff; transform:
    translateX(14px); }
164 .admin-profile { display: flex; min-width: 0; align-items: center; gap: 10px; padding: 12px;
    border: 1px solid rgba(255,255,255,.08); border-radius: 13px; background: rgba(255,255,255,.04);
    }
165 .admin-avatar { display: grid; width: 34px; height: 34px; flex: 0 0 34px; place-items: center;
    border-radius: 10px; background: var(--slate-200); color: var(--ink-900); font-family: "DM
    Sans", system-ui, sans-serif; font-size: 18px; }
166 .admin-profile-copy { min-width: 0; }
167 .admin-profile-copy strong, .admin-profile-copy small { display: block; overflow: hidden;
    text-overflow: ellipsis; white-space: nowrap; }

```

static/admin-dashboard.css | continued, lines 168-185

```

168 .admin-profile-copy strong { color: #f5f2e9; font-size: 14px; font-weight: 600; }
169 .admin-profile-copy small { margin-top: 3px; color: rgba(235,241,237,.43); font-size: 12px; }
170 .logout-btn { display: flex; align-items: center; justify-content: center; gap: 8px; min-height:
    40px; border-radius: 10px; color: rgba(235,241,237,.6); font-size: 14px; font-weight: 600;
    text-decoration: none; transition: background 180ms ease, color 180ms ease; }
171 .logout-btn:hover { background: rgba(165,68,62,.15); color: #f3aaa4; }
172
173 /* Main workspace */
174 .content { min-height: 100vh; margin-left: var(--sidebar-width); padding: 46px
    clamp(28px,4vw,68px) 70px; }
175 .workspace-header { display: flex; align-items: flex-end; justify-content: space-between; gap:
    24px; max-width: 1460px; margin: 0 auto 30px; }
176 .eyebrow, .section-kicker { display: block; color: var(--accent-deep); font-size: 12px;
    font-weight: 700; letter-spacing: .17em; text-transform: uppercase; }
177 .workspace-header h1 { margin-top: 8px; font-family: "DM Sans", system-ui, sans-serif;
    font-size: clamp(36px,4vw,48px); font-weight: 700; letter-spacing: -.035em; line-height: .96; }
178 .workspace-header p { margin-top: 11px; color: var(--muted); font-size: 15px; }
179 .workspace-status { display: inline-flex; flex: 0 0 auto; align-items: center; gap: 8px;
    margin-bottom: 4px; padding: 9px 13px; border: 1px solid var(--slate-200); border-radius: 999px;

```

```

    background: rgba(255,255,255,.72); color: var(--ink-700); font-size: 13px; font-weight: 600;
    backdrop-filter: blur(8px); }
180 .status-dot { width: 7px; height: 7px; border-radius: 50%; background: #4a9a70; box-shadow: 0 0
    0 4px rgba(74,154,112,.12); }
181
182 /* Feedback */
183 .flash-container { position: fixed; z-index: 100; top: 24px; right: 24px; display: grid; width:
    min(390px,calc(100vw - 48px)); gap: 9px; }
184 .flash-msg { display: flex; align-items: center; justify-content: space-between; gap: 16px;
    padding: 14px 16px; border: 1px solid var(--line); border-radius: 12px; background:
    var(--surface); box-shadow: var(--shadow-md); color: var(--text); font-size: 14px; transition:
    opacity 350ms ease, transform 350ms ease; }
185 .flash-success { border-left: 4px solid var(--success); }

```

static/admin-dashboard.css | continued, lines 186-203

```

186 .flash-error { border-left: 4px solid var(--danger); }
187 .flash-warning { border-left: 4px solid var(--warning); }
188 .flash-close { border: 0; background: transparent; color: var(--muted); cursor: pointer;
    font-size: 18px; }
189
190 /* Stats */
191 .stats-bar { display: grid; max-width: 1460px; margin: 0 auto 25px; grid-template-columns:
    repeat(6,minmax(0,1fr)); border: 1px solid var(--line); border-radius: var(--radius-md);
    overflow: hidden; background: rgba(255,255,255,.78); box-shadow: var(--shadow-sm); }
192 .stat-card { position: relative; display: flex; min-height: 132px; flex-direction: column;
    justify-content: center; padding: 22px 24px; border-right: 1px solid var(--line); transition:
    background 180ms ease; }
193 .stat-card:last-child { border-right: 0; }
194 .stat-card:hover { background: var(--surface); }
195 .stat-card::after { position: absolute; inset: auto 24px 0; height: 3px; border-radius: 3px 3px
    0 0; background: var(--slate-300); content: ""; opacity: 0; transform: scaleX(.5); transition:
    opacity 180ms ease, transform 180ms ease; }
196 .stat-card:hover::after, .stat-card-primary::after, .stat-verified::after { opacity: 1;
    transform: scaleX(1); }
197 .stat-card-primary::after { background: var(--accent); }
198 .stat-verified::after { background: var(--success); }
199 .stat-label { color: var(--muted); font-size: 12px; font-weight: 700; letter-spacing: .09em;
    text-transform: uppercase; }
200 .stat-number { margin-top: 6px; color: var(--ink-900); font-family: "DM Sans", system-ui,
    sans-serif; font-size: 35px; font-weight: 600; line-height: 1; }
201 .stat-note { margin-top: 8px; color: var(--faint); font-size: 12px; }
202
203 /* Panels */

```

static/admin-dashboard.css | continued, lines 204-221

```

204 .panel { display: none; max-width: 1460px; margin: 0 auto; animation: panel-in 260ms ease both;
    }
205 .panel.active { display: block; }
206 @keyframes panel-in { from { opacity: 0; transform: translateY(7px); } to { opacity: 1;
    transform: translateY(0); } }
207 .panel-heading { display: flex; align-items: flex-end; justify-content: space-between; gap:
    24px; margin: 35px 2px 18px; }
208 .header-title { margin-top: 5px; font-family: "DM Sans", system-ui, sans-serif; font-size:
    clamp(26px,3vw,32px); font-weight: 700; letter-spacing: -.025em; line-height: 1.1; }
209 .panel-heading p { margin-top: 7px; color: var(--muted); font-size: 14px; }
210 .panel-step { color: rgba(36,50,71,.14); font-family: "DM Sans", system-ui, sans-serif;
    font-size: 52px; line-height: .8; }
211
212 /* Upload form */
213 .card { max-width: 920px; padding: clamp(24px,4vw,42px); border: 1px solid var(--line);
    border-radius: var(--radius-lg); background: var(--surface); box-shadow: var(--shadow-sm); }
214 .card-intro { display: flex; align-items: center; gap: 14px; margin-bottom: 9px; padding-bottom:
    24px; border-bottom: 1px solid var(--line); }
215 .card-icon { display: grid; width: 46px; height: 46px; flex: 0 0 46px; place-items: center;
    border-radius: 14px; background: var(--accent-soft); color: var(--accent-deep); font-size: 22px;
    }

```

```

216 .card h3 { font-family: "DM Sans", system-ui, sans-serif; font-size: 21px; font-weight: 600; }
217 .card p { margin-top: 3px; color: var(--muted); font-size: 14px; }
218 .form-row { display: grid; grid-template-columns: 1fr 1fr; gap: 16px; }
219 .form-group { display: flex; flex-direction: column; }
220 label { display: block; margin: 18px 0 7px; color: var(--ink-800); font-size: 13px; font-weight:
      700; letter-spacing: .03em; }
221 .optional-tag { margin-left: 5px; color: var(--faint); font-size: 12px; font-weight: 600;
      text-transform: uppercase; }

```

static/admin-dashboard.css | continued, lines 222-239

```

222 select, input[type="text"], input[type="search"], textarea { width: 100%; border: 1px solid
      var(--line-strong); border-radius: var(--radius-sm); background: var(--surface-muted); color:
      var(--text); transition: border-color 160ms ease, box-shadow 160ms ease, background 160ms ease;
      }
223 select, input[type="text"], input[type="search"] { min-height: 44px; padding: 10px 13px;
      font-size: 14px; }
224 select { padding-right: 34px; appearance: none; background-image:
      linear-gradient(45deg,transparent 50%,var(--slate-600) 50%),
      linear-gradient(135deg,var(--slate-600) 50%,transparent 50%); background-position: calc(100% -
      17px) 19px, calc(100% - 12px) 19px; background-repeat: no-repeat; background-size: 5px 5px,5px
      5px; }
225 select:focus, input[type="text"]:focus, input[type="search"]:focus, textarea:focus {
      border-color: var(--accent); outline: none; background: #fff; box-shadow: 0 0 0 4px
      rgba(37,99,235,.11); }
226 input::placeholder, textarea::placeholder { color: #9aa29e; }
227 .file-drop-zone { position: relative; display: grid; min-height: 145px; margin-top: 8px;
      padding: 24px; place-items: center; border: 1.5px dashed var(--slate-300); border-radius: 14px;
      background: linear-gradient(135deg,rgba(237,241,247,.52),rgba(248,250,252,.55)); cursor:
      pointer; text-align: center; transition: border-color 180ms ease, background 180ms ease,
      transform 180ms ease; }
228 .file-drop-zone:hover, .file-drop-zone.drag-over { border-color: var(--accent); background:
      var(--accent-soft); transform: translateY(-1px); }
229 .file-drop-zone input[type="file"] { position: absolute; inset: 0; width: 100%; height: 100%;
      opacity: 0; cursor: pointer; }
230 .drop-icon { display: grid; width: 38px; height: 38px; margin: 0 auto 8px; place-items: center;
      border-radius: 50%; background: var(--ink-900); color: #fff; font-size: 18px; }
231 .drop-text p { margin: 0; color: var(--muted); font-size: 14px; }
232 .drop-text strong { color: var(--accent-deep); }
233 .drop-hint { display: block; margin-top: 5px; color: var(--faint); font-size: 12px; }
234 .file-preview { display: flex; align-items: center; justify-content: center; gap: 10px; }
235 .file-icon { font-size: 22px; }
236 .file-name { color: var(--ink-800); font-size: 14px; font-weight: 700; }
237 .file-size { padding: 3px 8px; border-radius: 999px; background: var(--slate-100); color:
      var(--slate-600); font-size: 12px; }
238 .hidden { display: none !important; }
239 .btn-submit { display: inline-flex; width: 100%; min-height: 49px; align-items: center;
      justify-content: center; margin-top: 24px; padding: 13px 22px; border: 0; border-radius: 11px;
      background: var(--accent); color: #fff; cursor: pointer; font-size: 14px; font-weight: 700;
      letter-spacing: .015em; box-shadow: 0 10px 22px rgba(37,99,235,.16); transition: transform 180ms
      ease, background 180ms ease, box-shadow 180ms ease; }

```

static/admin-dashboard.css | continued, lines 240-257

```

240 .btn-submit:hover:not(:disabled) { background: var(--accent-deep); box-shadow: 0 14px 28px
      rgba(37,99,235,.2); transform: translateY(-1px); }
241 .btn-submit:disabled { cursor: wait; opacity: .7; }
242 .btn-loading { display: inline-flex; align-items: center; gap: 8px; }
243 .btn-loading::before { width: 14px; height: 14px; border: 2px solid rgba(255,255,255,.3);
      border-top-color: #fff; border-radius: 50%; animation: spin 650ms linear infinite; content: "";
      }
244 @keyframes spin { to { transform: rotate(360deg); } }
245
246 /* Filters and tables */
247 .filter-bar, .user-toolbar { display: flex; align-items: center; gap: 9px; margin-bottom: 13px;
      padding: 10px; border: 1px solid var(--line); border-radius: 13px; background:
      rgba(255,255,255,.82); box-shadow: var(--shadow-sm); }
248 .filter-bar select { width: auto; min-width: 140px; min-height: 39px; background-color:

```

```

    var(--surface); font-size: 13px; }
249 .resource-count, .user-result-count { margin-left: auto; padding: 0 8px; color: var(--muted);
    font-size: 12px; font-weight: 700; letter-spacing: .05em; white-space: nowrap; text-transform:
    uppercase; }
250 .table-wrapper { border: 1px solid var(--line); border-radius: var(--radius-md); overflow-x:
    auto; background: var(--surface); box-shadow: var(--shadow-sm); }
251 .resource-table { width: 100%; min-width: 900px; border-collapse: collapse; }
252 .resource-table th { padding: 13px 15px; border-bottom: 1px solid var(--line); background:
    var(--surface-muted); color: var(--muted); font-size: 12px; font-weight: 700; letter-spacing:
    .09em; text-align: left; text-transform: uppercase; }
253 .resource-table td { padding: 15px; border-bottom: 1px solid #e8edf4; color: var(--text);
    font-size: 13px; vertical-align: middle; }
254 .resource-table tbody tr:last-child td { border-bottom: 0; }
255 .resource-table tbody tr { transition: background 150ms ease; }
256 .resource-table tbody tr:hover { background: #f8fafc; }
257 .td-title { max-width: 250px; }

```

static/admin-dashboard.css | continued, lines 258-275

```

258 .resource-link { display: block; overflow: hidden; color: var(--ink-800); font-size: 14px;
    font-weight: 700; text-decoration: none; text-overflow: ellipsis; white-space: nowrap; }
259 .resource-link:hover { color: var(--accent-deep); }
260 .td-filename { display: block; margin-top: 3px; overflow: hidden; color: var(--faint);
    font-size: 12px; text-overflow: ellipsis; white-space: nowrap; }
261 .td-date { color: var(--muted) !important; white-space: nowrap; }
262 .cat-badge, .status-badge, .branch-tag, .user-status, .warning-count { display: inline-flex;
    align-items: center; padding: 4px 8px; border-radius: 999px; font-size: 12px; font-weight: 700;
    letter-spacing: .04em; white-space: nowrap; }
263 .cat-pyq { background: #ebe4f6; color: #725495; }
264 .cat-assignment { background: var(--info-soft); color: var(--info); }
265 .cat-book { background: var(--warning-soft); color: var(--warning); }
266 .cat-notes { background: var(--success-soft); color: var(--success); }
267 .branch-tag { background: var(--slate-100); color: var(--slate-600); }
268 .status-verified, .user-status-active { background: var(--success-soft); color: var(--success);
    }
269 .status-unverified { background: var(--warning-soft); color: var(--warning); }
270 .user-status-banned { background: var(--danger-soft); color: var(--danger); }
271 .td-actions { min-width: 118px; white-space: nowrap; }
272 .td-actions form { display: inline !important; }
273 .action-btn { display: inline-flex; width: 29px; height: 29px; align-items: center;
    justify-content: center; margin-right: 3px; border: 1px solid var(--line); border-radius: 8px;
    background: var(--surface); color: var(--muted); cursor: pointer; font-size: 14px;
    text-decoration: none; transition: border-color 150ms ease, background 150ms ease, color 150ms
    ease, transform 150ms ease; }
274 .action-btn:hover { transform: translateY(-1px); }
275 .action-view:hover { border-color: #8ab2c1; background: var(--info-soft); color: var(--info); }

```

static/admin-dashboard.css | continued, lines 276-276

```

276 .action-toggle:hover { border-color: #d5b36d; background: var(--warning-soft); color:
    var(--warning); }

```

Unchanged source lines 277-279 omitted.

Source lines 280-314

```

280 .sr-only { position: absolute; width: 1px; height: 1px; padding: 0; margin: -1px; overflow:
    hidden; clip: rect(0,0,0,0); white-space: nowrap; border: 0; }
281 .user-search-form { display: flex; min-width: 0; flex: 1; align-items: center; gap: 8px; }
282 .user-search-form input[type="search"] { max-width: 520px; min-width: 220px; min-height: 39px;
    background: var(--surface); }
283 .user-search-btn, .user-clear-btn { display: inline-flex; min-height: 39px; align-items: center;
    justify-content: center; padding: 9px 14px; border-radius: 9px; font-size: 12px; font-weight:
    700; text-decoration: none; white-space: nowrap; }
284 .user-search-btn { border: 1px solid var(--ink-900); background: var(--accent); color: #fff;
    cursor: pointer; }
285 .user-search-btn:hover { background: var(--accent-deep); }
286 .user-clear-btn { border: 1px solid var(--line-strong); color: var(--muted); }
287 .user-clear-btn:hover { background: var(--surface); color: var(--text); }
288 .user-table { min-width: 1120px; }
289 .user-table td { vertical-align: top; }
290 .user-name-cell span, .user-email, .academic-primary, .user-meta { display: block; }
291 .preferred-name { color: var(--ink-900); font-size: 14px; font-weight: 700; }
292 .google-name { margin-top: 9px; color: var(--text); font-size: 12px; font-weight: 600; }
293 .name-caption { margin-top: 2px; color: var(--faint); font-size: 12px; font-weight: 700;
    letter-spacing: .08em; text-transform: uppercase; }
294 .user-email { max-width: 220px; color: var(--info); font-size: 12px; font-weight: 600;
    overflow-wrap: anywhere; }
295 .academic-primary { margin-bottom: 5px; color: var(--ink-800); font-size: 12px; font-weight:
    700; }
296 .user-meta { margin-top: 4px; color: var(--muted); font-size: 12px; white-space: nowrap; }
297 .warning-count { background: var(--slate-100); color: var(--slate-600); }

```

static/admin-dashbboard.css | continued, lines 298-314

```

298 .warning-count.has-warnings { background: var(--warning-soft); color: var(--warning); }
299 .warning-history { margin-top: 8px; }
300 .warning-history > summary { color: var(--accent-deep); cursor: pointer; font-size: 12px;
    font-weight: 700; list-style: none; }
301 .warning-history > summary::-webkit-details-marker, .moderation-warn::-webkit-details-marker {
    display: none; }
302 .warning-history-list { display: grid; width: 250px; gap: 7px; margin-top: 7px; padding: 9px;
    border: 1px solid var(--line); border-radius: 10px; background: var(--surface-muted); }
303 .warning-history-item { padding-bottom: 7px; border-bottom: 1px solid var(--line); }
304 .warning-history-item:last-child { padding-bottom: 0; border-bottom: 0; }
305 .warning-history-item p { margin: 0 0 6px; color: var(--text); font-size: 12px; line-height:
    1.45; white-space: pre-wrap; }
306 .warning-history-meta { display: flex; align-items: center; justify-content: space-between; gap:
    8px; color: var(--faint); font-size: 12px; }
307 .warning-remove-btn { padding: 4px 7px; border: 1px solid #d99a95; border-radius: 6px;
    background: var(--surface); color: var(--danger); cursor: pointer; font-size: 12px; font-weight:
    700; }
308 .warning-remove-btn:hover { background: var(--danger-soft); }
309 .user-actions { min-width: 260px; }
310 .user-action-row { display: flex; align-items: flex-start; gap: 7px; }
311 .moderation-btn, .warning-send-btn { border-radius: 8px; cursor: pointer; font-size: 12px;
    font-weight: 700; }
312 .moderation-btn { display: inline-flex; min-height: 30px; align-items: center; padding: 6px
    10px; border: 1px solid var(--line-strong); background: var(--surface); color: var(--muted);
    list-style: none; white-space: nowrap; user-select: none; }
313 .moderation-ban { color: var(--danger); }
314 .moderation-ban:hover { border-color: #d99a95; background: var(--danger-soft); }

```

Unchanged source lines 315-317 omitted.

Source lines 318-332

```

318 .moderation-warn:hover, .warning-composer[open] .moderation-warn { border-color: #d5b36d;
    background: var(--warning-soft); }
319 .warning-form { width: 280px; margin-top: 8px; padding: 12px; border: 1px solid var(--line);
    border-radius: 11px; background: var(--surface-muted); box-shadow: var(--shadow-sm); }
320 .warning-form label { margin: 0 0 7px; }
321 .warning-form textarea { min-height: 78px; padding: 9px 10px; resize: vertical; font-size: 12px;
    }
322 .warning-send-btn { width: 100%; margin-top: 8px; padding: 8px 11px; border: 1px solid
    var(--warning); background: var(--warning); color: #fff; }
323 .warning-send-btn:hover { background: #8d5c14; }
324
325 /* Empty states */
326 .empty-state { padding: 70px 30px; border: 1px dashed var(--line-strong); border-radius:
    var(--radius-md); background: rgba(255,255,255,.5); color: var(--muted); text-align: center; }
327 .empty-icon { display: grid; width: 52px; height: 52px; margin: 0 auto 14px; place-items:
    center; border-radius: 16px; background: var(--slate-100); font-size: 24px; }
328 .empty-state h3 { font-family: "DM Sans", system-ui, sans-serif; font-size: 20px; font-weight:
    600; }
329 .empty-state p { margin-top: 5px; color: var(--faint); font-size: 13px; }
330
331 /* Mobile */
332 .mobile-nav-toggle, .mobile-scrim { display: none; }

```

Unchanged source lines 333-333 omitted.

Source lines 334-351

```

334 /* Dark theme */
335 html[data-theme="dark"] {
336     color-scheme: dark;
337     --paper: #0e1420;
338     --surface: #182131;
339     --surface-muted: #1e293b;
340     --line: #303e52;
341     --line-strong: #46556b;
342     --text: #edf2fa;
343     --muted: #aebcd0;
344     --faint: #94a3b8;
345     --slate-100: #243247;
346     --slate-200: #33445e;
347     --slate-300: #526986;
348     --accent-soft: #20365c;
349     --success-soft: #203b2e;
350     --warning-soft: #46391f;
351     --danger-soft: #432725;

```

Unchanged source lines 352-353 omitted.

Source lines 354-360

```

354     --shadow-md: 0 18px 50px rgba(0,0,0,.3);
355 }
356 html[data-theme="dark"] body {
357     background: radial-gradient(circle at 91% 2%, rgba(37,99,235,.11), transparent 24rem),
    var(--paper);
358 }
359 html[data-theme="dark"] .workspace-header h1,
360 html[data-theme="dark"] .header-title,

```

Unchanged source lines 361-362 omitted.

Source lines 363-375

```
363 html[data-theme="dark"] .resource-link,  
364 html[data-theme="dark"] .preferred-name,  
365 html[data-theme="dark"] .academic-primary {  
366     color: #edf2fa;  
367 }  
368 html[data-theme="dark"] .workspace-status,  
369 html[data-theme="dark"] .stats-bar,  
370 html[data-theme="dark"] .filter-bar,  
371 html[data-theme="dark"] .user-toolbar {  
372     background: rgba(24,33,49,.86);  
373 }  
374 html[data-theme="dark"] .stat-card:hover,  
375 html[data-theme="dark"] .resource-table tbody tr:hover,
```

Unchanged source lines 376-376 omitted.

Source lines 377-386

```
377 html[data-theme="dark"] input[type="text"]:focus,  
378 html[data-theme="dark"] input[type="search"]:focus,  
379 html[data-theme="dark"] textarea:focus {  
380     background: #243247;  
381 }  
382 html[data-theme="dark"] .file-drop-zone {  
383     background: linear-gradient(135deg, rgba(36,50,71,.68), rgba(30,41,59,.58));  
384 }  
385 html[data-theme="dark"] .file-drop-zone:hover,  
386 html[data-theme="dark"] .file-drop-zone.drag-over {
```

Unchanged source lines 387-392 omitted.

Source lines 393-408

```
393 }  
394 html[data-theme="dark"] .btn-submit:hover:not(:disabled),  
395 html[data-theme="dark"] .user-search-btn:hover {  
396     background: #3974f0;  
397 }  
398 html[data-theme="dark"] .resource-table th { background: #141d2c; }  
399 html[data-theme="dark"] .resource-table td { border-bottom-color: #303e52; }  
400 html[data-theme="dark"] .action-btn,  
401 html[data-theme="dark"] .moderation-btn,  
402 html[data-theme="dark"] .warning-remove-btn {  
403     background: #1e293b;  
404 }  
405 html[data-theme="dark"] .empty-state { background: rgba(24,33,49,.5); }  
406 html[data-theme="dark"] .cat-pyq { background: #352d45; color: #c3a6e4; }  
407 html[data-theme="dark"] .cat-assignment { color: #8dc3d5; }  
408 html[data-theme="dark"] .cat-book { color: #e3bd70; }
```

Unchanged source lines 409-422 omitted.

Source lines 423-436

```
423 @media (max-width: 820px) {
424   .mobile-nav-toggle { position: fixed; z-index: 60; top: 16px; left: 16px; display: grid;
    width: 42px; height: 42px; padding: 11px; place-content: center; gap: 4px; border: 1px solid
    rgba(255,255,255,.15); border-radius: 11px; background: var(--ink-950); box-shadow:
    var(--shadow-md); cursor: pointer; }
425   .mobile-nav-toggle span { display: block; width: 18px; height: 2px; border-radius: 2px;
    background: #fff; }
426   .mobile-scrim { position: fixed; inset: 0; z-index: 20; background: rgba(15,23,42,.45);
    backdrop-filter: blur(3px); }
427   .sidebar { width: min(286px,86vw); transform: translateX(-104%); transition: transform 240ms
    ease; }
428   body.sidebar-open .sidebar { transform: translateX(0); }
429   body.sidebar-open .mobile-scrim { display: block; }
430   body.sidebar-open { overflow: hidden; }
431   .content { margin-left: 0; padding: 88px 22px 54px; }
432   .workspace-header { align-items: flex-start; }
433   .workspace-header h1 { font-size: 40px; }
434   .stats-bar { grid-template-columns: repeat(2,1fr); }
435   .stat-card, .stat-card:nth-child(3), .stat-card:nth-child(n+4) { border-right: 1px solid
    var(--line); border-bottom: 1px solid var(--line); }
436   .stat-card:nth-child(even) { border-right: 0; }
```

Unchanged source lines 437-441 omitted.

Source lines 442-448

```
442   .content { padding-inline: 14px; }
443   .workspace-header { display: block; }
444   .workspace-status { margin-top: 18px; }
445   .workspace-header h1 { font-size: 36px; }
446   .stats-bar { grid-template-columns: 1fr 1fr; }
447   .stat-card { min-height: 112px; padding: 17px; }
448   .stat-number { font-size: 29px; }
```

Unchanged source lines 449-459 omitted.

Source lines 460-550

```

460 @media (prefers-reduced-motion: reduce) {
461   *, *::before, *::after { scroll-behavior: auto !important; animation-duration: .01ms
      !important; animation-iteration-count: 1 !important; transition-duration: .01ms !important; }
462 }
463 .contributor-badge-form { display: grid; gap: 7px; margin-top: 16px; min-width: 160px; }
464 .contributor-badge-form label { color: var(--muted); font-size: 13px; }
465 .contributor-badge-form select { width: 100%; padding: 8px; border: 1px solid var(--line);
      border-radius: 7px; background: var(--surface); color: var(--text); font: inherit; font-size:
      14px; }
466 .contributor-badge-form button { justify-self: start; }
467 .active-students-stat { color: inherit; text-decoration: none; background: #eaf1ff; box-shadow:
      inset 0 4px 0 #2563eb; }
468 .active-students-stat:hover { background: #dce8ff; }
469 .active-students-stat .stat-number { color: #1d4ed8; font-family: 'DM Sans', sans-serif;
      font-size: clamp(40px, 4vw, 54px); line-height: 1.15; font-weight: 700; font-variant-numeric:
      tabular-nums; }
470 .active-students-stat .stat-label { color: #1d4ed8; }
471 html[data-theme="dark"] .active-students-stat { background: #20365c; box-shadow: inset 0 4px 0
      #6699ff; }
472 html[data-theme="dark"] .active-students-stat .stat-number, html[data-theme="dark"]
      .active-students-stat .stat-label { color: #b3cdff; }
473 .presence-dot { display: inline-block; width: 7px; height: 7px; border-radius: 50%; background:
      var(--success); margin-right: 4px; }
474 .active-students-panel { padding: 24px; margin-bottom: 24px; border: 1px solid var(--line);
      border-radius: var(--radius-md); background: var(--surface); scroll-margin-top: 24px; }
475 .active-students-heading { display: flex; flex-wrap: wrap; justify-content: space-between;
      align-items: center; gap: 16px; }
476 .active-students-overview { display: flex; align-items: center; gap: 16px; flex: 1 1 320px;
      min-width: 0; }
477 .presence-total { font-size: clamp(38px, 4vw, 52px); line-height: 1.1; color:
      var(--accent-deep); font-variant-numeric: tabular-nums; flex-shrink: 0; }

```

static/admin-dashboard.css | continued, lines 478-495

```

478 .active-students-controls { display: flex; flex-wrap: wrap; gap: 8px; }
479 .active-students-heading h3 { margin: 0 0 6px; }
480 .active-students-heading p, #activeStudentStatus { font-size: 14px; color: var(--muted);
      line-height: 1.6; }
481 #refreshActiveStudents, #toggleActiveStudents { padding: 10px 14px; border: 1px solid
      var(--line); border-radius: 8px; background: var(--surface); color: var(--text); cursor:
      pointer; }
482 #toggleActiveStudents { background: var(--accent-soft); color: var(--accent-deep); font-weight:
      600; }
483 #activeStudentList { display: grid; grid-template-columns: repeat(auto-fit,
      minmax(min(210px,100%),1fr)); gap: 12px; list-style: none; margin: 16px 0 0; padding: 0; }
484 #activeStudentList[hidden] { display: none; }
485 #activeStudentList a { display: grid; gap: 5px; padding: 14px; border: 1px solid var(--line);
      border-radius: 10px; text-decoration: none; color: var(--text); overflow-wrap: anywhere; }
486 #activeStudentList a:hover { background: var(--accent-soft); }
487 #activeStudentList span, #activeStudentList time { font-size: 13px; color: var(--muted); }
488
489 /* Readable admin controls and student cards. Resource tables keep their columns. */
490 body { font-size: 15px; line-height: 1.5; }
491 .nav-btn { min-height: 52px; }
492 .brand-subtitle, .nav-label { color: #a7b4c8; }
493 .admin-profile-copy small { color: #a7b4c8; }
494 .stat-card { padding-inline: 18px; }
495 .stat-number { font-variant-numeric: tabular-nums; }

```

static/admin-dashboard.css | continued, lines 496-513

```

496 .stat-label { letter-spacing: .04em; }
497 .panel-step { font-size: 36px; font-weight: 600; }
498 .user-search-btn, .user-clear-btn, .user-search-form input[type="search"],
499 .filter-bar select, .moderation-btn, .warning-remove-btn, .warning-send-btn,
500 #refreshActiveStudents, #toggleActiveStudents { min-height: 44px; font-size: 14px; }

```

```

501 .action-btn { width: 38px; height: 38px; font-size: 16px; }
502 .td-actions { min-width: 150px; }
503 .user-table-wrapper { border: 0; border-radius: 0; background: transparent; box-shadow: none;
  overflow: visible; }
504 .user-table { display: block; min-width: 0; }
505 .user-table thead { position: absolute; width: 1px; height: 1px; padding: 0; margin: -1px;
  overflow: hidden; clip: rect(0,0,0,0); white-space: nowrap; }
506 .user-table tbody { display: grid; gap: 20px; }
507 .user-table tbody tr { display: grid; grid-template-columns: repeat(3,minmax(0,1fr)); gap: 24px;
  padding: 24px; border: 1px solid var(--line); border-radius: var(--radius-md); background:
  var(--surface); box-shadow: var(--shadow-sm); }
508 .user-table tbody tr:hover { background: var(--surface); }
509 .user-table tbody td { display: block; min-width: 0; padding: 0; border: 0; font-size: 15px;
  overflow-wrap: anywhere; }
510 .user-cell-label { display: block; margin-bottom: 10px; color: var(--muted); font-size: 12px;
  font-weight: 700; letter-spacing: .06em; text-transform: uppercase; }
511 .preferred-name { font-size: 18px; line-height: 1.4; }
512 .google-name, .academic-primary, .user-email { font-size: 15px; }
513 .user-email { max-width: none; }

```

static/admin-dashboard.css | continued, lines 514-531

```

514 .user-meta { font-size: 13px; white-space: normal; }
515 .name-caption { font-size: 12px; font-weight: 400; letter-spacing: 0; text-transform: none; }
516 .contributor-badge-form { min-width: 0; max-width: 320px; padding-top: 14px; border-top: 1px
  solid var(--line); }
517 .contributor-badge-form label { margin: 0; font-size: 13px; }
518 .contributor-badge-form select { min-height: 44px; padding-right: 32px; font-size: 15px; }
519 .user-action-row { flex-wrap: wrap; gap: 10px; }
520 .user-actions { min-width: 0; }
521 .warning-composer { min-width: 0; max-width: 100%; }
522 .warning-composer[open] { flex-basis: 100%; }
523 .warning-form, .warning-history-list { width: 100%; min-width: 0; }
524 .warning-form textarea { font-size: 15px; }
525 .warning-form label, .warning-history-item p { font-size: 14px; }
526 .warning-history > summary { width: fit-content; padding: 10px 0; font-size: 14px; }
527 .warning-history-meta { flex-wrap: wrap; font-size: 12px; }
528 .presence-total { color: var(--accent-deep); }
529 html[data-theme="dark"] { --accent-deep: #9dbfff; --info: #9dbfff; --slate-600: #aebcd0; }
530 html[data-theme="dark"] .workspace-status, html[data-theme="dark"] label,
531 html[data-theme="dark"] .file-name { color: var(--text); }

```

static/admin-dashboard.css | continued, lines 532-549

```

532 html[data-theme="dark"] .user-table tbody tr:hover { background: var(--surface); }
533 html[data-theme="dark"] .panel-step { color: #536680; }
534 html[data-theme="dark"] .moderation-ban { color: #f4a8a2; }
535 html[data-theme="dark"] .warning-remove-btn { color: #f4a8a2; }
536 html[data-theme="dark"] .admin-avatar { color: #edf2fa; }
537 html[data-theme="dark"] .moderation-warn { color: #e2ba67; }
538 html[data-theme="dark"] .moderation-unban { color: #83c89f; }
539 @media (min-width: 641px) and (max-width: 1280px) {
540   .user-table tbody tr { grid-template-columns: repeat(2,minmax(0,1fr)); }
541 }
542 @media (max-width: 640px) {
543   .user-table tbody tr { grid-template-columns: minmax(0,1fr); gap: 20px; padding: 20px; }
544   .user-table tbody td + td { padding-top: 18px; border-top: 1px solid var(--line); }
545   .contributor-badge-form { max-width: none; }
546   .active-students-panel { padding: 20px; }
547   .active-students-overview { flex-basis: 100%; }
548   input[type="text"], input[type="search"], select, textarea,
549   .contributor-badge-form select, .warning-form textarea { font-size: 16px; }

```

static/admin-dashboard.css | continued, lines 550-550

```

550 }

```

A23 templates/admin-pyqs.html

Single-file upload, subject/year search, paper list, optimistic rename input, delete confirmation, and pagination. This page imports no standalone-manager module.

Source: templates/admin-pyqs.html | 93 total lines | SHA-256 ed349ca6fb613956...

Source lines 1-93

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Manage PYQs · Bookshelf</title>
7      <link rel="stylesheet" href="{{ url_for('static', filename='admin-pyqs.css') }}">
8  </head>
9  <body>
10     <div class="archive-shell">
11         <nav class="archive-nav" aria-label="Page navigation">
12             <a class="brand" href="{{ url_for('admin_dashboard') }}"><span
class="brand-mark">B</span> Bookshelf <span class="admin-label">ADMIN</span></a>
13             <a href="{{ url_for('admin_dashboard') }}">← Admin dashboard</a>
14         </nav>
15         <header class="archive-heading">
16             <div><p class="eyebrow">EXAM ARCHIVE</p><h1>Manage PYQs</h1><p
class="muted">Organise your papers. Keep every subject easy to find.</p></div>
17             <div class="archive-total"><strong>{{ '{:,}'.format(archive.total)
}}</strong><span>papers in the shared library</span></div>
18         </header>

```

templates/admin-pyqs.html | continued, lines 19-36

```

19         {% for category, message in get_flashed_messages(with_categories=true) %}
20         <p class="notice {{ category }}" role="status">{{ message }}</p>
21         {% endfor %}
22         <details class="upload-panel">
23             <summary><span>\uFF0B Upload a PDF</span><span class="muted">Add a single
paper</span></summary>
24             <form action="{{ url_for('admin_upload_pyq') }}" method="post"
enctype="multipart/form-data" class="upload-form">
25                 <input type="hidden" name="_csrf_token" value="{{ csrf_token() }}">
26                 <label>Subject code<input name="subject_code" required maxlength="100"
placeholder="e.g. CS301"></label>
27                 <label>Subject name<input name="subject_name" required maxlength="255"
placeholder="e.g. Data Structures"></label>
28                 <label>Exam year<input name="year" type="number" required min="1900" max="9999"
placeholder="2025"></label>
29                 <label>PDF · up to 50 MB<input name="file_upload" type="file"
accept=".pdf,application/pdf" required></label>
30                 <button class="primary" type="submit">Upload PYQ</button>
31             </form>
32         </details>
33         <section class="library" aria-label="PYQ library">
34             <form class="search-bar" method="get" role="search">
35                 <label class="search-field"><span class="sr-only">Subject code or
name</span><input name="q" value="{{ search }}" placeholder="Search subject code or name..."
maxlength="200"></label>
36                 <label><span class="sr-only">Exam year</span><select name="year"><option
value="">All years</option>

```

templates/admin-pyqs.html | continued, lines 37-54

```

37         {% for available_year in archive.years %}<option value="{{ available_year
38     }}"% if year == available_year %} selected{% endif %}>{{ available_year }}</option>{% endfor %}
39     {% if year and year not in archive.years %}<option value="{{ year }}"
40     selected>{{ year }}</option>{% endif %}
41     </select></label>
42     <button type="submit" class="primary">Search</button>
43     {% if search or year %}<a href="{{ url_for('admin_pyqs') }}">Clear</a>{% endif
44     %}
45 </form>
46 <div class="list-caption"><span>{{ ':,'.format(total) }} papers{% if search or
47 year %} found{% endif %}</span><span>Showing up to 50 per page</span></div>
48 <ul class="pyq-list">
49     {% for paper in papers %}
50     <li class="pyq-row">
51         <div class="paper-heading">
52             <span class="pdf-icon" aria-hidden="true">PDF</span>
53             <div class="paper-info"><h2>{{ paper.title }}</h2><p class="filename">{{
54 paper.original_filename }}</p></div>
55             <span class="year-pill">{{ paper.year }}</span>
56             <a class="open-link" href="{{ paper.blob_url }}" target="_blank"
57 rel="noopener">Open PDF \u2197</a>
58         </div>
59         <div class="row-controls">
60             <details class="name-editor">

```

templates/admin-pyqs.html | continued, lines 55-72

```

55         <summary>Edit name</summary>
56         <form method="post" action="{{ url_for('admin_rename_pyq',
57 paper_id=paper.id) }}" class="rename-form">
58             <input type="hidden" name="_csrf_token" value="{{ csrf_token()
59 }}">
60             <input type="hidden" name="expected_name" value="{{
61 paper.subject_name }}">
62             <input type="hidden" name="q" value="{{ search }}">
63             <input type="hidden" name="filter_year" value="{{ year or ''
64 }}">
65             <input type="hidden" name="page" value="{{ page }}">
66             <label for="name-{{ paper.id }}">Subject name</label>
67             <div class="rename-fields"><input id="name-{{ paper.id }}"
68 name="subject_name" value="{{ paper.subject_name }}" required maxlength="255"><button
69 type="submit" class="primary">Save name</button><button type="button"
70 class="cancel-edit">Cancel</button></div>
71             <p class="muted">Updates this paper's display name for everyone.
72             Subject code, year and PDF stay the same.</p>
73         </form>
74     </details>
75     <form action="{{ url_for('admin_delete_pyq', paper_id=paper.id) }}"
76     method="post" class="delete-form" onsubmit="return confirm('Permanently delete this PYQ?');">
77         <input type="hidden" name="_csrf_token" value="{{ csrf_token()
78 }}"><button class="delete-button" type="submit" aria-label="Delete PYQ: {{ paper.title
79 }}">Delete PYQ</button>
80     </form>
81 </div>
82 </li>
83     {% else %}<li class="empty-state"><h2>No papers found</h2><p>Try another subject
84     or year, or upload a PDF to get started.</p></li>{% endfor %}

```

templates/admin-pyqs.html | continued, lines 73-90

```

73     </ul>
74 </section>
75 <nav class="pagination" aria-label="Paper pages">
76     <span>Page {{ page }}</span><div>

```

```
77         {% if page > 1 %}<a href="{{ url_for('admin_pyqs', page=page-1, q=search, year=year
or '') }}">< Previous</a>{% endif %}
78         {% if page * 50 < total %}<a href="{{ url_for('admin_pyqs', page=page+1, q=search,
year=year or '') }}">Next ></a>{% endif %}
79     </div>
80 </nav>
81 </div>
82 <script>
83     document.querySelectorAll('.cancel-edit').forEach(button => {
84         button.addEventListener('click', () => {
85             button.closest('form').reset();
86             const editor = button.closest('details');
87             editor.open = false;
88             editor.querySelector('summary').focus();
89         });
90     });
```

templates/admin-pyqs.html | continued, lines 91-93

```
91     </script>
92 </body>
93 </html>
```

A24 static/admin-pyqs.css

Shared neutral/blue visual foundation for the admin archive and communications pages. Styles paper rows, year labels, rename forms, responsive navigation, and form controls.

Source: static/admin-pyqs.css | 78 total lines | SHA-256 202ea0dc5b258a54...

Source lines 1-78

```

1 * { box-sizing: border-box; }
2 body { margin: 0; background: #f4f6fa; color: #202c40; font-family: 'Segoe UI', Arial,
  sans-serif; font-size: 16px; }
3 a { color: inherit; text-decoration: none; }
4 button, input, select { font: inherit; }
5 button, summary { cursor: pointer; }
6 button, select, input { border: 1px solid #cbd5e1; border-radius: 8px; padding: 11px 14px;
  background: white; color: #202c40; }
7 button:hover, a:hover { color: #1d4ed8; }
8 :focus-visible { outline: 2px solid #2563eb; outline-offset: 3px; }
9 .archive-shell { max-width: 1200px; margin: auto; padding: 28px 32px 48px; }
10 .archive-nav { display: flex; align-items: center; justify-content: space-between; gap: 18px;
  padding-bottom: 30px; border-bottom: 1px solid #e0e6ef; }
11 .brand { display: flex; align-items: center; gap: 12px; font-size: 21px; font-weight: 650; }
12 .brand-mark { display: grid; place-items: center; width: 38px; height: 38px; background:
  #2563eb; color: white; border-radius: 10px; font-family: inherit; }
13 .admin-label { font-size: 12px; letter-spacing: .12em; color: #64748b; font-weight: 500; }
14 .archive-heading { display: flex; align-items: center; justify-content: space-between; gap:
  24px; margin: 36px 0 28px; }
15 .eyebrow { font-size: 12px; letter-spacing: .16em; color: #1d4ed8; font-weight: 600; }
16 h1 { font-family: inherit; font-size: 42px; font-weight: 700; letter-spacing: -.035em; margin:
  10px 0; }
17 .muted { color: #596a80; font-size: 15px; line-height: 1.6; }
18 .archive-total { display: flex; flex-direction: column; gap: 5px; text-align: right; color:
  #596a80; }

```

static/admin-pyqs.css | continued, lines 19-36

```

19 .archive-total strong { font-size: 32px; font-weight: 500; color: #2563eb; }
20 .upload-panel, .library { background: #ffffff; border: 1px solid #e0e6ef; border-radius: 16px;
  box-shadow: 0 3px 16px rgba(15,23,42,.03); }
21 .upload-panel { margin-bottom: 24px; }
22 .upload-panel > summary { display: flex; justify-content: space-between; align-items: center;
  gap: 12px; padding: 19px 22px; font-weight: 600; list-style: none; }
23 summary::-webkit-details-marker { display: none; }
24 .upload-panel[open] > summary { border-bottom: 1px solid #e8edf4; }
25 .upload-form { padding: 22px; display: flex; flex-wrap: wrap; align-items: end; gap: 18px; }
26 .upload-form label { display: grid; gap: 8px; flex: 1 1 180px; }
27 .upload-form input { width: 100%; }
28 .primary { color: #fff; background: #2563eb; border-color: #2563eb; white-space: nowrap; }
29 .primary:hover { color: #fff; background: #1d4ed8; }
30 .search-bar { display: flex; align-items: center; gap: 12px; padding: 22px; border-bottom: 1px
  solid #e8edf4; }
31 .search-field { flex: 1; }
32 .search-field input { width: 100%; }
33 .list-caption { display: flex; justify-content: space-between; gap: 12px; color: #596a80;
  padding: 16px 24px; font-size: 14px; background: #f8fafc; }
34 .pyq-list { list-style: none; margin: 0; padding: 0; }
35 .pyq-row { padding: 20px 24px 14px; border-top: 1px solid #e8edf4; }
36 .pyq-row:hover { background: #f8fafc; }

```

static/admin-pyqs.css | continued, lines 37-54

```

37 .paper-heading { display: grid; grid-template-columns: 42px minmax(0, 1fr) auto auto;
  align-items: center; gap: 16px; }
38 .pdf-icon { display: grid; place-items: center; height: 46px; border: 1px solid #cfddf7;
  background: #eaf1ff; border-radius: 7px; font-size: 12px; font-weight: 600; color: #1d4ed8; }
39 .paper-info { min-width: 0; }

```

```

40 .paper-info h2 { margin: 0 0 7px; font-size: 17px; line-height: 1.5; font-weight: 600;
    overflow-wrap: anywhere; }
41 .filename { margin: 0; color: #64748b; font-size: 14px; text-overflow: ellipsis; white-space:
    nowrap; overflow: hidden; }
42 .year-pill { background: #edf1f7; color: #45546b; border-radius: 6px; padding: 6px 10px;
    font-size: 14px; }
43 .open-link { font-size: 14px; padding: 8px; white-space: nowrap; }
44 .row-controls { display: flex; gap: 16px; align-items: start; margin: 12px 0 0 58px; }
45 .name-editor { flex: 1; min-width: 0; }
46 .name-editor > summary { font-size: 14px; color: #1d4ed8; padding: 5px 0; list-style: none;
    width: fit-content; }
47 .rename-form { margin-top: 12px; padding: 18px; background: #eef3fc; border-radius: 8px; }
48 .rename-form label { display: block; margin-bottom: 8px; font-size: 14px; font-weight: 600; }
49 .rename-fields { display: flex; gap: 10px; }
50 .rename-fields input { min-width: 0; flex: 1; }
51 .rename-form .muted { margin-bottom: 0; font-size: 14px; }
52 .delete-button { border: 0; background: none; color: #9a7066; padding: 5px 8px; font-size: 14px;
    }
53 .delete-button:hover { color: #a23827; }
54 .pagination { display: flex; justify-content: space-between; align-items: center; margin-top:
    24px; color: #596a80; }

```

static/admin-pyqs.css | continued, lines 55-72

```

55 .pagination div { display: flex; gap: 12px; }
56 .pagination a { padding: 10px 16px; border: 1px solid #cbd5e1; border-radius: 7px; background:
    #ffffff; }
57 .notice { padding: 14px 18px; background: #eaf1ff; border-radius: 8px; }
58 .notice.error { background: #faece7; color: #913d2d; }
59 .empty-state { padding: 40px 24px; text-align: center; color: #596a80; }
60 .sr-only { position: absolute; width: 1px; height: 1px; padding: 0; margin: -1px; overflow:
    hidden; clip: rect(0,0,0,0); white-space: nowrap; border: 0; }
61 @media (max-width: 640px) {
62     .archive-shell { padding: 20px 16px 32px; }
63     .archive-nav { flex-wrap: wrap; padding-bottom: 20px; font-size: 14px; }
64     .archive-heading { align-items: start; gap: 14px; }
65     h1 { font-size: 32px; }
66     .archive-total { min-width: 80px; max-width: 110px; font-size: 12px; }
67     .archive-total strong { font-size: 26px; }
68     .search-bar { flex-wrap: wrap; padding: 16px; }
69     .search-field { flex-basis: 100%; }
70     .list-caption, .pyq-row { padding-left: 16px; padding-right: 16px; }
71     .paper-heading { grid-template-columns: 34px minmax(0, 1fr) auto; gap: 10px; }
72     .open-link { grid-column: 2 / -1; justify-self: start; padding-left: 0; }

```

static/admin-pyqs.css | continued, lines 73-78

```

73     .row-controls { margin-left: 44px; gap: 8px; flex-wrap: wrap; }
74     .name-editor { flex-basis: 100%; }
75     .rename-fields { flex-wrap: wrap; }
76     .rename-fields input { flex-basis: 100%; }
77     .upload-panel > summary { align-items: start; }
78 }

```

A25 static/admin-communications.css

Adds a two-column communications workspace, form spacing, recipient results/chips, and history. At narrower widths the grid becomes one column so inputs remain readable.

Source: static/admin-communications.css | 26 total lines | SHA-256 117964deb6beeeefe...

Source lines 1-26

```

1  .communications-grid { display: grid; grid-template-columns: repeat(2, minmax(0, 1fr)); gap:
  24px; align-items: start; }
2  .communications-card { background: #fff; border: 1px solid #e0e6ef; border-radius: 18px;
  padding: 28px; min-width: 0; }
3  .communications-card h2 { margin: 10px 0 12px; font-size: 23px; }
4  .communications-card p { line-height: 1.6; }
5  .communications-form { display: grid; gap: 18px; margin-top: 22px; }
6  .communications-form label { display: grid; gap: 8px; font-size: 15px; font-weight: 600; }
7  .communications-form input, .communications-form textarea, .communications-form select { width:
  100%; min-width: 0; box-sizing: border-box; border: 1px solid #cbd5e1; border-radius: 9px;
  padding: 12px; background: #fff; color: #202c40; font: inherit; }
8  .communications-form textarea { resize: vertical; }
9  .communications-form p { margin: 0; font-size: 14px; }
10 .communications-actions { display: flex; flex-wrap: wrap; gap: 10px; }
11 .publication-state { display: inline-block; background: #eaf1ff; padding: 6px 10px;
  border-radius: 7px; font-size: 14px; }
12 .recipient-results { max-height: 250px; overflow-y: auto; margin: 12px 0; }
13 .recipient-results label { display: flex; align-items: start; gap: 10px; padding: 12px 6px;
  border-bottom: 1px solid #e8edf4; cursor: pointer; }
14 .recipient-results input { width: 18px; flex-shrink: 0; margin-top: 3px; }
15 .recipient-results small { display: block; font-weight: 400; overflow-wrap: anywhere; }
16 .selected-recipients { display: flex; flex-wrap: wrap; gap: 8px; }
17 .selected-recipients button { max-width: 100%; overflow-wrap: anywhere; }
18 .sent-history { margin-top: 24px; }

```

static/admin-communications.css | continued, lines 19-26

```

19 .sent-history ul { list-style: none; padding: 0; }
20 .sent-history li { display: flex; justify-content: space-between; align-items: start; gap: 20px;
  padding: 20px 0; border-top: 1px solid #e0e6ef; }
21 .sent-history li > div { min-width: 0; }
22 .sent-history strong, .sent-message { overflow-wrap: anywhere; white-space: pre-wrap; }
23 .sent-history small, .sent-history li > span { color: #596a80; font-size: 14px; }
24 .sent-history li > span { flex-shrink: 0; }
25 [hidden] { display: none !important; }
26 @media (max-width: 850px) { .communications-grid { grid-template-columns: minmax(0, 1fr); }
  .communications-card { padding: 20px; } .sent-history li { flex-direction: column; gap: 10px; }
}

```

A26 templates/resource-catalogue.html

One template for notes, assignments, and books: retain filters in links, show verified records, render official/contributor authorship, handle empty/error states, and paginate.

Source: templates/resource-catalogue.html | 58 total lines | SHA-256 9f6c2b0578314353...

Source lines 1-58

```

1 <!DOCTYPE html>
2 {% from '_contributor_badge.html' import contributor_badge %}
3 <html lang="en">
4 <head>
5     <meta charset="UTF-8"><meta name="viewport" content="width=device-width, initial-scale=1.0">
6     <title>{{ title }} · Bookshelf</title>
7     <link rel="stylesheet" href="{{ url_for('static', filename='books.css') }}">
8     <link rel="stylesheet" href="{{ url_for('static', filename='resource-catalogue.css') }}">
9     <link rel="stylesheet" href="{{ url_for('static', filename='contributor-badges.css') }}">
10 </head>
11 <body>
12 <div class="page-wrapper">
13     <nav class="top-nav"><strong>BOOKSHELF</strong><a href="{{ url_for('dashboard') }}"
class="back-btn">← Return to Dashboard</a></nav>
14     <header class="header-section"><div class="page-title"><h1>{{ title }}</h1><p>Browse
uploaded study materials for your department and semester.</p></div></header>
15     <nav class="library-tabs" aria-label="Study materials">
16         {% for target, label in [('notes', 'Notes'), ('assignments', 'Assignments'), ('books',
'Digital Books')] %}
17         <a href="{{ url_for(target, branch=branch, semester=semester) }}" {% if endpoint ==
target %} aria-current="page" {% endif %}>{{ label }}</a>
18         {% endfor %}

```

templates/resource-catalogue.html | continued, lines 19-36

```

19 </nav>
20 <form class="resource-filters" method="get" role="search">
21     <label class="search-label">Search<input name="q" value="{{ search }}" maxlength="200"
placeholder="Title or subject name"></label>
22     <label>Department<select name="branch"><option value="">All departments</option>{% for
code in ['cse', 'it', 'ece', 'ee', 'me', 'ce'] %}<option value="{{ code }}" {% if branch == code
%} selected{% endif %}>{{ code|upper }}</option>{% endfor %}</select></label>
23     <label>Semester<select name="semester"><option value="">All semesters</option>{% for
value in range(1, 9) %}<option value="{{ value }}" {% if semester == value|string %} selected{%
endif %}>Semester {{ value }}</option>{% endfor %}</select></label>
24     <button type="submit">Search</button>
25     {% if search or branch or semester %}<a href="{{ url_for(endpoint) }}">Clear
filters</a>{% endif %}
26 </form>
27 {% if error %}<p class="library-message" role="alert">{{ error }}</p>
28 {% else %}<p class="result-count">{{ total }} resources{% if branch %} · {{ branch|upper
}}{% endif %}{% if semester %} · Semester {{ semester }}{% endif %}</p>
29 <main class="content-grid">
30     {% for resource in resources %}
31     <article class="resource-card">
32         <span class="resource-course">{{ resource.branch|upper }} · Semester {{
resource.semester }}</span>
33         {% if resource.is_admin_upload %}
34         <span class="admin-upload-badge" title="Uploaded directly by a Bookshelf
administrator.">
35         <svg width="16" height="16" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="1.8" stroke-linecap="round" stroke-linejoin="round"
aria-hidden="true" focusable="false"><path d="M12 3 4 6v6c0 5 8 9 8 9s8-4 8-9V6l-8-3Z"/><path
d="m8.5 12 2.3 2.3 4.7-4.8"/></svg>

```

36 Official · Admin upload

templates/resource-catalogue.html | continued, lines 37-54

```

37                   </span>
38                   {% else %}
39                   {{ contributor_badge(resource.contributor_badge) }}
40                   {% endif %}
41                   <h2>{{ resource.title }}</h2>
42                   {% if resource.subject_name %}<p class="resource-subject">{{ resource.subject_name
}}</p>{% endif %}
43                   <p class="resource-uploader">Uploaded by <strong>{{ resource.uploader_label
}}</strong></p>
44                   {% if resource.file_name %}<p class="resource-filename">{{ resource.file_name
}}</p>{% endif %}
45                   {% if resource.file_size %}<p class="resource-size">{{
'%.2f'|format(resource.file_size / 1048576) }} MB</p>{% endif %}
46                   <a class="resource-open" href="{{ resource.blob_url }}" target="_blank"
rel="noopener">Open file \u2197</a>
47                   </article>
48                   {% else %}<div class="placeholder-card"><h2>No {{ title|lower }} found</h2><p>Try
another search or clear the filters. New verified uploads will appear here.</p></div>{% endfor
%}
49                   </main>
50                   <nav class="resource-pagination" aria-label="Resource pages"><span>Page {{ page
}}</span><div>
51                    {% if page > 1 %}<a href="{{ url_for(endpoint, branch=branch, semester=semester,
q=search, page=page-1) }}">< Previous</a>{% endif %}
52                    {% if page * 50 < total %}<a href="{{ url_for(endpoint, branch=branch,
semester=semester, q=search, page=page+1) }}">Next ></a>{% endif %}
53                   </div></nav>
54                   {% endif %}

```

templates/resource-catalogue.html | continued, lines 55-58

```

55 </div>
56 {% include '_student_activity.html' %}
57 </body>
58 </html>

```

A27 static/resource-catalogue.css

Verified-resource cards, filter controls, author labels, official Admin badge, pagination, and mobile wrapping. A shared macro adds student awards; this stylesheet preserves the stronger official treatment.

Source: static/resource-catalogue.css | 24 total lines | SHA-256 c748cd9e11ef0d81...

Source lines 1-24

```

1 .library-tabs { display: flex; flex-wrap: wrap; gap: 12px; margin-bottom: 28px; }
2 .library-tabs a, .resource-pagination a { color: #f4efe3; border: 1px solid #675039;
  border-radius: 8px; padding: 10px 15px; text-decoration: none; }
3 .library-tabs a[aria-current] { background: #f2c14e; color: #251b10; border-color: #f2c14e; }
4 .resource-filters { display: flex; flex-wrap: wrap; align-items: end; gap: 14px; margin-bottom:
  24px; }
5 .resource-filters label { display: flex; flex-direction: column; gap: 8px; font-size: 13px;
  color: #d1c2ab; }
6 .resource-filters .search-label { flex: 1 1 240px; }
7 .resource-filters input, .resource-filters select, .resource-filters button { padding: 12px;
  border: 1px solid #675039; border-radius: 8px; background: #291f16; color: #f4efe3; font:
  inherit; min-width: 0; }
8 .resource-filters button { cursor: pointer; background: #f2c14e; color: #251b10; }
9 .resource-filters a { color: #f2c14e; align-self: center; }
10 .result-count { font-size: 13px; color: #cbb99d; margin: 24px 0; }
11 .resource-card { padding: 24px; border: 1px solid #675039; border-radius: 12px; display: flex;
  flex-direction: column; gap: 12px; min-width: 0; }
12 .resource-card h2 { font-size: 20px; line-height: 1.4; margin: 0; overflow-wrap: anywhere; }
13 .resource-card p { margin: 0; }
14 .resource-uploader { font-size: 13px; color: #cbb99d; }
15 .resource-uploader strong { color: #f4efe3; font-weight: 500; }
16 .admin-upload-badge { display: inline-flex; align-items: center; align-self: flex-start; gap:
  7px; max-width: 100%; box-sizing: border-box; padding: 6px 10px; border: 1px solid #416759;
  border-radius: 999px; background: #1e352d; color: #bce8d4; font-size: 11px; font-weight: 600;
  line-height: 1.4; letter-spacing: .02em; }
17 .admin-upload-badge svg { flex-shrink: 0; }
18 .resource-course, .resource-size { font-size: 12px; color: #cbb99d; }

```

static/resource-catalogue.css | continued, lines 19-24

```

19 .resource-filename { font-size: 12px; color: #b2a58f; overflow-wrap: anywhere; }
20 .resource-open { color: #f2c14e; margin-top: auto; padding-top: 12px; }
21 .resource-pagination { display: flex; justify-content: space-between; align-items: center; gap:
  16px; margin-top: 28px; }
22 .resource-pagination div { display: flex; gap: 12px; }
23 .library-message { padding: 24px; background: #332318; border-radius: 12px; }
24 @media (max-width: 420px) { .content-grid { grid-template-columns: minmax(0, 1fr); }
  .resource-filters label { width: 100%; } }

```

A28 templates/pyqs.html

Renders the actual shared paper list, archive totals, subject/year filters, and pagination instead of static placeholder content. Adds the student activity include.

Source: templates/pyqs.html | 86 total lines | SHA-256 4521fe9aaf967df0...

Unchanged source lines 1-14 omitted.

Source lines 15-86

```

15         
16         </div>
17
18         <a href="{{ url_for('admin_dashboard' if admin else 'dashboard') }}"
    class="back-btn">
19             <span></span> Return to Dashboard
20         </a>
21     </nav>
22     <header class="header-section">
23         <div class="page-title">
24             <h1>{{ 'Manage PYQs' if admin else 'Previous Papers' }}</h1>
25             <p>Shared papers for all students. Search by subject and exam year.</p>
26         </div>
27
28         <form method="get" class="minimal-search-box">
29             <svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" fill="none"
    stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round">
30                 <circle cx="11" cy="11" r="8"></circle>
31                 <line x1="21" y1="21" x2="16.65" y2="16.65"></line>
32             </svg>

```

templates/pyqs.html | continued, lines 33-50

```

33         <input name="q" value="{{ search }}" aria-label="Subject code or name"
    class="minimal-search-input" placeholder="Subject code or name" maxlength="200">
34         <select name="year" aria-label="Exam year">
35             <option value="">All years</option>
36             {% for available_year in archive.years %}
37             <option value="{{ available_year }}" {% if year == available_year %}
    selected{% endif %}>{{ available_year }}</option>
38             {% endfor %}
39             {% if year and year not in archive.years %}<option value="{{ year }}"
    selected>{{ year }}</option>{% endif %}
40         </select>
41         <button type="submit">Search</button>
42         {% if search or year %}<a class="clear-search" href="{{ url_for('admin_pyqs' if
    admin else 'pyqs') }}">Clear</a>{% endif %}
43     </form>
44 </header>
45
46     {% for category, message in get_flashed_messages(with_categories=true) %}
47     <p role="status">{{ message }}</p>
48     {% endfor %}
49     {% if admin %}
50     <h2>Upload a PDF</h2>

```

templates/pyqs.html | continued, lines 51-68

```

51     <form class="paper-form" action="{{ url_for('admin_upload_pyq') }}" method="post"
    enctype="multipart/form-data">
52         <input type="hidden" name="_csrf_token" value="{{ csrf_token() }}">
53         <label>Subject code <input name="subject_code" required maxlength="100"></label>

```

```

54         <label>Subject name <input name="subject_name" required maxlength="255"></label>
55         <label>Exam year <input name="year" type="number" required min="1900"
max="9999"></label>
56         <label>PDF (up to 50 MB) <input name="file_upload" type="file"
accept=".pdf,application/pdf" required></label>
57         <button type="submit">Upload PYQ</button>
58     </form>
59     {% endif %}
60     <p>{{ '{:,'.format(total) }} papers found · {{ '{:,'.format(archive.total) }} in the
shared archive</p>
61     <main class="content-grid">
62         {% for paper in papers %}
63         <article class="paper-card">
64             <h2>{{ paper.title }}</h2>
65             <p>Exam year: {{ paper.year }}</p>
66             <a href="{{ paper.blob_url }}" target="_blank" rel="noopener">Open PDF
\u2197</a>
67             {% if admin %}
68             <p>{{ paper.original_filename }}</p>

```

templates/pyqs.html | continued, lines 69-86

```

69         <form action="{{ url_for('admin_delete_pyq', paper_id=paper.id) }}"
method="post" onsubmit="return confirm('Permanently delete this PYQ?');">
70             <input type="hidden" name="_csrf_token" value="{{ csrf_token() }}">
71             <button type="submit">Delete PYQ</button>
72         </form>
73         {% endif %}
74     </article>
75     {% else %}<p>No papers found.</p>{% endfor %}
76 </main>
77 <nav class="pagination" aria-label="Paper pages">
78     {% if page > 1 %}<a href="{{ url_for('admin_pyqs' if admin else 'pyqs', page=page-1,
q=search, year=year or '') }}">Previous</a>{% endif %}
79     <span>Page {{ page }}</span>
80     {% if page * 50 < total %}<a href="{{ url_for('admin_pyqs' if admin else 'pyqs',
page=page+1, q=search, year=year or '') }}">Next</a>{% endif %}
81 </nav>
82 </div>
83
84 {% include '_student_activity.html' %}
85 </body>
86 </html>

```

A29 static/pyqs.css

Student archive layout/filter/year/list styling and responsive changes. The archive is shared; frontend styles do not add branch or semester constraints to its database.

Source: static/pyqs.css | 218 total lines | SHA-256 fbd31528cfc45ade...

Unchanged source lines 1-186 omitted.

Source lines 187-218

```

187     .content-grid {
188         gap: 16px !important;
189     }
190 }
191
192 .paper-form { display: flex; flex-wrap: wrap; gap: 16px; margin: 24px 0 40px; }
193 .paper-form label { display: flex; flex-direction: column; gap: 8px; }
194 .paper-form input, .paper-form button, .minimal-search-box input, .minimal-search-box button {
195     box-sizing: border-box; padding: 12px; border: 1px solid #876b3f;
196     border-radius: 8px; background: #251c13; color: #f4efe3; max-width: 100%;
197 }
198 .paper-card { padding: 24px; border: 1px solid #876b3f; border-radius: 12px; overflow-wrap:
    anywhere; }
199 .paper-card h2 { font-size: 1.2rem; margin: 0 0 12px; }
200 .paper-card a, .pagination a { color: #f2c14e; }
201 .paper-card p { margin: 12px 0; }
202 .pagination { display: flex; gap: 24px; margin: 24px 0; }
203 .minimal-search-box select {
204     padding: 12px;

```

static/pyqs.css | continued, lines 205-218

```

205     border: 1px solid #876b3f;
206     border-radius: 8px;
207     background: #251c13;
208     color: #f4efe3;
209 }
210 .clear-search { align-self: center; color: #f2c14e; padding: 8px; }
211 .minimal-search-box { display: flex; flex-wrap: wrap; gap: 8px; }
212 .minimal-search-box svg { display: none; }
213 .minimal-search-box input[name="year"] { width: 110px; }
214 .paper-card button, .paper-form button, .minimal-search-box button { cursor: pointer; }
215 @media (max-width: 400px) {
216     .content-grid { grid-template-columns: minmax(0, 1fr); }
217     .paper-form label { width: 100%; }
218 }

```

A30 static/dashboard.css

Scoped department text sizing fixes badge alignment. Mobile navigation becomes five grid slots, with safe-area padding and bounded settings placement. Announcement layout reuses the earlier archive-card visual shell.

Source: static/dashboard.css | 1088 total lines | SHA-256 c3c3b8145900e133...

Unchanged source lines 1-14 omitted.

Source lines 15-48

```
15     overflow: hidden;
16 }
17
18 .pyq-archive-card {
19     display: flex;
20     align-items: center;
21     justify-content: space-between;
22     gap: 24px;
23     margin: 0 0 32px;
24     padding: 26px 30px;
25     border: 1px solid #d8c6a5;
26     border-radius: 18px;
27     background: #fbf6e9;
28     color: #30251a;
29     text-decoration: none;
30 }
31 .pyq-archive-card:hover, .pyq-archive-card:focus-visible {
32     border-color: #a67c36;
```

static/dashboard.css | continued, lines 33-48

```
33     box-shadow: 0 5px 20px rgba(72, 48, 13, .08);
34 }
35 .pyq-archive-label { font-size: 11px; letter-spacing: .12em; text-transform: uppercase; }
36 .pyq-archive-card h2 { margin: 8px 0; font-size: 23px; }
37 .pyq-archive-card p { font-size: 14px; line-height: 1.5; }
38 .pyq-archive-count { display: block; margin-top: 12px; font-size: 13px; font-weight: 600; }
39 .pyq-archive-action { flex-shrink: 0; font-size: 14px; font-weight: 600; }
40 body.dark-mode .pyq-archive-card { background: #30251a; color: #f4efe3; border-color: #755a33; }
41 @media (max-width: 640px) {
42     .pyq-archive-card { flex-direction: column; align-items: flex-start; padding: 22px; gap:
43     16px; }
44     .pyq-archive-card h2 { font-size: 20px; }
45 }
46
47 #loader {
48     position: fixed;
```

Unchanged source lines 49-334 omitted.

Source lines 335-356

```

335     letter-spacing: 0.02em;
336 }
337
338 .dept-card > div {
339     width: 100%;
340     min-width: 0;
341 }
342
343 .dashboard-nav-icon, .nav-label-mobile { display: none; }
344
345 #step-sem, #step-category { scroll-margin-top: 24px; }
346
347 .dept-card h3 {
348     font-family: "Inter", sans-serif;
349     font-size: 18px;
350     font-weight: 600;
351     color: #1A130D;
352     line-height: 1.3;

```

static/dashboard.css | continued, lines 353-356

```

353     min-height: 2.6em;
354 }
355
356

```

Unchanged source lines 357-880 omitted.

Source lines 881-896

```

881     flex-direction: column !important;
882     height: auto !important;
883     min-height: 100vh !important;
884     overflow: visible !important;
885 }
886
887 .main-content {
888     margin: 0 !important;
889     border-radius: 0 !important;
890     padding: 24px 20px calc(120px + env(safe-area-inset-bottom)) 20px !important;
891     box-shadow: none !important;
892     min-height: 100vh !important;
893     overflow: visible !important;
894 }
895
896 .top-header {

```

Unchanged source lines 897-922 omitted.

Source lines 923-938

```

923     max-width: 100% !important;
924 }
925
926 @media (min-width: 600px) and (max-height: 500px) {
927     .semester-grid { grid-template-columns: repeat(4, minmax(0, 1fr)) !important; }
928 }
929
930 /* 4. PREMIUM FLOATING "PILL" BOTTOM NAV */
931 .sidebar {
932     position: fixed !important;
933     display: grid !important;
934     grid-template-columns: repeat(5, minmax(0, 1fr));
935     gap: 2px;
936
937     bottom: calc(20px + env(safe-area-inset-bottom)) !important;
938     left: 16px !important;

```

Unchanged source lines 939-966 omitted.

Source lines 967-1001

```

967     .sidebar-header { display: none !important; }
968
969     .sidebar-nav, .sidebar-footer {
970         display: contents !important;
971     }
972
973     .sidebar-footer > div { min-width: 0; }
974
975     .sidebar .nav-item {
976         display: flex !important;
977         flex-direction: column;
978         align-items: center;
979         justify-content: center;
980         gap: 5px;
981         width: 100%;
982         min-width: 0;
983         min-height: 52px;
984         padding: 8px 2px !important;

```

static/dashboard.css | continued, lines 985-1001

```

985         font-size: 10px !important;
986
987         font-weight: 500 !important;
988         text-align: center !important;
989         border-radius: 24px !important;
990         transition: all 0.3s ease !important;
991
992         white-space: nowrap !important;
993     }
994
995     .sidebar .dashboard-nav-icon { display: block; flex-shrink: 0; }
996     .sidebar .nav-label-desktop { display: none; }
997     .sidebar .nav-label-mobile { display: inline; }
998
999     .nav-item.active {
1000         background: #F2C14E !important;
1001         color: #1A130D !important;

```

Unchanged source lines 1002-1003 omitted.

Source lines 1004-1015

```

1004     }
1005
1006     .settings-popup {
1007         bottom: calc(100% + 20px) !important;
1008         left: auto !important;
1009         right: 0;
1010         width: min(220px, calc(100vw - 48px));
1011         margin-left: 0;
1012         transform: none !important;
1013     }
1014     /* 5. ONBOARDING MODAL (ROUNDED FLOATING CARD) */
1015     .onboarding-overlay {

```

Unchanged source lines 1016-1088 omitted.

A31 templates/contribute.html

Adds the shared activity include to the existing contribution page. This change alone does not turn the admin-only upload route into a student contribution backend.

Source: templates/contribute.html | 135 total lines | SHA-256 4e0d0414e8f753dd...

Unchanged source lines 1-129 omitted.

Source lines 130-135

```
130         </main>
131     </div>
132 <script src="{ { url_for('static', filename='contribute.js') } }"></script>
133 {% include '_student_activity.html' %}
134 </body>
135 </html>
```

A32 static/script.js

Adds a desktop animation scheduler and caches layout values for the landing-page bulb/rope. Visibility and one outstanding frame reduce redundant work; fallback behaviour remains available.

Source: static/script.js | 827 total lines | SHA-256 b8082f63612343e6...

Source lines 1-34

```

1 // Desktop effects share a single-flight scheduler. The original mobile/tablet
2 // paths remain in use, and visible animations still run at the display's cadence.
3 window.createDesktopAnimationLoop = function (element, render, isEnabled = () => true) {
4   const desktop = window.matchMedia('(min-width: 961px) and (hover: hover) and (pointer: fine)');
5   if (!desktop.matches) return null;
6   let frame = null;
7   let visible = false;
8   const active = () => !desktop.matches || (visible && !document.hidden && isEnabled());
9   function request() {
10    if (frame !== null || !active()) return;
11    frame = requestAnimationFrame(time => {
12      frame = null;
13      if (active()) render(time);
14    });
15  }
16  function cancel() {
17    if (frame !== null) cancelAnimationFrame(frame);
18    frame = null;

```

static/script.js | continued, lines 19-34

```

19  }
20  function sync() {
21    if (active()) request(); else cancel();
22  }
23  new IntersectionObserver(entries => {
24    visible = entries[0].isIntersecting;
25    sync();
26  }).observe(element);
27  document.addEventListener('visibilitychange', sync);
28  desktop.addEventListener('change', sync);
29  return { request, cancel, get desktop() { return desktop.matches; } };
30 };
31
32 gsap.registerPlugin(ScrollTrigger);
33 if ("scrollRestoration" in history){
34   history.scrollRestoration = "manual";

```

Unchanged source lines 35-145 omitted.

Source lines 146-165

```
146
147 let anchorX = 0;
148 let mouseX = 0;
149 let heroBounds = { left: 0, width: 0 };
150 let ropeSize = { width: 0, height: 0 };
151
152 function computeAnchor() {
153   const heroRect = heroEl.getBoundingClientRect();
154   anchorX = heroRect.width * 0.60;
155   heroBounds = { left: heroRect.left, width: heroRect.width };
156 }
157
158 function sizeRopeCanvas() {
159   const dpr = Math.min(window.devicePixelRatio || 1, 2);
160   ropeSize = { width: ropeCanvas.clientWidth, height: ropeCanvas.clientHeight };
161   ropeCanvas.width = ropeSize.width * dpr;
162   ropeCanvas.height = ropeSize.height * dpr;
163   ropeCtx.setTransform(dpr, 0, 0, dpr, 0, 0);
```

static/script.js | continued, lines 164-165

```
164 }
165
```

Unchanged source lines 166-281 omitted.

Source lines 282-289

```
282 }
283
284 function drawRope() {
285   const cw = bulbLoop?.desktop ? ropeSize.width : ropeCanvas.clientWidth;
286   const ch = bulbLoop?.desktop ? ropeSize.height : ropeCanvas.clientHeight;
287   ropeCtx.clearRect(0, 0, cw, ch);
288
289   const bulbPos = getBulbPos();
```

Unchanged source lines 290-325 omitted.

Source lines 326-337

```
326 }
327
328
329 const bulbLoop = window.createDesktopAnimationLoop(heroEl, animateBulb);
330 if (bulbLoop) {
331   new ResizeObserver(() => { computeAnchor(); sizeRopeCanvas(); }).observe(heroEl);
332   ScrollTrigger.addEventListener('refresh', computeAnchor);
333 }
334
335 function animateBulb() {
336   if (!bulbRunning) return;
337
```

Unchanged source lines 338-340 omitted.

Source lines 341-347

```

341  angleVel += gravityForce;
342
343
344  const heroRect = bulbLoop?.desktop ? heroBounds : heroEl.getBoundingClientRect();
345  const relMouse = (mouseX - heroRect.left) / heroRect.width;
346  const targetAngle = (relMouse - 0.6) * 0.06;
347  const mouseForce = (targetAngle - angle) * MOUSE_STRENGTH;

```

Unchanged source lines 348-367 omitted.

Source lines 368-380

```

368  // Apply parallax to canvas
369  ropeCanvas.style.transform = `translateY(${bulbParallaxY}px)`;
370
371  if (bulbLoop) bulbLoop.request(); else requestAnimationFrame(animateBulb);
372  }
373
374  function startBulb() {
375    if (bulbLoop) bulbLoop.request(); else animateBulb();
376  }
377  startBulb();
378
379  // Turn on glow
380  bulbEl.classList.add("on");

```

Unchanged source lines 381-586 omitted.

Source lines 587-596

```

587  trigger: ".hero",
588  start: "top bottom",
589  end: "bottom top",
590  onEnter: () => { bulbRunning = true; startBulb(); },
591  onLeave: () => { bulbRunning = false; if (bulbLoop) bulbLoop.cancel(); },
592  onEnterBack: () => { bulbRunning = true; startBulb(); },
593  onLeaveBack: () => { bulbRunning = false; if (bulbLoop) bulbLoop.cancel(); }
594  });
595
596  // PENCIL DRAW TRAIL (page2) //

```

Unchanged source lines 597-728 omitted.

Source lines 729-735

```

729  onUpdate: (self) => {
730    scrollCue.style.opacity = 1 - self.progress;
731    scrollCue.style.pointerEvents = self.progress > 0.8 ? "none" : "auto";
732    if (window.requestScrollCueRender) window.requestScrollCueRender();
733  }
734  });
735  }

```

Unchanged source lines 736-823 omitted.

Source lines 824-827

```

824      fcRender();
825    }, { passive: true });
826  }
827  }

```

A33 static/grid-bg.js

Caches reusable grid paths and dimensions, redraws on changes, and schedules further frames only while easing is changing the desktop state. SVG/Canvas geometry is unrelated to database operations.

Source: static/grid-bg.js | 145 total lines | SHA-256 fb2d7499fb1c42c3...

Unchanged source lines 1-11 omitted.

Source lines 12-21

```
12     const LERP = 0.14;
13
14     let mx = -9999, my = -9999, cmx = -9999, cmy = -9999;
15     let drawLoop = null;
16     let size = { width: 0, height: 0 };
17     let segments = null;
18     let dirty = true;
19
20     function resize() {
21         const r = page2.getBoundingClientRect();
```

Unchanged source lines 22-24 omitted.

Source lines 25-34

```
25         canvas.style.width = r.width + 'px';
26         canvas.style.height = r.height + 'px';
27         ctx.setTransform(dpr, 0, 0, dpr, 0, 0);
28         size = { width: canvas.clientWidth, height: canvas.clientHeight };
29         segments = null;
30         dirty = true;
31         if (drawLoop) drawLoop.request();
32     }
33     resize();
34     window.addEventListener('resize', resize, { passive: true });
```

Unchanged source lines 35-36 omitted.

Source lines 37-48

```
37         const r = page2.getBoundingClientRect();
38         mx = e.clientX - r.left;
39         my = e.clientY - r.top;
40         if (drawLoop) drawLoop.request();
41     });
42     page2.addEventListener('mouseleave', () => {
43         mx = -9999;
44         my = -9999;
45         if (drawLoop) drawLoop.request();
46     });
47
48     function smoothstep(t) { return t * t * (3 - 2 * t); }
```

Unchanged source lines 49-56 omitted.

Source lines 57-74

```

57     return [cmx + dx * scale, cmy + dy * scale, t];
58   }
59
60   function drawSmoothLine(x1, y1, x2, y2, path) {
61     if (path) {
62       // Outside the distortion circle, replay precisely the same 22-step
63       // path instead of recalculating and allocating every point.
64       const dx = cmx - Math.max(x1, Math.min(x2, cmx));
65       const dy = cmy - Math.max(y1, Math.min(y2, cmy));
66       if (cmx < 0 || dx * dx + dy * dy > RADIUS * RADIUS) {
67         ctx.strokeStyle = 'rgba(242, 193, 78, 0.06)';
68         ctx.stroke(path);
69         return;
70       }
71     }
72     let maxT = 0;
73     ctx.beginPath();
74     for (let i = 0; i <= STEPS; i++) {

```

Unchanged source lines 75-84 omitted.

Source lines 85-126

```

85   }
86
87   function draw() {
88     const previousX = cmx, previousY = cmy;
89     cmx += (mx - cmx) * LERP;
90     cmy += (my - cmy) * LERP;
91
92     const desktop = drawLoop?.desktop && typeof Path2D !== 'undefined';
93     const unchanged = cmx === previousX && cmy === previousY;
94     if (desktop && !dirty && unchanged) return;
95     const cw = desktop ? size.width : canvas.clientWidth;
96     const ch = desktop ? size.height : canvas.clientHeight;
97     ctx.clearRect(0, 0, cw, ch);
98     ctx.lineWidth = 0.75;
99
100    const cols = Math.ceil(cw / SPACING) + 1;
101    const rows = Math.ceil(ch / SPACING) + 1;
102

```

static/grid-bg.js | continued, lines 103-120

```

103    if (desktop) {
104      if (!segments) {
105        segments = [];
106        function cache(x1, y1, x2, y2) {
107          const path = new Path2D();
108          for (let i = 0; i <= STEPS; i++) {
109            const f = i / STEPS;
110            const x = x1 + (x2 - x1) * f, y = y1 + (y2 - y1) * f;
111            if (i === 0) path.moveTo(x, y); else path.lineTo(x, y);
112          }
113          segments.push([x1, y1, x2, y2, path]);
114        }
115        for (let r = 0; r <= rows; r++) {
116          for (let c = 0; c < cols; c++) cache(c * SPACING, r * SPACING, (c + 1) *
SPACING, r * SPACING);
117        }
118        for (let c = 0; c <= cols; c++) {
119          for (let r = 0; r < rows; r++) cache(c * SPACING, r * SPACING, c * SPACING,

```

```
        (r + 1) * SPACING);
120      }
```

static/grid-bg.js | continued, lines 121-126

```
121      }
122      segments.forEach(segment => drawSmoothLine(...segment));
123    } else {
124      for (let r = 0; r <= rows; r++) {
125        for (let c = 0; c < cols; c++) {
126          drawSmoothLine(c * SPACING, r * SPACING, (c + 1) * SPACING, r * SPACING);
```

Unchanged source lines 127-130 omitted.

Source lines 131-145

```
131          drawSmoothLine(c * SPACING, r * SPACING, c * SPACING, (r + 1) * SPACING);
132        }
133      }
134    }
135
136    dirty = false;
137    // Stop only on exact convergence: no changed easing, thresholds or snapping.
138    if (drawLoop) {
139      if (!desktop || !unchanged) drawLoop.request();
140    } else requestAnimationFrame(draw);
141  }
142  drawLoop = window.createDesktopAnimationLoop?.(page2, draw);
143  if (drawLoop) drawLoop.request(); else draw();
144  if (drawLoop) new ResizeObserver(resize).observe(page2);
145  })();
```

A34 static/page-wheel.js

Caches unchanged style writes in a WeakMap, caches layout height, and wakes animation for progress/input/resize. CSS transforms still use the original wheel calculations.

Source: static/page-wheel.js | 170 total lines | SHA-256 a59ecf2cd3fa67a5...

Unchanged source lines 1-16 omitted.

Source lines 17-46

```
17     const MIN_ROT = -(N - 1) * STEP;
18     const MAX_ROT = 0;
19
20     let layoutLoop = null;
21     let height = container.clientHeight;
22     let dirty = true;
23     const writtenStyles = new WeakMap();
24     function setStyle(element, property, value) {
25         if (!layoutLoop?.desktop) {
26             writtenStyles.delete(element);
27             element.style[property] = value;
28             return;
29         }
30         let previous = writtenStyles.get(element);
31         if (!previous) { previous = {}; writtenStyles.set(element, previous); }
32         if (previous[property] === value) return;
33         previous[property] = value;
34         element.style[property] = value;
```

static/page-wheel.js | continued, lines 35-46

```
35     }
36
37     let rotation = 0, targetRotation = 0;
38     let tiltX = 0, tiltY = 0, targetTiltX = 0, targetTiltY = 0;
39
40     // exposed so ScrollTrigger can drive this directly from scroll progress
41     window.setPageWheelProgress = function (p) {
42         targetRotation = MAX_ROT + (MIN_ROT - MAX_ROT) * p;
43         if (layoutLoop) layoutLoop.request();
44     };
45
46     function makeContentSVG() {
```

Unchanged source lines 47-92 omitted.

Source lines 93-112

```

93     const r = container.getBoundingClientRect();
94     targetTiltY = ((e.clientX - r.left) / r.width - 0.5) * 8;
95     targetTiltX = ((e.clientY - r.top) / r.height - 0.5) * -5;
96     if (layoutLoop) layoutLoop.request();
97   });
98
99   function layout() {
100     const ch = layoutLoop?.desktop ? height : container.clientHeight;
101     const previousRotation = rotation, previousTiltX = tiltX, previousTiltY = tiltY;
102     const cx = -60, cy = ch / 2, R = Math.min(260, ch * 0.42);
103
104     rotation += (targetRotation - rotation) * 0.09;
105     tiltX += (targetTiltX - tiltX) * 0.06;
106     tiltY += (targetTiltY - tiltY) * 0.06;
107     const unchanged = previousRotation === rotation && previousTiltX === tiltX &&
previousTiltY === tiltY;
108     if (layoutLoop?.desktop && unchanged && !dirty) return;
109     setStyle(stage, 'transform', `rotateX(${tiltX}deg) rotateY(${tiltY}deg)`);
110

```

[static/page-wheel.js](#) | continued, lines 111-112

```

111     const sorted = [];
112     for (let i = 0; i < N; i++) {

```

Unchanged source lines 113-130 omitted.

Source lines 131-170

```

131     if (wrapped < -Math.PI) wrapped += Math.PI * 2;
132     const rotY = -wrapped * (180 / Math.PI) * 0.85;
133
134     setStyle(card, 'left', (x - 66) + 'px');
135     setStyle(card, 'top', (y - 86) + 'px');
136     setStyle(card, 'transform', `translateZ(${front * 40}px) scale(${scale})
rotateY(${rotY}deg)`);
137     setStyle(card, 'opacity', opacity);
138     setStyle(card, 'filter', `blur(${blur}px)`);
139     setStyle(card, 'zIndex', Math.round(front * 1000) + 2000);
140
141     const isFront = front > 0.9;
142     const frontT = Math.max(0, (front - 0.9) / 0.1);
143     setStyle(card, 'boxShadow', isFront
144       ? `inset 0 1px 0 rgba(255,255,255,0.6), 0 30px 70px rgba(0,0,0,0.5), 0 0 0 1px
rgba(242,193,78,${0.5 * frontT}), 0 0 45px rgba(242,193,78,${0.3 * frontT})`
145       : `inset 0 1px 0 rgba(255,255,255,0.6), 0 20px 40px rgba(0,0,0,0.35)`);
146     setStyle(card, 'borderColor', isFront ? `rgba(242,193,78,${0.5 * frontT})` :
'rgba(140,100,32,0.1)');
147
148     setStyle(label, 'left', (x + 100) + 'px'); setStyle(label, 'top', (y - 18) + 'px');

```

[static/page-wheel.js](#) | continued, lines 149-166

```

149     setStyle(label, 'opacity', isFront ? frontT : 0);
150     setStyle(desc, 'left', (x + 100) + 'px'); setStyle(desc, 'top', (y + 16) + 'px');
151     setStyle(desc, 'opacity', isFront ? frontT * 0.9 : 0);
152     setStyle(line, 'left', (x + 100) + 'px'); setStyle(line, 'top', (y + 2) + 'px');
setStyle(line, 'width', '70px');
153     setStyle(line, 'opacity', isFront ? frontT * 0.7 : 0);
154   });
155
156   dirty = false;
157   if (layoutLoop) {

```

```
158         if (!layoutLoop.desktop || !unchanged) layoutLoop.request();
159     } else requestAnimationFrame(layout);
160 }
161 layoutLoop = window.createDesktopAnimationLoop?.(container, layout);
162 if (layoutLoop) {
163     new ResizeObserver(() => {
164         height = container.clientHeight;
165         dirty = true;
166         layoutLoop.request();
167     }).observe(container);
168     layoutLoop.request();
169 } else layout();
170 }());
```

static/page-wheel.js | continued, lines 167-170

```
167     }).observe(container);
168     layoutLoop.request();
169 } else layout();
170 }());
```

A35 static/page2-narrative.js

Routes the narrative watch loop through the shared scheduler when available, retaining its existing fallback and entry timing.

Source: static/page2-narrative.js | 252 total lines | SHA-256 6ebe5d37d16869f4...

Unchanged source lines 1-204 omitted.

Source lines 205-215

```
205
206     let watching = false;
207     let cueHidden = false;
208     const watchLoop = window.createDesktopAnimationLoop?.(page2, watch);
209
210     function watch() {
211         if (!watching) return;
212         if (watchLoop) watchLoop.request(); else requestAnimationFrame(watch);
213         const i = frontIndex();
214         if (i < 0) return;
215         if (i !== rendered) swapTo(i);
```

Unchanged source lines 216-237 omitted.

Source lines 238-244

```
238         setTimeout(function () {
239             ticks.forEach(function (tick) { tick.style.transitionDelay = '0s'; });
240             watching = true;
241             if (watchLoop) watchLoop.request(); else requestAnimationFrame(watch);
242             }, reduceMotion ? 200 : 1000);
243
244             }, 6000);
```

Unchanged source lines 245-248 omitted.

Source lines 249-252

```
249         if (entry.isIntersecting) start();
250     });
251     }, { threshold: 0.28 }).observe(page2);
252 }());
```

A36 static/three.js

Connects 3D book and scroll-cue render loops to visibility-aware scheduling. The optional enabled predicate avoids rendering a fully hidden cue.

Source: static/three.js | 320 total lines | SHA-256 cf5222470054688a...

Unchanged source lines 1-228 omitted.

Source lines 229-239

```
229 // RENDER LOOP
230 let running = true;
231 let baseX = 2.3;
232 const renderLoop = window.createDesktopAnimationLoop?.(container, animate);
233
234 function animate(t) {
235   if (!running) return;
236   if (renderLoop) renderLoop.request(); else requestAnimationFrame(animate);
237   const e = t * 0.001;
238
239   if (auraMesh && introPlayed) {
```

Unchanged source lines 240-262 omitted.

Source lines 263-275

```
263
264 renderer.render(scene, camera);
265 }
266 if (renderLoop) renderLoop.request(); else requestAnimationFrame(animate);
267
268 if (!renderLoop) {
269   new IntersectionObserver(es => {
270     es.forEach(e => { running = e.isIntersecting; if (running) requestAnimationFrame(animate); });
271   }, { threshold: 0.05 }).observe(container);
272 }
273 }
274
275 function initScrollCueBook(sourceBookGroup) {
```

Unchanged source lines 276-307 omitted.

Source lines 308-320

```
308
309 cueScene.add(cueBook);
310
311 const cue = cueCanvas.closest('.scroll-cue');
312 const cueLoop = window.createDesktopAnimationLoop?.(cueCanvas, animateCue, () =>
  cue?.style.opacity !== '0');
313 if (cueLoop) window.requestScrollCueRender = cueLoop.request;
314 function animateCue(t) {
315   if (cueLoop) cueLoop.request(); else requestAnimationFrame(animateCue);
316   cueBook.rotation.y = t * 0.0005;
317   cueRenderer.render(cueScene, cueCamera);
318 }
319 if (cueLoop) cueLoop.request(); else requestAnimationFrame(animateCue);
320 }
```

A37 .gitignore

Patterns keep local credentials, caches, importer data/tooling, and test dependencies outside the tracked website. The current tests/ exclusion is a review finding: it also excludes the regression suite from the push.

Source: .gitignore | 11 total lines | SHA-256 9c14f2fe082e8570...

Unchanged source lines 1-2 omitted.

Source lines 3-11

```
3 static.zip
4 .env
5 .venv-test/
6 .pyq-test-deps/
7 pyq-import-report.json
8 admin only/manifest.json
9 admin only/pdfs/
10 admin only/
11 tests/
```

A38 COMMUNICATIONS.md

Operator-facing instructions describe the independent banner/inbox behaviours, recent presence, deployment schema changes, and mocked verification. Read "active" in broadcast eligibility as non-banned registered accounts.

Source: COMMUNICATIONS.md | 40 total lines | SHA-256 268409171cdd9389...

Source lines 1-40

```

1 # Official announcements and notifications
2
3 Open **Admin panel → Notices & notifications** (`/admin/communications`).
4
5 - Publish or edit the single official announcement at the top of the student dashboard. The
  title and message are required; a link is optional. Remove hides the bar and keeps the last text
  in the editor for reuse. Students see changes when they open or refresh the dashboard.
6 - Send a separate notification to all currently registered, active students, or search and
  select up to 500 students by name, email, URN or CRN. Administrator accounts and banned accounts
  are excluded. Selecting everyone captures the eligible students at send time; future
  registrations do not receive earlier messages.
7 - Students have a notification bell beside their Google profile photo, with an initial fallback
  when no photo is available. Existing signed-in sessions acquire the photo on their next Google
  sign-in. Notifications persist across sessions; students can mark one or all messages as read
  and load older messages.
8 - The unread count refreshes every 30 seconds while the dashboard is visible. Opening the bell
  fetches the latest messages. The send history shows the latest 50 sends with recipient and read
  counts.
9
10 These are in-site notifications, not email, operating-system notifications, or browser push.
  Publishing a banner does not automatically send a notification.
11
12 ## Active students
13
14 The admin overview includes an **Active now** count linked to the User database's active-student
  list. It refreshes every 30 seconds and lists the latest 50 students, with a total covering all
  active students. Click a name to find that account in the user database.
15
16 Active means a signed-in, non-banned student whose visible dashboard or study-resource page
  reported activity within five minutes. Pages report immediately and once a minute while visible;
  repeated reports within 30 seconds do not rewrite the database timestamp. Closing, hiding, or
  signing out naturally expires the student's activity within five minutes. This is approximate
  recent presence, not proof that the student is continuously interacting. Administrators are
  excluded.
17
18 Startup adds the nullable `users.last_seen_at` timestamp and its index. Historical accounts
  start with no recorded activity; presence failures display an unavailable state rather than zero
  active students.

```

COMMUNICATIONS.md | continued, lines 19-36

```

19
20 ## Deployment and storage
21
22 No new dependencies or environment credentials are required. Restart or redeploy the website
  normally. After the existing users schema initializes, startup creates three additive PostgreSQL
  tables in the same database:
23
24 | Table | Stores |
25 | --- | --- |
26 | `official_announcement` | One editable banner, visibility, revision and admin audit fields |
27 | `student_notifications` | Message content, audience, sender, time and a unique send key |

```

```
28 | `student_notification_recipients` | Recipient membership and each student's read time |
29
30 Startup publishes no banner and sends no notifications. Initialization is repeatable and
  preserves existing data. The standalone `admin only` importer does not use these tables or
  import this feature.
31
32 Admin mutations require admin authentication and CSRF protection. Student endpoints resolve the
  recipient from the authenticated account, never a client-supplied user ID. Messages are plain
  text; links accept HTTP(S) URLs or internal paths. A single transaction stores each notification
  and its recipients. A unique form key makes retrying an uncertain send safe. Stale announcement
  edits are rejected using the saved revision.
33
34 If schema initialization fails, check the website startup logs and the configured database
  user's create-table permissions before restarting. The admin workspace reports unavailability
  rather than pretending to publish or send.
35
36 ## Verification
```

COMMUNICATIONS.md | continued, lines 37-40

```
37
38 With the website dependencies installed, run `python -m unittest discover -s tests -p
  "test_*.py"` from the repository root. Tests mock database and cloud calls; they never send real
  notifications. Coverage includes admin and student access controls, CSRF, announcement
  publishing/removal, text escaping, recipient selection, duplicate sends, transaction rollback,
  private read state and pagination.
39
40 Browser verification should use local fixtures to check the bell, unread count, individual/all
  read actions, recipient selection across searches, empty/error states, keyboard dismissal,
  profile-photo fallback, and desktop/mobile layouts.
```

A39 static/images/ce-icon.svg

Local branch SVG icon. viewBox establishes scalable coordinates, fill controls interiors, stroke draws the lines, and path/shape commands define the recognizable branch symbol.

Source: static/images/ce-icon.svg | 3 total lines | SHA-256 7fc9140234c893f8...

Source lines 1-3

```
1 <svg xmlns="http://www.w3.org/2000/svg" width="32" height="32" viewBox="0 0 256 256" fill="none"  
  stroke="#000000" stroke-width="16" stroke-linecap="round" stroke-linejoin="round">  
2   <path d="M16 168h224M56 216V40m144 176V40M16 112c16 0 28-24 40-48 40 88 104 88 144 0 12 24 24  
  48 40 48M88 112v56m40-32v32m40-56v56"/>  
3 </svg>
```

A40 static/images/ece-icon.svg

Local branch SVG icon. viewBox establishes scalable coordinates, fill controls interiors, stroke draws the lines, and path/shape commands define the recognizable branch symbol.

Source: static/images/ece-icon.svg | 8 total lines | SHA-256 3a184f211ed3703a...

Source lines 1-8

```
1 <svg xmlns="http://www.w3.org/2000/svg" width="32" height="32" viewBox="0 0 256 256" fill="none"
  stroke="#000000" stroke-width="16" stroke-linecap="round" stroke-linejoin="round">
2   <rect x="88" y="88" width="80" height="80" rx="8"/>
3   <path d="M112 88V48H64M144 88V24M168 112H40V64M168 144H64M144 168V40H48M112 168V64M88
  144H48V48M88 112H24"/>
4   <circle cx="48" cy="48" r="16"/>
5   <circle cx="208" cy="48" r="16"/>
6   <circle cx="208" cy="208" r="16"/>
7   <circle cx="48" cy="208" r="16"/>
8 </svg>
```

A41 static/images/ee-icon.svg

Local branch SVG icon. viewBox establishes scalable coordinates, fill controls interiors, stroke draws the lines, and path/shape commands define the recognizable branch symbol.

Source: static/images/ee-icon.svg | 3 total lines | SHA-256 373cb33996563abe...

Source lines 1-3

```
1 <svg xmlns="http://www.w3.org/2000/svg" width="32" height="32" viewBox="0 0 256 256" fill="none"
  stroke="#000000" stroke-width="16" stroke-linecap="round" stroke-linejoin="round">
2   <path d="M144 24 48 144h72l-8 88 96-120h-72l8-88Z"/>
3 </svg>
```

A42 static/images/me-icon.svg

Local branch SVG icon. viewBox establishes scalable coordinates, fill controls interiors, stroke draws the lines, and path/shape commands define the recognizable branch symbol.

Source: static/images/me-icon.svg | 4 total lines | SHA-256 83a9960b045d408c...

Source lines 1-4

```
1 <svg xmlns="http://www.w3.org/2000/svg" width="32" height="32" viewBox="0 0 256 256" fill="none"  
  stroke="#000000" stroke-width="16" stroke-linecap="round" stroke-linejoin="round">  
2   <path d="m104 24 48 0 6 28 16 9 27-9 24 40-21 20v32l21 20-24 40-27-9-16 9-6  
  28h-48l-6-28-16-9-27 9-24-40 21-20v-32l31 92l24-40 27 9 16-9 6-28Z"/>  
3   <circle cx="128" cy="128" r="36"/>  
4 </svg>
```

Verification record and source coverage

The source atlas was assembled directly from `git diff HEAD^ HEAD` and checked to include every added/replacement line in each changed text file. The document generator does not import `app.py` and did not execute production schema or cloud operations. A source digest is shown for each file so this guide can be tied to the exact reviewed snapshot.

Check	Recorded result
Regression suite	58 tests passed during the preceding review; database and Azure calls mocked.
Syntax	Tracked Python and JavaScript syntax checks passed.
Credential check	No pattern matches in scanned tracked text; no tracked <code>.env</code> . Not a full historical secret audit.
UI checks	Admin desktop/mobile fixtures, dark mode, navigation, expanded warnings, keyboard search, award selection, active-list toggle.
Known issues	Blob-first deletion, public uploader emails, missing Secure cookie flag, ignored tests. Reported, not repaired.
Live deployment	No live migration test, cloud upload, production notification send, or push performed for this guide.
Source coverage	42 changed files; 2,239 added/replacement source lines covered.

Read the source and the explanation together. When the website changes, regenerate or update this guide; do not rely on old line numbers as proof of current behaviour. The highest-priority learning path is to trace one selected send, one retry, one private read, and one failed deletion without involving live data.